# AI-Assisted Domain-Driven Mainframe Modernization

## A PATTERN CATALOG FROM PROJECT ROSETTA

PIERRE MILET

# AI-Assisted Domain-Driven Mainframe Modernization

*A Pattern Catalog from Project Rosetta*

Pierre Milet Llobet

LegacyLabs

*Twenty-eight patterns where Domain-Driven Design*
*meets blackfield mainframe modernization*
*some validated inside the Rosetta prototype, some in active construction,*
*some projected from validated principles.*

First edition, 2026

Published as a research prototype companion document.

Patterns marked *working* have been validated inside the Rosetta prototype.
Patterns marked *in progress* are in active development.
Patterns marked *next* are designed but not yet built.

# Contents

# AI-Assisted Domain-Driven Mainframe Modernization

# A Pattern Catalog from Project Rosetta

*Twenty-eight patterns where Domain-Driven Design meets blackfield mainframe modernization.*

---

# How to read this catalog

This catalog documents twenty-eight patterns I've encountered building Project Rosetta, a research prototype for AI-assisted modernization of COBOL/CICS mainframe systems. It follows the structure popularised in software engineering by the Gang of Four's *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994) — each pattern named, situated in its context, articulated as a solution to a recurring problem with its forces, consequences, and relationships to other patterns. The form is established; what's specific here is the territory.

The name *Project Rosetta* is deliberate. The Rosetta Stone made ancient Egyptian readable by presenting the same decree in three scripts — hieroglyphic, demotic, Greek — and allowing meaning to be recovered through triangulation between representations. Project Rosetta does the same with legacy mainframe systems: the COBOL source, the structural graph (Pattern 3), the domain ontology (Pattern 5), and the intermediate representation (Pattern 9) are different representations of the same system, and meaning is recovered through triangulation between them. The legacy is not abandoned and rewritten; it is read, understood, and translated — with the running system as oracle (Pattern 2) and the substrates as the parallel inscriptions that let understanding survive the translation.

The catalog has a thesis: **AI-assisted mainframe modernization is, at its core, a Domain-Driven Design activity at scale**. The scale is what makes the work hard — and what makes AI assistance necessary. Strategic design (recovering the domain, identifying bounded contexts, establishing ubiquitous language) and tactical design (modelling aggregates, domain events, handlers) are the work the modernization actually performs. AI agents accelerate the mechanical parts; humans direct the strategic ones; the boundary between agentic and human work is itself an architectural commitment that needs explicit design. These are the patterns I've discovered while building Project Rosetta — the ones I think make sense for treating true blackfield legacy modernization through a DDD lens. Some are validated; some are in active construction; some are projected from validated principles. None is definitive. Other practitioners will find other patterns. This is mine. And one operational note: fewer patterns, denser articulation. More is not better. Less is better.

DDD has been applied extensively to greenfield development and to incremental refactoring of in-life systems. It has been applied less systematically to the territory where it is most needed: legacy mainframe systems decades old, written in languages whose original specifications were never written down, where the domain has drifted from whatever it once was and accumulated three definitions of every important concept. *Brownfield* is the established term for software work on existing systems; this catalog uses **blackfield** for the harder case — systems where the original engineers have moved on, the documentation is lost, and the domain knowledge has decayed into operational dialect, fragmented across modules. Brownfield is "existing systems you understand." Blackfield is "existing systems where understanding itself has to be recovered." Mainframe

modernization is blackfield at its hardest: COBOL written in the eighties and nineties, CICS transactions evolved across decades, JCL and copybooks woven through millions of lines of code that the original team has long since left. AI assistance changes the economics of this work — what was previously infeasible at scale becomes tractable. This catalog is a contribution to that practice.

The patterns are calibrated to mainframe COBOL/CICS modernization, where Rosetta has been validated. Many patterns generalise — the compiler principle, the IR contract, twin verification, the harness state machine, the cockpit — apply to any AI-assisted modernization where deterministic and probabilistic work need clear separation. Other patterns are mainframe-specific in their implementation — the linguistic cues for slice discovery (`XCTL`, `START TRANSID`, `EXEC CICS LINK`), the Raincode-compiled COBOL container as oracle, the CICS pseudo-conversational boundaries — and would need adaptation for other legacy stacks (PL/I, RPG, Adabas Natural, Java legacy). Each pattern body indicates where the underlying principle is universal versus where the implementation is mainframe-specific.

The catalog is a mix of three kinds of patterns. **Original patterns** describe practices that emerged from building Project Rosetta and that the field has not yet articulated as patterns in their own right — the compiler principle for agentic workflows, the intermediate representation as contract between analysis and generation, the heuristic catalog as queryable infrastructure, architecture documentation as pluggable emitter. **Adapted patterns** apply established practices to mainframe modernization with explicit recalibration — Martin Fowler's strangler fig at bounded context granularity, Eric Evans' anti-corruption layer as migration infrastructure, the modular monolith as transitional architecture. **DDD re-articulations** recast canonical DDD patterns through the lens of legacy mainframe modernization — the domain ontology as substrate, commands and events as logical boundaries, source provenance as audit infrastructure. Each pattern indicates its lineage explicitly: where the principle originates, what this catalog adds, what's calibrated to mainframe specifically. The catalog stands on shoulders — Eric Evans, Vaughn Vernon, Martin Fowler, Sam Newman, Alberto Brandolini, Cyrille Martraire, Birgitta Böckeler, Charity Majors, Nick Tune, Uberto Barbini, Kent Beck — and names them as it builds.

I write for two audiences. **DDD practitioners** will recognise vocabulary they already use — bounded contexts, ubiquitous language, subdomain types, aggregates, domain events, anti-corruption layers — and find them deployed in a domain DDD has rarely entered: mainframe modernization at decade scale. **Mainframe modernization practitioners** less familiar with DDD will encounter the vocabulary deliberately. Where a DDD concept appears for the first time, I provide a brief inline gloss; the glossary at the end gives more complete definitions and pointers to canonical sources. The catalog rewards readers from either community — though differently.

Of the twenty-eight patterns, fifteen are validated inside the Rosetta prototype (status: *working*). Eight are in active construction (status: *in progress*). Four are designed from validated principles but not yet built (status: *next*). None has yet been validated against real customer engagements — that's the next phase, not yet started. I include all three categories deliberately. This catalog records the same kind of evidence traditional pattern catalogs do, but explicitly: each pattern carries a marker showing whether the principle has been validated, is in construction, or is projected from validated foundations. The mix is intentional. The markers are honest. The reader who wants only validated patterns can filter by status; the reader interested in the architectural reasoning can engage with all of them.

Each pattern follows the same shape. **Status** says where the pattern stands in the prototype today. **Context** describes the situation in which the pattern applies. **Problem** describes the tension or

difficulty the pattern addresses. **Forces** describes the factors in conflict. **Pattern** is the solution, articulated as principle. **Consequences** describes what applying the pattern produces — both gains and costs. **Related patterns** points to other entries that depend on, support, or are supported by this one.

The patterns are grouped into four parts that follow the structure of DDD as discipline:

- **Part I — Strategic Recovery.** What is the domain, what bounded contexts exist, what is the ubiquitous language. The patterns that recover what the legacy *is* and what it *should be about*.
- **Part II — Tactical Generation.** How each bounded context materialises in modern code. Aggregates, domain events, handlers, scaffolds matched to subdomain type.
- **Part III — Verification.** Validating that strategic and tactical designs preserve behavioural equivalence to the legacy.
- **Part IV — Governance.** Governing the agentic system that performs the recovery, generation, and verification work.

The grouping is for navigation — patterns interact across groups, and the *Related patterns* sections trace those interactions.

A short section names seven antipatterns this catalog is built against. They're not bad practices in the abstract; they're failure modes the field has encountered, and naming them helps clarify what the patterns are correcting.

After the antipatterns, a glossary defines the DDD and modernization terms used throughout the catalog. After that, a final section maps each pattern to the concrete technology that realises it in Rosetta today. The pattern bodies stay abstract because principles outlive implementations. The reference section names what's currently doing the work, snapshot in time.

I came to mainframes as an outsider and have spent 15 years on mainframe modernization projects, with a foundation in DDD practice for .NET work. I learned the COBOL/CICS world the slow way — through migrations done before AI assistance was practical, when modernization was a matter of careful manual work. Project Rosetta is the experiment in what becomes possible when AI assistance changes that economics. The patterns here are reports from inside the experiment, not claims about a finished system.

This is the long version of what I've been writing about on LinkedIn under the LegacyLabs name. The shorter posts and newsletter there were the introduction. This is the working catalog.

---

# Part I: Strategic Recovery

---

**[ILLUSTRATION] P1 — *Strategic Recovery: archaeology of the running system***

*Style: editorial illustration, drawn or watercolour quality, in keeping with the cover's earth-tone palette. Subject: an archaeologist's brush gently uncovering a fragment of stone with overlapping inscriptions in different scripts (cuneiform, hieroglyphic, modern code) — visual metaphor for recovering what the legacy is and what the domain it implements is. The overlapping scripts hint at the ontological drift the patterns address. Format: full-page or two-thirds page opener for Part I.*

---

The patterns in this group address the strategic question that opens any DDD engagement: *what is the domain, and what bounded contexts compose it?* In a greenfield context, strategic design starts from a clean slate; in legacy modernization, it starts from a system that has been running for decades and has its own answers — partial, contradictory, and frequently undocumented. The patterns here recover those answers from the legacy as evidence, and ground them against canonical domain understanding that the legacy alone cannot provide.

Without strategic recovery, generation has nothing to work from and verification has nothing to compare against. Without honest distinction between behavioural recovery (what the legacy does) and ontological grounding (what the domain really is), strategic recovery silently inherits the legacy's accumulated drift.

---

# Pattern 1: Business-Aligned Capability Strategy

*Status: working.*

## Context

A mainframe modernization engagement that touches a system spanning decades of accumulated business logic — millions of lines of COBOL, hundreds of CICS transactions, dozens of bounded contexts implementing capabilities the business depends on. Some of these capabilities are strategic differentiators; some are commodity work; some are legacy debt that exists only because no one has authorised retiring it. The modernization team must decide what to modernize, how deeply, and what to leave alone — and these decisions must precede any technical work.

The default frame for modernization is technical: convert COBOL to C#, move from mainframe to cloud, declare victory. This frame skips the question that determines whether the modernization delivers value: which business capabilities deserve which treatment, and why.

## Problem

The dominant antipattern in the field is technical-first modernization. Teams treat the legacy code as if it were the specification — every paragraph must be migrated, every transaction must be preserved, every batch job must be replatformed. The result is a brilliant technical transformation that costs millions and delivers a modernized system functionally identical to the legacy. The business strategy isn't served; opportunities are missed; capabilities that should have been retired survive into the modern system, carrying their maintenance burden forward.

The deeper failure is treating all capabilities as if they were equivalent. A small percentage of transactions typically represents 80% of the value, the traffic, the risk, the maintenance cost — the Pareto distribution applies almost everywhere in mainframe systems. Treating all capabilities with the same modernization discipline produces over-investment in commodity work and under-investment in core. Sometimes the highest-ROI decision is *not* to modernize at all — to retire a capability, replace it with SaaS, or simply turn it off — and a technical-first modernization never surfaces these options because it never asks the question.

## Forces

The business has a strategy: differentiate where it matters, achieve parity where it doesn't, retire what no longer earns its place. The legacy mainframe encodes a snapshot of past business decisions, frozen in COBOL since the eighties or nineties. The mismatch between current strategy and encoded snapshot is the territory modernization must navigate. Done well, the modernization closes the gap. Done poorly, it preserves the snapshot in modern syntax.

Investment must be distributed unevenly. Treating all bounded contexts as equal squanders resources in commodity work and starves strategic core. But the distribution cannot be improvised — it must be grounded in evidence about each capability's role, value, volatility, and cost.

## Pattern

Before any technical work begins — before identifying feature slices to extract, before deciding which contexts deserve which architecture, before generating any scaffold — map the business capabilities the legacy implements and classify each along four dimensions:

- **Strategic value** — is this capability a differentiator (something the business does better than competitors, where investment compounds) or commodity (something everyone in the industry does, where parity is sufficient)?
- **Functional volatility** — does the business logic for this capability change frequently (regulatory updates, market shifts, product evolution) or has it been stable for years?
- **Operational criticality** — what is the cost of failure? A real-time payment authoriser failing is catastrophic; an internal reporting batch failing is recoverable.
- **Current maintenance cost** — how much does this capability cost to maintain today, in developer hours, infrastructure, and operational incidents?

The cross-product of these dimensions produces five distinct modernization strategies — not one playbook:

- **Core differentiator + volatile** → cloud-native rewrite, deep DDD investment, full hexagonal architecture, dedicated team. This is where the modernization must excel.
- **Core differentiator + stable** → AI-assisted migration with semantic preservation. The behaviour matters; the architecture can stay conservative. Twin Verification (Pattern 13) is critical here.
- **Commodity + stable** → lift & shift, or replace with SaaS / managed service. Investment beyond parity is waste.
- **Commodity + high maintenance** → retire, consolidate with another capability, or externalize. The maintenance cost itself signals that this capability is overdue for elimination.
- **Obsolete / no real usage** → turn off. This is more frequent than expected. Systems running for decades accumulate code paths that no one exercises but no one has authorised retiring. Identifying and shutting these down is often the highest-ROI work of the modernization.

The classification is not done by the technical team alone. It requires business stakeholders — product owners, capability leads, finance partners — collaborating with architects to map the capabilities, validate the four-dimensional assessment, and agree on the strategy per cell. Workshop techniques apply: Event Storming (Brandolini) for capability discovery, Wardley Mapping for value/commodity classification, Domain Storytelling (Hofer & Schwentner) for behavioural understanding.

The output is a capability map: every business capability in scope, with its four-dimensional profile, its chosen modernization strategy, and the rationale that connects business strategy to technical decision. This map is the primary input to every subsequent pattern. Slice discovery (Pattern 7) derives slices within capabilities, prioritised by strategic value. Tier-aware scaffolding (Pattern 10) operationalises the chosen strategy per bounded context. Rollout and cutover (Pattern 27) sequences the migration to deliver business value early.

## Consequences

Modernization investment becomes proportionate to business value. The team spends deep effort where it matters and light effort where it doesn't. Capabilities that should be retired are identified and shut down rather than migrated by default. The modernization delivers business value, not just technical transformation.

The capability map becomes a strategic artifact in its own right. Updated as business strategy evolves, it provides a continuous reference for "what are we doing and why." Future modernizations of adjacent systems have a precedent to follow. Business stakeholders have a shared vocabulary with the technical team for discussing what's being built and what isn't.

The cost is the discipline of doing the strategic work before the technical work. Many teams skip this because it feels slower than starting to code. The teams that skip it pay later — in over-engineered commodity work, in under-engineered core domain, in migrated capabilities that should have been retired. The work pays back; it must be done.

The capability map also exposes uncomfortable truths. Some capabilities that the business has been treating as strategic differentiators turn out to be commodity. Some that have been ignored as commodity turn out to be where genuine differentiation lives. The four-dimensional classification surfaces these mismatches and forces conversations the organization may have been avoiding.

The notion that *modernization is not code translation — it is reconstructing the business, segmenting capabilities by value and volatility, and designing the transition path* is what this pattern operationalises. Wardley Mapping (Wardley, 2018) provides the conceptual foundation for the strategic-value dimension. Eric Evans' subdomain classification (Evans, 2003) — core, supporting, generic — is the canonical DDD anchor; this pattern extends Evans by adding the volatility, criticality, and maintenance-cost dimensions, and by making *retirement* an explicit option, which standard DDD treatments do not.

## Related patterns

Pattern 7 (*Vertical Slice Discovery*) operates within the capabilities this pattern maps — slices are derived only after capabilities are classified. Pattern 10 (*Tier-Aware Scaffolding*) operationalises the strategy per bounded context: core differentiator → full hexagonal; commodity → vertical slice. Pattern 5 (*Domain Ontology as Independent Substrate*) draws its vocabulary primarily from the capabilities this pattern identifies. Pattern 27 (*Rollout and Cutover at Bounded Context Granularity*) sequences the migration in capability-priority order. The *Frozen Architecture* antipattern names what happens when modernization proceeds without this strategic framing — the legacy's accidental architectural decisions are preserved into the modern system because no one questioned whether they should be.

# Pattern 2: The Legacy as Oracle

*Status: working.*

## Context

A legacy system that has been running in production for years or decades, processing real business workloads. The original specifications are typically lost, outdated, or never existed in written form. Tribal knowledge has moved with the people who built the system. The modernization effort needs ground truth about what the system does — the behavioural foundation on which any DDD strategic recovery has to stand.

## Problem

Modernization platforms treat the legacy system as a starting point: something to be read, understood, eventually replaced. Once enough understanding has been extracted, the legacy fades to background. The modernization effort moves to the new system. Verification then becomes a question of "does the new system pass the tests we wrote." But the tests had to come from somewhere — and where they came from is the original system the team is trying to replace, mediated through human interpretation.

The result is that the modernization grades its own homework. With manual migrations, the team's misunderstandings shape both the new code and the tests for it. With AI-assisted migrations, the agents internalise the same misunderstandings whether they're generating code or generating tests. Either way, the verification is contaminated by interpretation.

## Forces

The legacy must eventually be decommissioned, which makes the instinct to plan around its absence reasonable. But during modernization the legacy is the most reliable source of truth available — it has been producing correct outputs every day for thirty years. Building a parallel ground truth (specifications, behavioural models, test suites) is expensive and lossy. Yet running the legacy continuously throughout the modernization is operationally complex.

## Pattern

Treat the running legacy as the live oracle for behavioural verification. Instrument it so it can be queried, observed, and compared against. Package it for local execution where possible — the Raincode-compiled COBOL container (Raincode compiles COBOL to .NET IL, packageable as Docker) is one realisation of this. Make it available to the inner loop of the agentic workflow, not just to humans during review.

The legacy is more useful running than off. Decommissioning is the last step, not the first frame.

A precision worth stating explicitly: *the legacy is a behavioural oracle, not an ontological one*. The legacy reliably tells you what the system does. It does not reliably tell you what the domain really is. A system that has been running for thirty years has often accumulated ontological drift — three definitions of "active customer" across billing, support, and product modules; two incompatible interpretations of "balance" depending on which paragraph computes it; vocabulary that meant one thing in 1992 and another by 2008. Behavioural equivalence to such a legacy preserves the drift. The legacy answers *what happens*; the modernization team still has to answer *what should be true* about the domain.

> *For the complementary recovery this pattern does not perform — establishing what the domain itself is, independent of what the legacy happens to do — see Pattern 5: Domain Ontology as Independent Substrate.*

---

**[FIGURE] 2.1 — *The legacy as oracle: inner-loop architecture***

*Diagram showing the agentic inner loop with the legacy as live verification target. Components to depict: (a) the agent generating a candidate C# translation of a COBOL paragraph; (b) the Raincode-compiled COBOL legacy running in a Docker container locally; (c) parallel execution of both candidate and legacy against the same input; (d) semantic comparison of outputs; (e) verdict returning to the agent within milliseconds. Arrows indicate data flow. Annotation at the legacy container reads "behavioural oracle — what the system does." A second annotation near the agent reads "ontology lives elsewhere — see Pattern 5." Style: clean architectural diagram, accent color for the verdict path, labels in serif. Suggested width: full page or two-thirds page.*

---

## Consequences

The agents iterate against evidence rather than against assumptions. Verification becomes part of the inner loop instead of a downstream activity. The cost of being wrong drops sharply, which lets the agents explore more aggressively. The agents converge faster and more honestly because they're matching behaviour the system actually exhibits, not behaviour someone wrote down and trusted.

The cost is structural. The legacy must be packageable for local execution, which requires tooling that compiles the legacy runtime to a portable form (in the COBOL/CICS case, Raincode does this; for other legacy stacks the equivalent tooling may or may not exist). The legacy must remain observable throughout the modernization, which means the modernization platform has to integrate with it operationally, not just textually.

There is also a cost that the pattern alone does not pay: behavioural fidelity is necessary but not sufficient. The complementary discipline lives in Pattern 5 and the *Behavioural Equivalence Without Ontology* antipattern; without them, this pattern protects against the wrong thing.

I have only tested this pattern for COBOL/CICS. For that case, it has earned its place in Rosetta. The principle that legacy systems are more useful running than off is what generalises; the specific implementation is calibrated to CICS.

The notion of an "oracle" in software verification has a long history — William Howden formalised it in testing theory (Howden, 1978) as the source of authoritative answers against which test outputs are compared. This catalog applies the same notion at modernization scale: the legacy system itself, running, is the oracle. What is original here is not the concept of an oracle, but treating the running legacy as the oracle in the *agentic inner loop* — comparing candidate translations against legacy execution at millisecond latency, rather than against test suites the modernization team itself wrote.

## Related patterns

Pattern 5 (*Domain Ontology as Independent Substrate*) is the complementary recovery: the legacy is behavioural oracle, but ontology requires independent grounding. Pattern 16 (*Twin Verification*) is the operationalisation of this principle in the inner loop. Pattern 17 (*Hypothesis-Driven Verification*) extends it from dev mode to production mode. Without Pattern 2, neither of those is implementable.

---

# Pattern 3: The Graph as Projection

*Status: working.*

## Context

A legacy codebase consisting of thousands of source files in a language that LLMs read poorly. The architectural concepts that matter — bounded contexts (DDD's term for explicit boundaries within which a particular domain model applies), candidate slices for modernization, the side-effect surfaces of programs — are not stated explicitly anywhere. They must be derived from structural relationships within the code.

## Problem

Asking an LLM to reason about COBOL directly produces poor results. COBOL is verbose, full of historical idioms, and structurally unlike anything in the modern training corpus. The LLM pattern-matches to surface features and misses architectural intent. Multiple passes don't help because the underlying obstacle is representational: the LLM is trying to extract architecture from a representation that obscures it.

## Forces

Architectural recovery requires understanding relationships across the codebase: which programs call which, which data structures flow through which paragraphs, which CICS commands access which resources. This information exists in the source but is distributed and implicit. Capturing it as a queryable representation is expensive but unlocks downstream analysis. Doing so naively (as raw abstract syntax tree, capturing every token) produces a representation too large to query usefully and too detailed to be informative.

## Pattern

Parse the legacy source into a property graph. Capture programs, paragraphs, data structures, control flow, side effects, predicates, and entry points as nodes. Connect them with typed edges. Make the graph the queryable architectural projection of the source.

Keep ingestion deterministic and idempotent. Two runs over the same source must produce the same graph. Put semantic interpretations (bounded contexts, slice candidates, discriminator fields)

in a separate derived layer computed by analysis passes. The agents reason about the graph, not the source.

---

**[FIGURE] 3.1 — *The structural property graph schema for COBOL/CICS***

*Diagram showing the graph schema as a node-and-edge model. Nodes to depict: Program, Paragraph, DataStructure, CICSCommand, Resource, EntryPoint, Predicate. Edges to depict: CONTAINS (Program → Paragraph), USES (Paragraph → DataStructure), INVOKES (Paragraph → CICSCommand), ACCESSES (CICSCommand → Resource), PERFORMS (Paragraph → Paragraph), CALLS (Program → Program). Annotations near key edges explain semantic meaning. Style: clean ER-style diagram with colored node types, accent color for entry points and side-effect surfaces. Suggested width: full page. Goal: reader can see the schema at a glance and understand what graph queries can express.*

---

## Consequences

The agents reason about something LLMs handle well — graph relationships, named concepts, queryable structure — while the underlying COBOL stays in its place as the source of truth. Architectural concepts emerge through analysis rather than through interpretation. Community detection surfaces bounded contexts. Entry-point reachability and pseudo-conversational boundaries surface candidate slices for modernization. Predicate analysis at low depth surfaces discriminator fields. Each technique becomes a reusable lens over the graph.

The cost is the ingestion pipeline. Building a parser that captures the right level of structure (summarised, not full AST) requires careful schema design. Schema versioning becomes a discipline because the graph is the long-lived artifact; ingestion can be re-run, but the graph schema needs to evolve coherently.

Anthony Alcaraz has written about agentic GraphRAG as a general pattern. This catalog applies it to mainframe spec recovery, where the legacy's representational distance from modern code makes the graph projection essential. The intellectual lineage extends to program comprehension research (Storey 2005; Müller and Klashinsky's earlier work on software architecture recovery) which has long argued that source code alone is insufficient for understanding large legacy systems — derived representations are necessary. What this catalog adds is the framing of the graph as substrate for *agentic* reasoning specifically, not just for human comprehension.

## Related patterns

Pattern 4 (*Source Provenance Discipline*) is what makes the graph trustworthy as a foundation for downstream work. Pattern 5 (*Domain Ontology as Independent Substrate*) is what the structural graph alone cannot provide — recovering what entities exist in the domain and how they relate, beyond what the legacy artifacts encode. Pattern 6 (*The Graph and the Index as Complementary Substrates*) addresses how structural and semantic representations coexist. Pattern 9 (*The Intermediate Representation*) is how the graph crosses over into deterministic generation.

---

# Pattern 4: Source Provenance Discipline

*Status: working.*

## Context

A modernization pipeline that derives multiple representations from the original source: a graph (Pattern 3), an intermediate representation (Pattern 9), a generated C# scaffold, a translated handler body, a semantic index entry (Pattern 6). Reviewers and operators eventually need to trace any given artifact back to the source it derives from. Audit trails, diff views, debugging in production, regulatory compliance review — all depend on this. In regulated industries, traceability is not nice-to-have; it is what makes the modernization auditable.

The context also includes agentic systems that reason over derived representations. Agents propose translations, generate test cases, identify slice boundaries — and each proposal carries evidence drawn from the derived representations. Without provenance, the evidence cannot be defended back to source, and the agent's reasoning becomes opaque the moment any artifact crosses a transformation boundary.

## Problem

Without source coordinates carried through every layer of the pipeline, traceability breaks at the first transformation. A generated handler exists; the COBOL paragraph it derived from is somewhere; nobody knows the connection without re-deriving it. This may seem acceptable during development but becomes operationally untenable in production.

The failure modes are operationally severe. A bug surfaces in production: which legacy paragraph implements this behaviour? Without provenance, the team re-derives the answer from scratch — re-reading legacy code, re-running grep against thousands of files, hoping to find the right one. A regulator asks how a particular decision was reached: without provenance, the answer is "the agent decided" rather than "the agent reasoned from these specific lines of COBOL with these specific transformations applied." A behavioural test fails: without provenance, distinguishing real divergence from artifact requires manual investigation rather than automated cross-reference.

## Forces

Carrying source coordinates through every transformation adds discipline to the schema design. Every node in every representation must include file path, start line, end line. Every transformation

must preserve them. This is a tax paid during ingestion and analysis — every developer working on a new analysis pass must remember to thread provenance through, every emitter must carry it forward, every transformation must validate it.

The benefit is invisible until something goes wrong — at which point the absence of provenance is catastrophic. By the time you need it, it's too late to add. This temporal asymmetry is what makes provenance discipline so frequently neglected: the cost is paid daily by everyone building the pipeline; the benefit accrues to a future operator who isn't in the room when the schema is designed.

## Pattern

Make source coordinates a non-negotiable part of every representation in the pipeline. Every graph node carries source coordinates (`source_file`, `start_line`, `end_line` in Rosetta — naming is implementation-specific, the discipline is not). Every IR element carries the same, plus references to the graph nodes it derived from. Every generated C# artifact carries comments or metadata pointing back to the IR element that produced it.

Treat this as infrastructure, not as nice-to-have. The provenance enables every downstream capability that requires traceability: diff views in the cockpit, audit trails for compliance, reverse-lookup from a production exception in the C# back to the original COBOL paragraph. Schema validators enforce provenance presence — a graph node without source coordinates is rejected at ingestion, an IR element without graph references is rejected at construction, a generated artifact without metadata pointers fails the build.

This discipline extends to the semantic index (Pattern 6). Every vector entry must carry the same source coordinates as the graph nodes it derives from. When the agent queries the index for "paragraphs semantically similar to this," each match traces back to specific COBOL lines. Without this, semantic relationships become evidence the agent uses but cannot defend — and the audit trail breaks at the index boundary.

The discipline extends further to agent reasoning records (Pattern 24). Every agent decision cites the substrates it consulted; those citations carry provenance forward. A modernization decision can be traced from the agent's reasoning back through the substrates the agent queried back to the specific legacy code those substrates derived from. The audit trail is end-to-end.

## Consequences

The cockpit (Pattern 25) can show side-by-side comparisons between original COBOL and generated C# at any granularity — paragraph, slice, bounded context, whole system. Audit trails are reproducible: given any artifact and any moment in the pipeline, the reviewer can reconstruct what evidence the system had when it produced that artifact. Production debugging traces from the modernized C# back through the IR back through the graph back to the source COBOL. Compliance reviewers can verify any decision the system made — agent or human — by following the provenance chain.

The cost is schema rigour. Provenance fields can't be optional, can't be approximate, can't be lost in transformations. The pipeline becomes more expensive to build but more durable to operate.

Engineers building new analysis passes must thread provenance through every step; emitters must preserve it; validators must enforce it. The discipline is mechanical and tedious — and necessary.

This pattern looks like good practice. It's actually infrastructure. Without it, every other capability that depends on traceability collapses. With it, those capabilities become tractable.

Provenance as discipline is well-established in compilers (debug symbols, source maps), build systems (Bazel's deterministic dependency tracking), and audit infrastructure (signed transaction logs in regulated systems). What this catalog contributes is the framing of provenance as architectural commitment of an *agentic modernization pipeline* — every representation the agents reason over carries source coordinates, not just the artifacts humans inspect. The discipline has to extend into the substrates the agents themselves use, or the audit trail breaks at the agent boundary.

## Related patterns

Pattern 3 (*The Graph as Projection*) is where source coordinates first appear — the graph schema is where provenance becomes mandatory. Pattern 6 (*The Graph and the Index as Complementary Substrates*) is where the discipline extends to the semantic index. Pattern 9 (*The Intermediate Representation*) extends the discipline into the IR — every IR element references graph nodes by source coordinates. Pattern 24 (*Reasoning Telemetry as First-Class Output*) is where agent decisions cite the provenance of their evidence. Pattern 25 (*The Cockpit*) is the operational surface that consumes the provenance, surfacing side-by-side comparisons and audit traces to humans.

---

# Pattern 5: Domain Ontology as Independent Substrate

*Status: next.*

## Context

A modernization with structural graph (Pattern 3) and source provenance (Pattern 4) in place. The legacy is being treated as behavioural oracle (Pattern 2). The agents are reasoning over real artifacts of the legacy system. But the modernization team eventually has to answer a question that the legacy alone cannot answer: *what is the domain this system is supposed to be about*. In DDD vocabulary, this is the question of establishing the **ubiquitous language** — the shared vocabulary of the domain that the team and the system both speak — and articulating the **strategic design** that organises bounded contexts within that language.

## Problem

A system that has been running for years or decades has accumulated ontological drift — the kind already named in Pattern 2: three definitions of "active customer," two interpretations of "balance," vocabulary that meant one thing in 1992 and another by 2008. The legacy faithfully implements all of these, often in tension with each other, often without anyone noticing. In DDD terms, the ubiquitous language has been lost or never established; what survived is operational dialect, fragmented across modules.

Modernizing such a legacy through structural and behavioural fidelity alone preserves the drift. The agents reason over the schema, not over what the schema means. The pattern can be flawless and the output will still be confidently wrong — not because the translation is incorrect, but because the modernization has inherited the legacy's confusion about what the domain really is. Anthony Alcaraz has argued this point for agentic systems generally: orchestration patterns are downstream of ontological grounding, the pattern is interchangeable, the ontological substrate is not. The argument lands with particular force in legacy modernization, where the substrate has had decades to drift.

## Forces

Behavioural recovery (Patterns 1, 2, 3) is necessary. It is also tractable: the legacy is right there, observable, queryable, runnable. Ontological recovery is harder. The domain the legacy was sup-

posed to be about is not the same as what the legacy actually became. Recovering ontology requires sources the legacy alone does not provide — domain experts, business artifacts, regulatory definitions, the conversations between teams that produced the implementations in the first place. These sources are partial, sometimes contradictory, and require human judgement to reconcile.

Skipping ontological recovery is cheaper in the short run and catastrophic in the long run. The modernized system inherits whatever ontological confusion the legacy had, now expressed in clean modern code that makes the confusion harder to detect and harder to fix.

## Pattern

Treat the domain ontology — the formal articulation of what entities exist in the domain and how they relate — as an independent substrate of the modernization, separate from the structural graph (Pattern 3), separate from the semantic index (Pattern 6), separate from the implementation artifacts of the legacy. The ontology specifies what entities exist in the domain, how they relate, what the canonical vocabulary is, where the boundaries between concepts lie. It is the foundation of the ubiquitous language. It is not derived from the legacy; it is *grounded* in conversations with the people who understand the domain, validated against business sources, and reconciled where the legacy disagrees with itself.

Recovery of the ontology is partial from the legacy. Vocabulary inference from comments, display literals, naming conventions, the intermediate representation, and the data layer's DDL — DB2 schemas, VSAM definitions, DCLGEN copybooks — provides candidate terms. The data layer often preserves domain vocabulary better than the procedural code does: column names in DDL frequently retain canonical business terms that working-storage variables in COBOL paragraphs have abbreviated, prefixed, or renamed for technical convenience. Semantic similarity over code units (Pattern 6) surfaces clusters that may correspond to ontological concepts. These are starting points, not conclusions.

Workshop techniques like Event Storming (developed by Alberto Brandolini for collaboratively recovering domain understanding from systems and people) accelerate the human side of this work — domain experts, developers, and operators in a room mapping events, commands, and aggregates against shared vocabulary. The architect or domain expert validates each candidate against domain understanding, refines vocabulary, reconciles drift, articulates the canonical ontology that the modernized system should encode.

Eric Evans named this work *distillation* (Evans, 2003): separating what is essential about the business from what is incidental about how the legacy happened to express it. Vaughn Vernon (2013) elaborated the operational mechanics for in-life systems. What this catalog adds is the framing of ontology as a substrate of the modernization architecture — not just a mental model the team carries, but a queryable, versionable artifact independently rendered from the legacy substrates that fed it. The ontology lives independently of any implementation substrate. It does not change when the schema changes, when the architecture changes, when the framework changes. It changes only when the domain changes — when the business itself adopts new concepts or retires old ones. This independence is what makes ontology durable across modernizations and across the system's lifetime.

The modernization uses the ontology as a reconciliation reference. When the legacy has three definitions of "active customer," the ontology articulates the canonical one and the modernization team decides which legacy paragraphs implement which concept under the canonical definition.

Twin Verification (Pattern 16) confirms behavioural equivalence within each canonical concept; the ontology decides which paragraphs belong to which concept in the first place.

Ontology recovery is not a one-shot activity at the start of the modernization. Nick Tune has documented how the target model itself drifts during migration: concepts that began as straightforward renames end up restructured as the team's understanding sharpens through contact with the legacy and with domain experts (see *Drifting Domain Model* in the glossary). Pattern 5 accommodates this: the ontology is a living substrate, versioned and revisable, and the harness (Pattern 23) records each revision as a first-class event in the modernization's audit trail.

---

**[FIGURE] 5.1 — *Ontology as independent substrate***

*Diagram showing the layered relationship between substrates. Bottom layer (implementation artifacts): legacy COBOL paragraphs, current schema, current architecture — labelled "what is built today." Middle layer: structural graph (Pattern 3) and semantic index (Pattern 6) — labelled "what the legacy is." Top layer (independent, separately rendered): domain ontology — labelled "what the domain really is" with a thin connection back showing it is informed by but not derived from the lower layers. To the side: a panel showing "active customer" with three legacy definitions converging into one canonical ontology entry. Style: layered architecture diagram with the ontology layer visually elevated and lighter background to convey its independence. Annotations call out the directional flow: ontology informs implementation, but implementation does not author ontology.*

---

## Consequences

The modernization has a referent that is independent of the legacy. Disagreements within the legacy can be reconciled against the ontology rather than preserved by default. The vocabulary in the modernized C# matches the domain, not the historical accidents of how the legacy expressed the domain.

The ontology also functions as a defence against the *Behavioural Equivalence Without Ontology* antipattern. When the modernization preserves three behaviours that the ontology says should be one, the divergence is visible and addressable. When the modernization preserves vocabulary that the ontology says is wrong, the discrepancy becomes a deliberate decision (preserve for compatibility) or a refactoring target (rename to canonical), not an unnoticed inheritance of confusion.

The cost is that ontology recovery is genuinely hard work. It requires access to domain experts whose time is scarce. It requires reconciling sources that disagree. It requires judgement calls that are not derivable from the legacy and not obvious from any single domain artifact. Most modernizations do not do this work, which is why most modernizations inherit the ontological drift of their predecessors.

This pattern is *next* in Rosetta. The principle is articulated; the substrates that feed ontology recovery (vocabulary inference, semantic clustering) are operational; what remains is the ontology substrate itself as a first-class artifact — the operational machinery to validate, refine, and maintain canonical ontology independently of the structural and semantic substrates. The work is in design; the foundation it builds on is in place.

## Related patterns

The *Behavioural Equivalence Without Ontology* antipattern names the failure mode this pattern protects against — without Pattern 5, modernization preserves the legacy's confusion under the appearance of fidelity. Pattern 2 (*The Legacy as Oracle*) is what this pattern complements: behavioural fidelity is necessary but not sufficient, and Pattern 5 names the missing piece. Pattern 3 (*The Graph as Projection*) provides structural input that ontology recovery can draw on. Pattern 6 (*The Graph and the Index as Complementary Substrates*) provides semantic input through vocabulary inference and similarity, also a starting point for ontology rather than a substitute. Pattern 9 (*The Intermediate Representation*) consumes the ontology when it is available — IR vocabulary aligns with canonical ontology rather than with whatever the legacy happened to use.

---

# Pattern 6: The Graph and the Index as Complementary Substrates

*Status: in progress.*

## Context

A modernization pipeline with both a structural graph (typed relationships, deterministic ingestion) and a semantic index (vector representations, similarity search). The two representations are derived from the same source but answer different questions. Agents need to query across both. Operators need to maintain both as the source evolves.

## Problem

The instinct is to choose one substrate as canonical and treat the other as derived. Either the graph is the canonical representation and the index is computed from it, or the index is the canonical representation and the graph is a structural projection. Both choices create asymmetry: one substrate is authoritative, the other is secondary.

In practice, neither is fully derivable from the other. The graph captures relationships the index can't: a `CALL` edge between programs is structurally explicit but semantically obscure (calling another program looks similar to many other operations in vector space). The index captures similarities the graph can't: two paragraphs with no structural connection can be semantically near-duplicates. Treating either as canonical loses the value of the other.

## Forces

Operating two substrates is more complex than operating one. Synchronisation across them adds engineering cost. But forcing them into a single representation flattens what makes each useful.

The deeper issue is epistemological. The structural graph wants to be discrete, typed, exact — it answers questions with certainty: this program calls that one, this paragraph accesses that resource. The semantic index wants to be continuous, contextual, approximate — it answers questions with proximity: these paragraphs are near each other in meaning, with these confidence scores. These are different shapes for different reasons. Forcing similarity into typed edges loses the gradient; forcing typed relationships into vector space loses the precision.

## Pattern

Maintain the graph and the semantic index as complementary substrates over the same source. Define explicitly what information lives in each. The graph holds what is structurally explicit and queryable through traversal: containment, calls, accesses, predicates, entry points. The index holds what is semantically similar and queryable through proximity: vocabulary alignment, code-shape similarity, intent matching, naming convergence. The boundary between them is part of the architecture: when in doubt, ask which kind of question would the operator pose, and put the answer in the substrate that answers that kind of question naturally.

Synchronise the two through a shared ingestion pipeline. When source changes, both substrates update from the same input. Source provenance (Pattern 4) is what makes the dual substrate auditable. Every graph node has `source_file`, `start_line`, `end_line`; every index entry carries the same. When a hybrid query finds "paragraphs structurally connected to entry point X and semantically similar to paragraph Y," the result set is traceable on both axes. The agent doesn't just produce a working set — it produces a working set whose every member can be defended back to source.

Agent queries combine the two substrates by composition. A typical hybrid query starts in the graph (find the entry point for a specific transaction), expands structurally (follow PERFORMs and CALLs to depth N), then expands semantically (find paragraphs similar to those reached, even if not structurally connected). The reverse also works — start with semantic similarity, anchor the matches structurally — and complex queries iterate between the two. The result, regardless of direction, is a richer working set than either substrate alone would produce.

---

**[FIGURE] 6.1 — *Hybrid query: graph traversal meets semantic expansion***

*Diagram showing a hybrid query unfolding across both substrates. Left side: graph traversal — starting node (entry point) shown highlighted; arrows expand outward following PERFORM and CALL edges to a working set of structurally-connected paragraphs. Right side: semantic index — same paragraphs shown projected into vector space; nearest neighbors highlighted as "semantically similar but not structurally connected" — these are added to the working set. Bottom: combined working set rendered as a single bounded region. Annotations: "structural expansion: explicit relationships" and "semantic expansion: contextual similarity." Style: side-by-side layout with subtle visual contrast between the discrete graph (nodes and edges) and continuous index (point cloud). Goal: reader understands how the two substrates compose without forcing one into the other.*

---

## Consequences

Each substrate stays simple in what it does. The graph remains deterministic, reproducible, and exactly queryable. The index remains rich, semantic, and approximately queryable. Neither has to compromise to accommodate the other.

Hybrid queries become a first-class capability of the modernization pipeline. The agents have access to both kinds of relationships, and the platform doesn't prejudge which kind matters for a given query. The graph and index together render something the legacy modernization rarely produces directly: a context map. In DDD terms, a context map articulates how bounded contexts

relate — which integrate, which translate, which conflict. Here, that map emerges from the legacy as it actually is rather than as someone once described it.

The cost is operational. Two substrates require two ingestion pipelines (or one pipeline with two output paths), two query interfaces, two consistency disciplines. When one substrate updates, the other must follow. Schema changes ripple across both. The dual maintenance is real.

The benefit pays back over time. Modernization queries get richer, more nuanced, more capable of handling the messy reality of legacy code. The two substrates together approximate observations a domain expert with deep knowledge of the codebase would surface from intuition: "these unconnected paragraphs are doing the same thing," "this CALL is incidental, not semantic," "these similar-looking blocks implement different intents." That's not the totality of what an expert sees, but it is the part that scales with the codebase.

Combining knowledge graphs with vector indices is an emerging practice in retrieval-augmented generation systems generally — Microsoft's GraphRAG work, Anthony Alcaraz's writing on agentic GraphRAG, and the broader hybrid-retrieval literature have all articulated the value. What this catalog contributes is the framing of these as *complementary substrates with different epistemologies* — discrete and exact versus continuous and approximate — and the discipline of preserving both shapes rather than flattening them into a single representation. The boundary between them is part of the architecture, not an implementation detail.

## Related patterns

Pattern 3 (*The Graph as Projection*) defines the structural substrate. Pattern 5 (*Domain Ontology as Independent Substrate*) consumes the semantic substrate as one of its inputs — vocabulary inference and similarity clustering are starting points for ontology recovery, even though the ontology itself lives independently of the index. Pattern 4 (*Source Provenance Discipline*) is what enables cross-referencing between graph and index.

---

# Pattern 7: Vertical Slice Discovery from Structural and Behavioural Signals

*Status: in progress.*

## Context

A legacy codebase parsed as a graph (Pattern 3) and enriched with semantic signal (Patterns 4, 5). The graph contains thousands of paragraphs, hundreds of programs, dozens of bounded contexts. The modernization effort needs to identify *vertical slices* — coherent feature units that can be extracted, scaffolded, translated, and verified as a working whole. Each slice represents one user-visible behaviour from input through processing through output. In DDD vocabulary, slices map to use cases within bounded contexts; the aggregates a slice touches — aggregates are DDD's term for clusters of domain objects treated as a single unit for state changes, with invariants that hold across the cluster — define its consistency boundary.

## Problem

A bounded context is too large to be the unit of work. A paragraph is too small to be a feature. The right unit is between — a slice that includes the side effects it produces, not just the logic that triggers them. But slices are not stated explicitly anywhere in the legacy. They have to be derived.

Pure structural derivation produces slice candidates that are syntactically coherent but operationally meaningless. Pure observational derivation (which transactions actually run together) produces slices that reflect current usage patterns but miss the structural coherence. Either signal alone is insufficient.

## Forces

The slice must be small enough that the agents can scaffold and translate it as a unit, but large enough that it represents real business behaviour. Structural cohesion is one signal; behavioural cohesion is another. The two signals frequently agree but sometimes diverge — and the divergence is informative.

## Pattern

Derive slice candidates from a multi-signal pipeline. Five signal sources contribute, each with its own engineering cost and confidence weight:

- **Structural cohesion in the graph** — paragraph clusters with high internal cohesion (shared data, predicates, ancestry from a common entry point), low external coupling, and bounded side-effect surface.
- **Linguistic cues encoded by the original programmer** — `CALL` and `EXEC CICS LINK` mark structural coupling within a slice; `XCTL` typically marks transition between bounded contexts; `START TRANSID` marks a new transactional unit; `EXEC CICS WRITE TS QUEUE` and `EXEC CICS WRITE TD QUEUE` mark asynchronous communication points where slices can split; shared copybooks mark data coupling. Each construct is a decision the original programmer made about how the system is composed.
- **The data layer's DDL** — DB2 schemas, VSAM file definitions, DCLGEN copybooks. Field names in DDL frequently preserve domain vocabulary better than working-storage names. Foreign keys reveal relationships hidden by procedural code. Constraints capture domain invariants. Where DDL boundaries align with proposed slice boundaries, confidence increases; where they diverge, the divergence is informative.
- **Operational observation when available** — paragraphs exercised together by real transactions, with shared inputs and shared outputs.
- **Synthetic execution when production telemetry is not available** — the Legacy Twin (Pattern 2) is already runnable locally; synthetic test inputs exercise candidate slice boundaries and reveal which paragraphs activate together. The synthetic signal is not equivalent to production observation — it reflects test design, not real usage — but it catches many divergences between structural intuition and operational reality.

The heuristics that map signals to slice boundary candidates live as first-class artifacts, not as implicit knowledge in agent prompts. A heuristic catalog declares: "XCTL between paragraphs in different bounded contexts is strong evidence of slice transition; weight: 0.8." Specialized agents query the catalog when they need to interpret a cue in context. The catalog is queryable, versionable, and observable — when an agent proposes a slice boundary, the reasoning record (Pattern 24) cites the heuristics applied and the evidence supporting each.

For CICS specifically, pseudo-conversational boundaries (`RETURN TRANSID/COMMAREA` cycles) provide strong structural evidence: the system itself signals where one user-visible interaction ends and the next begins. Use this as the primary structural anchor.

The output is a set of candidate slices, each with: an entry point, a set of paragraphs that participate, a side-effect surface, an estimated tier classification (Pattern 10), and a confidence signal indicating how strongly the signals agree. Treat low-confidence slices as needing human validation; treat high-confidence slices as ready for scaffolding. The discovery process is not "find all slices automatically" — it is "propose slices with evidence and let architects validate or refine."

## Consequences

The modernization gets units of work that are operationally meaningful, not just syntactically coherent. The agents work on real features; architects validate slice boundaries based on combined evidence. Each slice carries its own provenance — traces back to the graph nodes, DDL fragments,

and observation traces (production or synthetic) that grounded it. This makes the slice a queryable artifact: the cockpit (Pattern 25) shows why a slice was proposed and where signals diverge.

Slices are not the unit of business value — capabilities are. A business capability ("process insurance claim," "underwrite policy renewal," "reconcile end-of-day") typically composes multiple slices. Slice discovery should produce slices that aggregate naturally into recognisable capabilities. When proposed slices do not compose into capabilities the business recognises, the discovery has fragmented something unitary or conflated something separable. Capability mapping is a validation lens for slice quality, not a substitute for slice discovery.

The cost is the multi-signal pipeline. Each signal has its own engineering cost; when a source is unavailable, slice discovery falls back to remaining signals with explicit confidence reduction. The audit trail records which signals validated each slice. Pattern 15 (*Architecture Documentation as Pluggable Emitter*) can render slices into specific views — slice maps showing the working set, communication maps showing queues and CALLs between slices, dependency diagrams showing the side-effect surface.

The pattern is in active development. Structural slice discovery is implemented and validated inside the Rosetta prototype. Behavioural integration is being built alongside the production-mode verification work in Patterns 14 and 15.

The notion of *vertical slice* as architectural unit comes from Jimmy Bogard and Steven Smith's articulation of vertical slice architecture (Bogard, 2018; Smith, 2018). Alberto Brandolini's Event Storming (Brandolini, 2013) provides the workshop technique for collaborative slice discovery with domain experts. What this catalog contributes is the methodology for slice discovery in legacy mainframe systems — the linguistic cues, the multi-signal pipeline (structural, linguistic, DDL, observational, synthetic), the heuristic catalog as queryable artifact — applied at codebase scale where Event Storming alone wouldn't reach.

## Related patterns

Pattern 3 (*The Graph as Projection*) provides the structural substrate. Pattern 5 (*Domain Ontology as Independent Substrate*) provides the canonical vocabulary slice boundaries align with — slices are coherent units of domain behaviour, and the ontology says what counts as a unit. Pattern 8 (*The Compiler Principle*) is why the heuristic catalog lives as deterministic infrastructure. Pattern 10 (*Tier-Aware Scaffolding*) consumes slice candidates and produces the appropriate scaffold. Pattern 15 (*Architecture Documentation as Pluggable Emitter*) renders slice candidates into views the architect can validate. Patterns 14 and 15 (*Hypothesis-Driven Verification*, *Behavioural Specifications*) provide the behavioural signal that refines structural slice discovery. Pattern 19 (*Bounded MCP Servers*) is where specialized slice-discovery agents live with explicit access to the heuristic catalog. Pattern 24 (*Reasoning Telemetry as First-Class Output*) makes heuristic application observable.

---

# Part II: Tactical Generation

---

**[ILLUSTRATION] P2 — *Tactical Generation: from substrate to scaffold***

*Style: editorial illustration matching the cover's aesthetic. Subject: a craftsperson's hands operating an old mechanical printing press, but the press itself is partially abstract — the type sets being composed are rendered as code structures (curly braces, class names, method signatures) rather than letters. Pages emerging from the press show fully formed C# classes. The metaphor: deterministic mechanism faithfully rendering structured intent into code, with the human craft visible at the input side. Format: full-page or two-thirds page opener for Part II.*

---

The patterns in this group address tactical design as DDD uses the term: how each bounded context materialises in modern code. Aggregates, domain events, handlers, the architecture chosen for each context. They presuppose strategic recovery from Part I — without bounded contexts identified and ubiquitous language established, generation produces structurally plausible code that doesn't reflect the domain.

The tactical patterns here are calibrated to AI-assisted generation. They specify what humans decide, what agents produce, what deterministic infrastructure renders, and how the boundaries between these stay clean.

---

# Pattern 8: The Compiler Principle

*Status: working.*

## Context

An agentic system for software engineering, where LLMs participate in code generation. The system has both deterministic decisions (what scaffold to render, what bounded contexts exist, what types to declare) and probabilistic decisions (how to translate idiomatic COBOL into idiomatic C#, how to handle edge cases, how to phrase things). How these two kinds of work are organised determines whether the system is reproducible and auditable, or opaque and unreliable.

## Problem

When the LLM is asked to make decisions of both kinds, the system becomes opaque and unreliable. Architectural choices made by the LLM aren't reproducible: ask twice, get different scaffolds. Boundary placements drift. Structural commitments are made on shaky ground. The deterministic infrastructure (compilers, linters, type systems) ends up checking outputs from a process that should never have produced them in that form.

The instinct to constrain the LLM through prompt engineering — telling it what kinds of decisions to make and which to defer — fails as the work scales. Prompts are advisory; the LLM treats them as suggestions, not contracts.

## Forces

LLMs are good at language transformation, idiom translation, and contextual disambiguation. They are bad at structural commitment, architectural integrity, and reproducible decision-making. Deterministic tools are the inverse. Both kinds of work are necessary in modernization. Mixing them in a single agent surface produces the worst of both.

## Pattern

Separate deterministic decisions from probabilistic ones at the architectural level, not at the prompt level. Put deterministic decisions in deterministic infrastructure: the graph holds architectural decisions, the architect validates them, the Roslyn-based emitter renders C# scaffolds from validated

decisions. Put probabilistic work inside the rendered scaffold: the agents translate paragraph-level COBOL into method bodies inside handler classes whose shape is already determined.

The agent doesn't choose architecture. The architecture is rendered from validated decisions, and the agent fills in the constrained space.

The deterministic side is itself made of explicit artifacts. The graph schema is queryable. The IR types are inspectable. The scaffold rendering rules are code, not prompt content. The heuristics that classify paragraphs into tiers, that propose slice boundaries, that detect anti-corruption layer candidates — these live as first-class catalogs that specialised agents query, not as implicit knowledge baked into agent prompts. Determinism doesn't end at "the compiler exists"; it extends through every rule the compiler applies.

In DDD terms, this is the boundary between strategic design (which the human architect performs, with agent assistance for analysis) and tactical execution (which agents perform, within the scaffold that strategic decisions produced). Strategic decisions about bounded contexts, subdomain types, and aggregate boundaries are not LLM decisions. Tactical decisions about how a particular paragraph translates to a method body, given the scaffold, are.

---

**[FIGURE] 8.1 — *The compiler principle: deterministic / probabilistic boundary***

*Diagram showing the architectural boundary between deterministic and probabilistic work. Left half (deterministic, cool tones): the structural graph (Pattern 3) feeds the IR (Pattern 9); the IR feeds the Roslyn-based emitter; the emitter renders C# scaffolds with all class structures, handler signatures, dependency wiring already determined. Right half (probabilistic, warm tones): the rendered scaffold contains explicit "method body" placeholders; agents work inside these constrained spaces translating COBOL paragraphs into C# logic. Vertical line down the middle labelled "the boundary the architecture defends." Annotations on the deterministic side: "graph holds decisions, compiler renders, architect validates." Annotations on the probabilistic side: "agents fill constrained space — cannot choose architecture." Style: split-panel diagram with strong visual contrast between sides. Goal: reader understands at a glance which work is mechanical and which is generative.*

---

**[CODE] 8.1 — *Roslyn SyntaxFactory rendering a deterministic handler scaffold***

*Code sample (~25 lines, C#) showing the emitter rendering a Wolverine handler class from IR data. The sample should depict: (1) reading a HandlerIntent from the IR; (2) using SyntaxFactory to construct the class declaration with name, namespace, dependencies; (3) generating the Handle method signature with command and event parameters typed; (4) leaving an explicit `// AGENT-FILL: method body` comment marker where the probabilistic work goes. The contrast between the deterministic structure (every line generated by code) and the explicit hand-off to the agent (the marker) makes the principle concrete. Include short prose introduction (~2 sentences) framing what the reader should notice.*

---

## Consequences

The system becomes reproducible at the architectural level. The same graph produces the same scaffold every time. The same scaffold constrains agents to the same kinds of work. Failures are

localised: when something goes wrong, it's clear whether the failure is in the architectural decision (wrong bounded context) or the agentic translation (wrong handler body). Each can be debugged independently. For regulated industries this matters operationally — auditors can review architectural decisions separately from generated code, and the audit trail at each layer is independently inspectable.

The cost is engineering discipline. The deterministic infrastructure has to actually be deterministic, which means a real compiler/emitter (Roslyn SyntaxFactory) rather than templated string interpolation. The graph has to actually hold the decisions, which means a schema rich enough to express them. The boundary between deterministic and probabilistic work has to be defended actively.

The principle isn't novel in absolute terms — compilers have always been deterministic infrastructure with creative work happening inside their constraints. What is new is applying this division to the architecture of agentic workflows themselves: agents as the creative work, deterministic infrastructure as the harness around them. Anthony Alcaraz has named this "Pierre Milet Llobet's Project Rosetta principle" — naming it explicitly because the field has not yet articulated it as a principle in its own right. Although Rosetta operates in mainframe modernization specifically, the principle itself applies wherever AI-assisted code generation must coexist with architectural integrity.

## Related patterns

Pattern 9 (*The Intermediate Representation*) is the contract between the deterministic and probabilistic sides. Pattern 10 (*Tier-Aware Scaffolding*) operates entirely on the deterministic side. Pattern 11 (*Pluggable Emitters*) is a corollary: if the principle is correct, the deterministic side should be replaceable without disturbing the probabilistic side.

---

# Pattern 9: The Intermediate Representation

*Status: working.*

## Context

A pipeline that has both a flexible substrate where analysis happens (the graph, where bounded contexts emerge and uncertainty lives) and a strict surface where generation happens (the emitter, where C# is produced through Roslyn). The two need to communicate. Naively connecting them couples the analysis to the generation in ways that resist evolution.

## Problem

Without a typed contract between analysis and generation, every change to either side propagates everywhere. A new pattern discovered in the analysis (say, a new kind of saga) requires changing the emitter to render it. A new architectural target for the emitter requires re-deriving everything from the graph. The pipeline stiffens.

## Forces

Analysis must remain flexible because the field is still discovering what to look for in legacy code. Generation must remain strict because rendering requires every piece of information in a specific shape. Both pressures are real and pull in opposite directions.

A second tension: the IR's vocabulary must be stable enough that downstream emitters can rely on it, but evolutionable enough that new analysis patterns can extend it. Treating the IR as frozen produces tools that can't grow with what's discovered. Treating it as fluid produces tools that break with every analysis change. Both extremes fail.

## Pattern

Place a typed intermediate representation between the graph and the emitter. The IR captures architectural intent — commands, events, handlers, sagas, aggregates, side-effect surfaces — in data structures the emitter can render. Each IR element is grounded back to the graph nodes that derived it.

This vocabulary is not arbitrary. It is the tactical design vocabulary of DDD as articulated by Eric Evans (2003) and elaborated by Vaughn Vernon (*Implementing Domain-Driven Design*, 2013): aggregates as consistency boundaries, domain events as observable facts about the past, commands as expressions of intent, sagas as long-running processes that span aggregates. The IR encodes the modernized system's tactical design in the same vocabulary the team uses to discuss the domain — closing the gap between what the architecture says and what the code expresses.

The IR draws its vocabulary from the domain ontology (Pattern 5), not from the legacy's accidental naming. When the IR captures an aggregate as `Customer`, that name comes from the canonical ubiquitous language the modernization team has validated — not from whatever the legacy happened to call it (`CUST-MAST-REC`, `CMR`, `CUSTREC01`). Where the ontology is not yet established, the IR carries provisional names from vocabulary inference, marked explicitly as provisional, awaiting validation. The data layer's DDL — schemas, foreign keys, constraints — also informs aggregate boundary discovery: foreign keys reveal containment relationships, NOT NULL and CHECK constraints capture invariants of the aggregate. The IR is one of the principal consumers of ontology recovery — without it, the IR encodes the legacy's confusion in modern syntax.

The IR is not a structurally neutral intermediate. It encodes architectural commitments: when the IR captures a `HandlerIntent` with typed commands and events, it has committed to Vertical Slice Architecture for that bounded context; when it captures a `PortIntent` with adapter slots, it has committed to Hexagonal Architecture; when it captures a `SagaIntent` with choreography steps, it has committed to event-driven coordination. These are hard decisions — they structure the important constructs of the generated code. They are not LLM-mutable; they are validated by the architect during strategic recovery (Part I) and frozen into the IR before generation begins.

The IR represents the hard decisions that structure the important constructs of the generated code. Everything that is structural — what gets generated, how it composes, which architecture style applies — lives in the IR. Everything that is creative — how a specific paragraph translates to a specific method body — lives in the agent's space within the scaffold the IR defined.

The IR is the contract. If the analysis can produce a valid IR, the emitter can render it. If the analysis can't, the analysis isn't done yet. New analysis patterns extend the IR; new generation targets consume the IR.

The rendering is deterministic and explicit. The emitter is implemented as Roslyn code generators — Roslyn is Microsoft's open-source C# and VB compiler platform, which exposes the compiler as a service: parsers, syntax trees, semantic models, and code generators are all addressable as APIs. Where traditional compilers are black boxes, Roslyn lets you treat the compiler itself as programmable infrastructure. Each architectural commitment in the IR has a corresponding Roslyn code generator that knows how to render it. Changing architecture style — moving a bounded context from VSA to hexagonal, or from hexagonal to layered clean architecture — is not a prompt change. It is writing new Roslyn code generators that consume the same IR types and emit the new style. The IR stays stable; the rendering changes. This is what makes Pattern 11 (*Pluggable Emitters*) tractable: architecture diversity at the rendering layer, not at the analysis layer.

In Rosetta, the IR is called WolverineIntentModel because the target it most directly serves is Wolverine handlers. The naming is specific to the implementation; the principle is not.

---

**[FIGURE] 9.1 — *The IR as contract between analysis and generation***

*Diagram showing the IR as the central contract layer. Top of diagram: multiple analysis passes (graph community detection, slice discovery, predicate analysis, ontology recovery) all producing IR fragments. Center of diagram: the typed IR — depicted as a structured tree showing key types (BoundedContext → Aggregate → Handler, with Commands and Events as leaves). Bottom of diagram: multiple emitters (vertical-slice scaffold, hexagonal scaffold, hybrid scaffold, documentation emitter) all consuming the same IR. Side annotations: "analysis side: extensible — new patterns add IR fragments" and "generation side: strict — emitter requires complete IR." The IR sits visually elevated as the pivot. Style: hub-and-spoke layout. Goal: reader sees that the IR is what allows the system to evolve in two directions independently.*

---

**[CODE] 9.1 — *The WolverineIntentModel IR in C#***

*Code sample (~30 lines, C#) showing the IR as typed C# classes. Depict the key types:* `BoundedContextIntent` *(with name, tier, child aggregates),* `AggregateIntent` *(with name, root entity, invariants),* `HandlerIntent` *(with name, command, events emitted),* `CommandIntent` *and* `EventIntent` *(with typed payloads). Include* `SourceProvenance` *field on each type linking back to graph nodes (Pattern 4). The sample should make concrete that the IR is real C# — typed, validated, discoverable through tooling — not configuration in YAML. Brief prose introduction (~2 sentences) noting that this is what analysis populates and what emitters consume.*

---

## Consequences

Analysis and generation evolve independently. New patterns extend the IR without touching the emitter; new targets add emitters without touching the analysis. The IR itself becomes the durable artifact of what the modernization understands about the legacy. For audit purposes, the IR is the bridge: any generated artifact traces back through its IR origin to the graph nodes and ontology terms that grounded it. Auditors can inspect architectural decisions at the IR layer separately from the rendered code; each layer is independently inspectable.

The cost is a layer that looks dull but isn't. The IR has to actually be typed, which means real schema with real validation. The grounding back to graph nodes has to be maintained through all transformations. The IR's vocabulary has to stay aligned with the architectural intent the analysis is trying to capture, which means the IR's design is itself an ongoing concern.

The intermediate representation as concept comes from compiler theory — LLVM IR (Lattner & Adve, 2004), GCC GIMPLE, and earlier compiler IRs articulated decades ago that separating front-end analysis from back-end generation requires a typed intermediate. What this catalog contributes is the framing of IR as the contract between agentic analysis and deterministic generation specifically: the surface where heuristic-driven recovery and rule-driven scaffolding meet, with each side enforcing its own discipline at the boundary. The naming WolverineIntentModel reflects the target this IR most directly serves; the principle generalises to any agentic modernization where what's discovered must be rendered into something compilable.

## Related patterns

Pattern 8 (*The Compiler Principle*) is what motivates the IR's existence. Pattern 5 (*Domain Ontology as Independent Substrate*) provides the canonical vocabulary the IR uses — without ontology, the IR

risks encoding the legacy's confusion in modern syntax. Pattern 11 (*Pluggable Emitters*) is what the IR enables on the generation side: architecture diversity at the rendering layer, with the IR as stable contract. Pattern 15 (*Architecture Documentation as Pluggable Emitter*) is what the IR enables on the documentation side: the same IR feeds both code emitters and documentation emitters. Pattern 4 (*Source Provenance Discipline*) extends through the IR — every IR element carries source coordinates back to the graph nodes that derived it. Pattern 25 (*The Cockpit*) surfaces IR fragments to architects during review, making the architectural commitments visible before generation begins.

---

# Pattern 10: Tier-Aware Scaffolding

*Status: working.*

## Context

A modernization pipeline producing C# scaffolds for many bounded contexts within a single legacy system. Each bounded context has different domain weight — some are core to the business and likely to evolve significantly, others are generic supporting logic that should be cheap to maintain.

## Problem

Most modernization platforms apply a single architectural template across the entire codebase. The result is that generic domains get over-engineered (hexagonal architecture for a lookup table is masochism) and core domains get under-engineered (vertical slices for a pricing engine collapse under their own weight in a few years).

The instinct to apply uniform architecture is operationally simpler — the scaffold renderer has one mode, every team works the same way, training is consistent. But the resulting code carries inappropriate complexity for most of the codebase and inadequate complexity for the parts that matter most.

## Forces

Architectural complexity has real costs (more abstractions to maintain, more code to read, more places to introduce bugs) and real benefits (clearer boundaries, easier evolution, better long-term durability). Both costs and benefits scale with how much the code matters and how often it changes. Uniform architecture forces a single trade-off across the entire codebase.

## Pattern

Annotate each bounded context with a domain tier based on importance and likely evolution. Use the tier to drive scaffold selection. The taxonomy is calibrated to CICS COBOL modernization but the underlying idea comes directly from Eric Evans' classification of subdomains in *Domain-Driven Design* (2003): some subdomains are **core** to what differentiates the business, some are **supporting** of the core, some are **generic** capabilities the business needs but does not differentiate on. Each subdomain type deserves different architectural investment.

In Rosetta, this maps to three scaffold tiers:

- **Tier 0–1** (generic and supporting subdomains): vertical slice architecture. Easy to read, easy to change, no shared abstractions for thin domain logic. Generic capabilities — reference-data lookups, validation rules, audit logging — do not benefit from heavy structure.
- **Tier 2** (core subdomains with moderate complexity): hybrid scaffold. Vertical slices for request handling, lightweight domain models for the underlying business logic. The middle ground for business logic that has weight but isn't deeply differentiated.
- **Tier 3** (strategic core, the crown jewels): full hexagonal architecture. Domain at the centre, ports and adapters, the structure that pays its tax over the system's lifetime. This is where Evans' insistence on protecting the core domain from infrastructural concerns earns its keep.

The tier classification lives in the graph as an annotation on each bounded context. It can be derived heuristically (cyclomatic complexity, coupling, change frequency from version control if available) and validated by an architect. It is not an LLM decision.

---

**[TABLE] 10.1 — *Tier-aware scaffolding: subdomain types, scaffolds, and trade-offs***

*Three-column table comparing the three tiers along multiple dimensions. Columns: Tier 0–1 (Generic/Supporting), Tier 2 (Core, moderate), Tier 3 (Strategic Core). Rows to include: DDD subdomain type; example legacy capability (reference-data lookup; pricing logic; underwriting engine); scaffold architecture (vertical slice; hybrid; full hexagonal); typical investment ratio (low; medium; high); evolution rate (low; moderate; high); review depth (light; standard; deep); ratio of code expected at this tier in CICS engagements (~30%; ~55%; ~15%). Style: clean three-column layout with subtle row banding. Goal: reader gets one-glance reference for the tier system.*

---

## Consequences

The architecture earns its complexity. Where complexity is justified, you pay for it. Where it isn't, you don't. Generic supporting code stays simple and cheap to maintain. Core code gets the structural investment it deserves.

The cost is the tier classification itself. Heuristic derivation is approximate; an architect must validate. The taxonomy may not transfer cleanly across all domains — some codebases may need different tiers, different boundaries, different scaffold shapes. The principle (match architecture to domain weight) is more general than the specific three-tier policy.

In the engagements I've examined, tier 2 dominates. Most code is moderately important business logic that doesn't deserve full hexagonal architecture but isn't trivial enough for pure vertical slices. The hybrid scaffold has earned its place by being the most-used.

## Related patterns

Pattern 8 (*The Compiler Principle*) is what makes tier-based scaffolding tractable: the deterministic emitter can select scaffolds based on tier annotations without LLM involvement. Pattern 11 (*Pluggable Emitters*) is what makes the scaffold variants implementable as separate emitters.

# Pattern 11: Pluggable Emitters

*Status: working.*

## Context

A modernization pipeline whose target architecture is one of multiple possible architectures. The choice of target depends on the engagement — some teams want vertical slice architecture (VSA) with Wolverine handlers, others want hexagonal architecture with explicit ports and adapters, others want a Java target instead of C#, others may want a hybrid approach for tier-2 bounded contexts that balances VSA velocity with hexagonal protection. The target may evolve as understanding of the legacy improves, or as the team's priorities shift across the lifetime of the modernization.

## Problem

If the target architecture is hard-coded into the pipeline, switching targets requires rebuilding the pipeline. Teams either commit to one architecture upfront and live with the consequences, or they fork the pipeline per target and maintain divergent toolchains. Both options are operationally expensive.

If the target architecture is configured but the configuration is shallow — a few flags, a few template variants — the pipeline supports limited variation around a single target shape but can't address fundamentally different targets. A pipeline configured to produce vertical slice variants can't suddenly produce hexagonal scaffolds; the configuration vocabulary is too narrow to express the structural difference.

The deeper issue is that target architecture is a first-class decision that should live somewhere identifiable in the pipeline's design, not as a side effect of how the pipeline was constructed.

## Forces

Different engagements have different correct targets. The same legacy may be modernized for different audiences with different architectural preferences — a bank's risk-engine team may want hexagonal for strategic-core contexts; the same bank's commodity reporting team may want lift-and-shift to VSA. Forcing a single target across all uses limits the pipeline's reach.

But supporting multiple targets requires architectural discipline: the pipeline must factor cleanly enough that the target-specific logic is replaceable. Every concession to target-agnosticism in the

analysis side is a constraint on what the IR can express. Every concession to target-specificity in the emitter side is duplication when targets share structure.

## Pattern

Make the emitter pluggable. The graph and the IR (Pattern 9) are target-agnostic; they describe the legacy and its architectural intent without committing to a specific output shape. The emitter is target-specific; it consumes the IR and produces a scaffold in whatever target architecture is appropriate.

A new target requires a new emitter, not a new pipeline. The same graph, fed to the same IR, fed to a different emitter, produces a different scaffold. The architectural decision lives in the choice of emitter — a deliberate, visible, replaceable choice — not in the pipeline's foundations.

Emitters do more than generate code. An emitter is responsible for the whole shape of the scaffold: directory structure, project files, dependency wiring, test scaffolds, build configuration, deployment metadata, even the documentation skeleton. A VSA emitter generates one shape; a hexagonal emitter generates another; a hybrid emitter generates a third. Each is its own Roslyn code generator (Pattern 9) — or, for non-C# targets, its own equivalent in the target language's tooling.

In Rosetta today, emitters exist for vertical slice architecture (Wolverine, C#) and for hexagonal architecture (Wolverine + explicit ports, C#), with a hybrid emitter for tier-2 contexts that combines VSA velocity with hexagonal protection at specific seams. Future emitters could target Java with Spring Boot for organizations standardised on JVM stacks; or Jeremy Miller's Jade for event-sourced workloads where Wolverine isn't the best fit; or even non-handler architectures like CQRS-with-separate-read-models for analytical bounded contexts.

The IR's job is to remain stable across emitter additions. When a new emitter is added, the IR shouldn't need new types — the existing IR types (HandlerIntent, AggregateIntent, CommandIntent, EventIntent) should be expressive enough that the new emitter can interpret them in its own architectural idiom. If a new emitter requires new IR types, that's a signal the IR was implicitly biased toward existing targets.

---

**[FIGURE] 11.1 — *Pluggable emitters: one IR, many targets***

*Diagram showing a single IR feeding multiple emitters in parallel. Center: the IR (Pattern 9) shown as a structured artifact. Radiating outward: multiple emitter boxes labelled "Vertical Slice (Wolverine, C#)," "Hexagonal (Wolverine, C#)," "Hybrid (tier-2)," "Event-sourced (Jade, C#)," and a faded/future box "Java Spring Boot." Each emitter produces a different scaffold, depicted as a small thumbnail of code structure. The diagram makes visible that the IR commits to architectural intent (aggregates, handlers, events) but not to target architecture. Annotations: "IR target-agnostic" near the center, "emitter target-specific" near each emitter. Style: hub-and-spoke with the IR as visual anchor. Goal: reader sees that the architectural decision lives in emitter selection, replaceable without disturbing analysis.*

---

## Consequences

The architectural decision lives in the choice of emitter, not in the graph or the IR or the agents. That decision becomes explicit, reviewable, replaceable. Teams modernize their codebase against the architecture they want, not against the one the pipeline imposed by accident.

Emitters compose into a library that grows over time. The first engagement contributes the first emitter; the second engagement may reuse it or add a variant. After several engagements, the catalog of emitters covers the most common target shapes, and new engagements typically need to extend an existing emitter rather than write a new one from scratch. The investment compounds across the engagement portfolio.

The cost is the discipline of keeping the IR target-agnostic. Every IR concept must be expressible in any reasonable target architecture, which constrains the IR's vocabulary. Adding a new target may reveal that the IR has been quietly target-specific in places, requiring refactoring. The IR's stability across emitter additions is something to maintain deliberately, not assume.

There is also a documentation parallel: Pattern 15 (*Architecture Documentation as Pluggable Emitter*) extends this same compiler architecture from code targets to documentation targets. The same IR that feeds the VSA emitter feeds the C4-diagram emitter; the same IR that feeds the hexagonal emitter feeds the aggregate-map emitter. The emitter abstraction is general — code is one kind of output, documentation is another.

Pluggable target architectures are a long-established practice in compiler back-ends — LLVM (Lattner & Adve, 2004) is the canonical modern example, with a single front-end feeding multiple architecture-specific back-ends for x86, ARM, RISC-V, and so on. What this catalog contributes is the application of the back-end pluggability principle to *architectural styles* rather than to *machine architectures*: the back-end isn't a CPU instruction set, it's an architectural pattern (VSA, hexagonal, event-sourced). The compiler discipline transfers; the granularity of the choice changes.

## Related patterns

Pattern 8 (*The Compiler Principle*) is what makes pluggable emitters possible: deterministic emission is what's being plugged in. Pattern 9 (*The Intermediate Representation*) is what the emitters consume — and is what must remain target-agnostic for pluggability to work. Pattern 10 (*Tier-Aware Scaffolding*) is one application of the pluggable-emitter pattern: tier policy is implemented as emitter selection (tier-0/1 uses VSA emitter; tier-3 uses hexagonal emitter; tier-2 uses hybrid). Pattern 15 (*Architecture Documentation as Pluggable Emitter*) extends the principle from code to documentation, using the same architectural mechanism.

---

# Pattern 12: Commands and Events as Logical Boundary, Independent of Physical Deployment

*Status: working.*

## Context

A modernized system structured as a set of bounded contexts, each generated through the patterns above. The bounded contexts must communicate with each other to implement business workflows that span them. The communication may need to be synchronous in-process initially (for performance, for transactional simplicity, for deployment minimalism) but may need to evolve toward asynchronous messaging or distributed services later (for scaling, for team boundary alignment, for operational independence).

## Problem

Most modernization projects pick a deployment model — monolith or microservices — and bake it into the architectural structure. The communication between bounded contexts is modeled as direct method calls (in monoliths) or as REST/RPC boundaries (in distributed systems). Either choice is hard to reverse: switching from method calls to messaging is a major refactor; switching from messaging back to method calls is also a major refactor.

The deeper problem is conflating two distinct decisions. *What contexts communicate, and what they communicate* is a logical question — which slice produces what command, which slice handles which event, what business intent is being expressed. *How they communicate, and where they're deployed* is a physical question — sync or async, in-process or over the network, monolith or microservices. The two questions have different answers at different times in the system's life. Conflating them couples the logical model to the deployment model in ways that resist evolution.

## Forces

In-process synchronous communication is operationally simpler — fewer moving parts, easier debugging, stronger transactional guarantees. Distributed asynchronous communication is more scalable — independent deployment, fault isolation, team autonomy. Most systems start in conditions

where the simpler choice is correct, but evolve toward conditions where the more scalable choice becomes necessary. The transition shouldn't require an architectural rewrite.

CICS transactional guarantees specifically must survive any transition. The legacy system has been preserving consistency semantics for decades; the modernized system loses something important if those semantics drop quietly during decomposition.

## Pattern

Model communication between bounded contexts as commands and events at the logical level, regardless of how they're physically implemented. A command expresses *something the system should do*; an event expresses *something the system has observed*. Both are first-class artifacts in the architecture: typed, named, owned by specific contexts, traceable through the audit trail.

The physical implementation is a separate concern. Initially, commands and events flow as in-process method calls within a modular monolith — the contexts share a process, communication is synchronous, transactional boundaries are simple. The framework underlying the implementation (Wolverine, Jade, or equivalents) handles the dispatching, but the public surface of each context is the command/event vocabulary, not method signatures.

When operational pressure justifies extraction — scaling demands, team-boundary alignment, deployment frequency divergence, regulatory isolation — the transition is mechanical. The same commands and events that flowed in-process now flow through messaging infrastructure. The receiving context's handlers are unchanged; the sending context's call sites are unchanged. What changes is the transport. The transactional boundary changes shape (eventual consistency replaces immediate consistency where appropriate), but the vocabulary stays the same.

The discipline: never let method-level coupling sneak in between contexts. Every cross-context interaction must go through a command or an event. The framework enforces this by making cross-context method calls inconvenient or impossible. Bounded contexts communicate only through their command/event surface.

---

**[FIGURE] 12.1 — *Commands and events: same logical boundary, different physical deployments***

*Diagram in two horizontal bands showing the same bounded contexts (e.g., Pricing, Billing, Notifications) under two physical deployments. Top band: "Modular monolith" — three bounded contexts within a single process boundary, with commands and events depicted as in-process arrows; one transactional boundary encompasses the cross-context flow. Bottom band: "Distributed services" — same three contexts now in separate process boundaries, with commands and events depicted as messages flowing through a message broker; multiple transactional boundaries with the transactional outbox pattern visible at each context. Vertical line connecting the two bands labelled "the logical boundary stays the same — only the transport changes." Style: parallel architectural diagrams with shared vocabulary across bands. Goal: reader sees that decomposition is mechanical, not architectural — the commands and events are the contract.*

---

**[CODE] 12.1 — *A Wolverine handler with command, event, and transactional outbox***

*Code sample (~25 lines, C#) showing a Wolverine handler that receives a command, performs work, emits an event, and uses the transactional outbox pattern. Depict: (1) command class definition with typed prop-*

*erties; (2) event class definition; (3) handler class with the Handle method receiving the command, the IDocumentSession (Marten) for persistence, and IMessageBus for emission; (4) the handler's body showing the transactional pattern — load aggregate, mutate, emit event, save — all within the implicit Wolverine transaction. The same handler code works whether the recipient lives in-process or out-of-process; that point is highlighted in the introductory prose (~2 sentences).*

---

## Consequences

The architectural decision of *what to deploy where* becomes operational and reversible. Teams can extract a context to its own service when operations justify it, without rewriting the consumer code. Teams can pull a struggling service back into the monolith if the operational complexity isn't paying for itself. The decomposition is a flywheel, not a one-way ratchet.

Where the modernized system must coexist with parts of the legacy that are not yet migrated, an **anti-corruption layer** (Eric Evans' term for a translation layer that prevents legacy concepts from polluting the modernized domain model) intercepts the legacy interaction and translates between the legacy's vocabulary and the canonical ubiquitous language of the modernized side. The anti-corruption layer is itself a bounded context with a single responsibility — protect the modernized domain from inheriting whatever the legacy still calls things.

CICS-style transactional guarantees survive the transition because the framework provides them. Wolverine's transactional outbox pattern, for example, ensures that commands published as part of a transaction are delivered reliably even when the recipient is now a different service. The consistency semantics are preserved through infrastructure, not through hoping nothing breaks.

Architectural decisions become localized. The choice of in-proc vs. distributed lives in the deployment configuration, not in the source code. The choice of which contexts share a process can change without code changes. Architectural evolution becomes operational, which is what mature engineering organizations need.

The cost is the discipline. Every cross-context interaction must be modeled as a command or event, which adds ceremony compared to just calling a method. New developers must learn the discipline; tools must enforce it; reviews must check it. The framework handles most of the mechanical cost, but the cognitive cost of thinking in commands and events is real, especially for teams used to method-level coupling.

This pattern is what allows the decomposition flywheel — the post-cutover architectural evolution from modular monolith toward selective service extraction — to work as a continuous process rather than as a major project. The architectural shift is mechanical because the logical boundaries were always there; only the physical boundaries change.

The same logical-boundary discipline supports two transitional techniques Nick Tune has named: *Expose Legacy Asset*, in which the legacy publishes events that modernized subsystems consume as their integration surface; and *Republishing Legacy Events*, in which an autonomous bubble consumes those legacy events and re-emits them in the canonical target vocabulary, allowing downstream consumers to decouple from the legacy model before the migration is complete. Both are forms of the same idea — the logical contract carries across the physical transition — applied at the legacy boundary rather than at the modernized boundary. Definitions and pointers are in the glossary.

## Related patterns

Pattern 9 (*The Intermediate Representation*) captures commands and events as first-class IR concepts; the IR's vocabulary is what makes this pattern possible at scaffold-rendering time. Pattern 11 (*Pluggable Emitters*) is what allows different physical implementations to be selected — an emitter for in-process Wolverine, an emitter for distributed Wolverine, an emitter for Jade for event-sourced workloads. Pattern 23 (*The Harness as State Machine*) governs the transition from one physical deployment to another, ensuring that extraction or consolidation is auditable and reversible.

---

# Pattern 13: Transactional Boundaries as First-Class Migration Concern

*Status: in progress.*

## Context

A modernization translating CICS COBOL transactions to C# handlers. The legacy system has implicit transactional boundaries everywhere: a CICS unit of work spanning from `EXEC CICS RETURN TRANSID` to the next `RETURN`, an IMS transaction surrounding a series of database updates, a batch JCL job step with checkpoint/restart semantics, an `EXEC CICS SYNCPOINT` marking a commit boundary. These boundaries are not decorative — they preserve invariants the business depends on. When the unit of work commits, related changes commit together; when it rolls back, related changes roll back together; when it fails, the system is in a known recoverable state.

The modernization must preserve these invariants. But the modern target — C# handlers, distributed services, event-driven workflows — has very different transactional primitives. Naive translation produces code that looks structurally similar but loses the transactional guarantees the legacy provided.

## Problem

Transactional boundaries in mainframe systems rarely survive migration intact. The default failure mode is silent: the modernization preserves syntactic structure but loses transactional semantics. The customer believes they have atomicity and discovers in production that they don't — partial updates persist, compensating transactions never fire, edge-case failures leave the system in inconsistent states that the legacy would have prevented.

The silent failure is the danger. If transactional boundaries were preserved or visibly violated, the team could see the problem. Because the violations are invisible — the code compiles, the tests pass, the happy path works — they accumulate until production traffic exposes them, often months after cutover.

Three categories of failure dominate. First: lost atomicity, where two changes that committed together in CICS now commit independently in the modernized system, opening windows where intermediate states leak. Second: lost rollback semantics, where a failure that would have aborted the entire transaction in the legacy now leaves partial state behind in the modern. Third: lost isolation, where reads that saw a consistent snapshot in CICS now see partial updates from concurrent

transactions in the modern system.

## Forces

CICS transactional guarantees are strong — pessimistic locking, distributed two-phase commit across DB2 and VSAM, recoverable units of work survived process restarts. Modern systems often relax these guarantees for scaling reasons — eventual consistency across services, optimistic locking within a service, sagas instead of distributed transactions. The relaxation is sometimes correct (the legacy's guarantees may have been over-engineered for the actual business need) and sometimes catastrophic (the business genuinely needs the guarantee, and silently losing it produces production failures).

Distinguishing the two cases requires explicit analysis, not implicit translation. The team must articulate what each transactional boundary guarantees, decide whether the guarantee is essential to the business, and design the modern equivalent — strict preservation, controlled relaxation, or saga compensation — with the trade-offs visible.

There is also a regulatory force. Banks, insurers, and government systems often have audit requirements about transactional behaviour: a settlement is atomic; an audit log is durable; a regulatory report sees a consistent snapshot. Losing these guarantees silently is not just a technical bug; it is a compliance failure.

## Pattern

Treat transactional boundaries as first-class migration artifacts. During strategic recovery (Part I), identify every transactional boundary in scope: CICS units of work, IMS transactions, batch JCL job steps, `EXEC CICS SYNCPOINT` commit points, database transactional scopes. The graph (Pattern 3) captures these as annotated edges and nodes; analysis surfaces them as candidate boundaries for review.

For each transactional boundary, document the invariants it preserves: which writes commit together, which reads see consistent snapshots, what rolls back together when a failure occurs. These invariants are part of the domain ontology (Pattern 5) — they are statements about what the business considers atomic, not statements about how the implementation happens to enforce atomicity.

Decide explicitly how each boundary maps in the modernized system. Three categories of decision:

- **Preserved strict** — the modern system preserves the legacy's transactional guarantees through equivalent mechanisms (a single-aggregate transaction in C#, a Wolverine handler with implicit transactional scope, a distributed transaction with two-phase commit where the data layer supports it). Used when the business genuinely requires the guarantee.
- **Relaxed deliberately** — the modern system intentionally loosens the guarantee, with explicit documentation of what's been relaxed and why. For example, eventual consistency across two bounded contexts where the legacy had distributed transactions. Used when analysis shows the strict guarantee was unnecessary and the relaxation enables better scaling.
- **Replaced with saga compensation** — the modern system implements the legacy's atomicity through saga choreography: each step is locally atomic, with explicit compensating actions if a later step fails. Used when distributed transactions are not viable and the business requires effective atomicity.

The decision is documented in the IR (Pattern 9) as part of the architectural commitment for each bounded context. Generated code includes the chosen mechanism — Wolverine's transactional outbox, MartenDB's session-scoped transactions, saga state machines. Twin Verification (Pattern 16) specifically tests transactional behaviour: simulate failures at every commit point, verify that the modernized system's recovery matches the legacy's.

CICS-specific cues guide the analysis. `EXEC CICS SYNCPOINT` marks a commit point; `EXEC CICS ROLLBACK` marks an explicit abort; `EXEC CICS START TRANSID` marks the beginning of a new transactional unit; pseudo-conversational boundaries (`RETURN TRANSID / COMMAREA`) mark transitions between transactional contexts. Each is a decision the original programmer made about what should be atomic. The modernization team must consciously confirm or revise each decision.

## Consequences

Transactional behaviour becomes a designed property of the modernized system, not an accident. When production exposes a transactional issue, the team can trace it back to a specific recovery decision — "we chose relaxed eventual consistency here; the production traffic shows that decision was wrong, and we need to redesign as a saga." Without the explicit decision trail, the team can only debug the symptom.

The pattern surfaces over-engineering. The legacy may have wrapped operations in transactions that did not need to be atomic — a habit of CICS programming rather than a business requirement. Explicit analysis distinguishes "this must be atomic" from "this happened to be atomic," allowing the modernization to relax constraints where appropriate and gain scaling room.

The cost is the analysis discipline. Every transactional boundary in scope must be reviewed, documented, and decided. For systems with thousands of transactions, this is non-trivial work. It cannot be automated entirely — analysis can surface the boundaries and suggest classifications, but business decisions about which guarantees are essential require human judgment. The capability map (Pattern 1) helps prioritise: strategic-core capabilities get deep transactional analysis; commodity capabilities can default to preserved strict for simplicity.

Vaughn Vernon's *Implementing Domain-Driven Design* (Vernon, 2013) articulates aggregate consistency boundaries as the canonical DDD anchor for transactional discipline. Pat Helland's work on eventual consistency and the limits of distributed transactions (Helland, 2007) informs the relaxation decision. Saga compensation patterns trace back to Garcia-Molina and Salem (1987) and have been extensively elaborated in microservices literature. What this catalog contributes is the framing of transactional boundaries as first-class migration artifacts in mainframe modernization specifically, where CICS conventions encode transactional decisions that must be consciously confirmed or revised — not silently translated.

## Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines which capabilities deserve deep transactional analysis. Pattern 3 (*The Graph as Projection*) captures the transactional cues in the legacy. Pattern 5 (*Domain Ontology as Independent Substrate*) holds the invariants as canonical statements about the business. Pattern 9 (*The Intermediate Representation*) encodes the chosen transactional mechanism per bounded context. Pattern 12 (*Commands and Events as Logical Boundary*) provides the cross-context communication primitives that sagas use. Pattern 16 (*Twin Verification*) verifies

transactional behaviour, not just functional behaviour. Pattern 17 (*Hypothesis-Driven Verification*) categorises transactional violations as a specific divergence class. The *Silent Semantics Loss* antipattern names what happens when transactional boundaries are translated implicitly rather than explicitly.

---

# Pattern 14: Transitional Architecture: The Modular Monolith as Migration Vehicle

*Status: working.*

## Context

A modernization that will run across many months — sometimes years — moving capabilities from mainframe to modern stack. The team has classified capabilities (Pattern 1), identified bounded contexts, decided the target architecture for each context. The temptation now is to design the *target state* — the eventually-distributed microservices architecture, the cloud-native deployment, the service mesh, the event bus — and start building toward it directly.

This temptation needs resisting. The target state is the destination, not the path. The architecture *during* the migration matters as much as the architecture at the end — sometimes more — because the migration's duration is when the team learns whether the boundaries it drew on a whiteboard survive contact with real code.

## Problem

Modernization teams fragment too early. Microservices feel modern; the cloud-native deployment is what the customer paid for; the architecture diagrams show separate services with their own databases. The team begins extracting bounded contexts into separate services from day one — each with its own deployment pipeline, its own data store, its own operational overhead. The result: an immature decomposition prematurely committed to physical separation, with all the operational complexity of distributed systems and none of the scaling benefits.

The operational cost is real. Each extracted service brings: a deployment pipeline, observability stack, alerting rules, on-call rotation, network configuration, security policies, data store, backup strategy, integration tests. For a team modernizing 30 bounded contexts, that is 30 services to operate during the migration period — when the team is *also* keeping the legacy mainframe running, *also* learning the modern stack, *also* discovering that the bounded contexts they drew on a whiteboard need refinement once they encounter the real code.

The deeper failure is decomposing before the boundaries have earned their independence. A bounded context whose boundary is provisional should not be a service yet; it should be a module inside a larger deployable unit, where boundary changes are cheap. Premature service extraction freezes provisional boundaries into operational reality.

## Forces

The target architecture may genuinely be distributed services — that's the modernization's eventual destination. But the migration period needs different optimization criteria than the destination. During migration: minimise operational overhead, maximise boundary flexibility, accelerate learning about which boundaries are real. After migration: optimise for independent scaling, independent deployment, independent team ownership where those properties are needed.

The modular monolith resolves the migration-period forces. Bounded contexts are explicit — separate modules, separate vocabularies, separate data ownership within the deployable — but they share a single deployment, a single observability surface, a single database (with schema separation), a single set of operational concerns. Boundary changes are cheap because they don't cross network boundaries. Operational overhead is minimal because there is one thing to deploy.

When a bounded context's boundary stabilises and its scaling/deployment/ownership profile diverges from the rest, *then* it earns extraction into its own service. The extraction is informed by operational evidence, not by architectural aspiration.

## Pattern

Default to the modular monolith as the transitional architecture for mainframe modernization. Bounded contexts are first-class within the monolith — each has its own module, its own ubiquitous language, its own data ownership (enforced through schema separation or row-level ownership rules), its own aggregate roots. Communication between contexts happens through commands and events (Pattern 12), with the same logical contracts that a distributed system would use. The contracts survive any future extraction.

Within the monolith, the boundaries are enforced at compile time: modules declare their public interfaces; cross-module access goes through these interfaces; direct access to another module's internals is impossible. This compile-time enforcement is stronger than runtime enforcement (which can be bypassed through reflection or workarounds) and lighter than network enforcement (which requires actual services).

Operational concerns are unified during the migration. One deployment pipeline, one observability stack, one database (with schema separation), one set of integration tests. The team's operational attention is spent keeping the legacy mainframe running and learning what the modern stack actually requires — not on managing 30 services.

Extraction comes later, when specific bounded contexts have earned their independence. The extraction criteria are operational:

- **Scaling profile diverges** — this bounded context's traffic shape is different from the rest, and shared scaling produces waste (over-provisioning the quiet contexts, under-provisioning the busy one).
- **Deployment cadence diverges** — this bounded context changes frequently while others are stable, and shared deployment produces unnecessary risk for the stable contexts.
- **Team ownership diverges** — this bounded context belongs to a team that wants independent deployment authority, separate from the modernization team's release cycle.
- **Compliance boundary diverges** — this bounded context has regulatory or security requirements distinct from the rest, and shared deployment confuses the audit boundary.

When one of these criteria applies, the bounded context is extracted into its own service. The extraction is straightforward because the contracts already exist — commands and events flow over network instead of through in-process method calls, but the logical boundary is unchanged. Pattern 12 (*Commands and Events as Logical Boundary*) makes this extraction mechanical, not architectural.

The modular monolith is also what the agentic platform (Pattern 19) is. Bounded MCP servers are modules within a single platform, not separate services. The recursion is deliberate: the discipline that works for the modernized business system works for the platform that produces it.

The modular monolith is one transitional shape; other shapes are sometimes appropriate within the same modernization. Nick Tune has documented two complementary forms: the *Bubble*, in which a new subsystem with a clean target model accesses legacy data exclusively through an anti-corruption layer without local persistence; and the *Autonomous Bubble*, in which the new subsystem holds a local data store populated asynchronously from legacy events. The modular monolith is the right default when the modernization owns both code and data; the Bubble forms are useful when the target model can be designed independently but the underlying data must remain in the legacy during the transition. Definitions and pointers are in the glossary.

## Consequences

Operational complexity during migration is minimised. The team can focus on the modernization work itself — slice discovery, scaffold generation, behavioural verification — without simultaneously absorbing the cost of operating distributed systems. The migration completes faster because fewer things compete for operational attention.

Boundary refinement is cheap. As the team discovers that a bounded context drawn on a whiteboard needs to be split, or that two contexts should be merged, or that an aggregate boundary needs to move, these changes are local refactorings inside the monolith — not cross-service migrations with their attendant data movement and contract negotiation.

The path to distributed services remains open. Bounded contexts that earn their independence are extracted with minimal architectural disruption because the contracts are already explicit. The modular monolith is not a permanent destination; it is a way station that preserves optionality.

The cost is the discipline of resisting the fragmentation instinct. Teams trained in cloud-native architectures may feel that the modular monolith is "not modern enough" — that microservices are what 2026 looks like. This is wrong for migration contexts but socially difficult to articulate. The catalog must make the case explicitly, repeatedly: distributed services are an *outcome* of validated boundaries, not a *starting point*.

The discipline also requires that the modular monolith be built correctly. A monolith with all modules sharing a database freely is not modular; it is a big ball of mud with hopeful naming. Schema separation must be enforced, public interfaces must be the only cross-module access path, the compile-time enforcement must actually compile.

Sam Newman's *Monolith to Microservices* (Newman, 2019) provides the canonical articulation of incremental extraction strategies. Simon Brown's modular monolith advocacy (Brown, multiple talks 2017-2023) names the architectural pattern this catalog applies. What this catalog contributes is the framing of the modular monolith specifically as the *transitional architecture for mainframe modernization* — where the migration period's operational pressures are extreme and the boundary uncertainty is high, both of which favour deferred decomposition. The principle (don't fragment

before boundaries earn independence) generalises beyond mainframe; the specific application to long-running brownfield mainframe migrations is what's articulated here.

## Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines which contexts will eventually need independent extraction. Pattern 10 (*Tier-Aware Scaffolding*) operates within the modular monolith — tier-2 and tier-3 contexts may eventually be extracted; tier-0 and tier-1 may remain modules permanently. Pattern 11 (*Pluggable Emitters*) renders each module's scaffold; extraction changes the deployment target but not the emitter. Pattern 12 (*Commands and Events as Logical Boundary*) is what makes future extraction mechanical — the logical boundary survives the physical change. Pattern 19 (*Bounded MCP Servers*) applies the same discipline to the agentic platform itself. The *Frozen Architecture* antipattern names what happens when modernization adopts the legacy's accidental decomposition rather than designing the right transitional architecture.

---

# Pattern 15: Architecture Documentation as Pluggable Emitter

*Status: next.*

## Context

A modernization that has produced a working system from canonical substrates: the structural graph (Pattern 3), the intermediate representation (Pattern 9), the domain ontology (Pattern 5), the source provenance discipline (Pattern 4) that ties everything back to legacy code. The agents have done their work. But humans — architects reviewing strategic design, data engineers validating ER models, DDD practitioners checking aggregate boundaries, business analysts reading context maps, operators tracing data flows, regulators auditing decisions — need to *understand* the system at multiple levels of abstraction. Each constituency wants a different view of the same architecture.

## Problem

Hand-authored documentation drifts. The diagrams produced for kickoff are inaccurate by month three; the wiki page that explained the architecture is stale by month six; the C4 model the architect drew on the whiteboard exists only in someone's screenshot folder. When the system changes — and it always changes — the documentation lags or fragments until the gap between system and documentation becomes unbridgeable, and the team gives up.

Auto-generated documentation from code has a different problem: it produces documentation at the wrong level of abstraction. Class diagrams that mirror code structure are useless for strategic understanding. Sequence diagrams generated from method calls capture mechanics but not intent. The strategic constituencies need views that exist *above* the code, not below it.

The deeper issue is treating documentation as a separate artifact from the system. As long as documentation lives in a parallel track — written by hand, stored separately, maintained through discipline alone — it will drift. The only sustainable answer is for documentation to share its source of truth with the system itself.

## Forces

Multiple constituencies each want a different view of the architecture. Each view changes when the architecture changes. Manual maintenance is unsustainable across constituencies and over time.

Auto-generation from code produces views at the wrong abstraction level for most constituencies. The substrates that produced the code (graph, IR, ontology) already encode the strategic and tactical decisions — but they encode them as data, not as visual or narrative documentation.

Each constituency has canonical formats they expect: C4 for software architects (Simon Brown's notation), ER diagrams for data engineers, aggregate maps and context maps for DDD practitioners (Eric Evans, Vaughn Vernon), data flow diagrams for systems analysts. Documentation that doesn't speak the constituency's standard format adds friction even when the content is correct.

## Pattern

Treat architectural documentation as a rendered view of the canonical substrates, not as an authored artifact maintained in parallel. Each documentation view has its own emitter that consumes some combination of graph, IR, and ontology, and produces the appropriate representation. Documentation regenerates from substrates the same way scaffolds regenerate from the IR. The documentation is always current because it is never authored — it is always rendered.

The principle generalises Pattern 11 (*Pluggable Emitters*) from code targets to documentation targets. The same compiler architecture that allows a vertical-slice emitter and a hexagonal emitter to consume the same IR allows a C4-diagram emitter and an ER-diagram emitter to consume the same substrates. The emitters are different; the substrates are the same.

Specific views the catalog of documentation emitters can produce:

- **C4 model views** — rendered from bounded-context boundaries (graph community detection), deployment intent (IR), and external system relationships.
- **Entity-relationship diagrams** — rendered from data structures in the graph and aggregate boundaries in the IR; one ER diagram per bounded context.
- **Aggregate maps** (DDD-specific) — rendered from IR aggregate definitions and graph relationships; shows aggregate roots, contained entities, value objects, emitted events.
- **Context maps** (DDD-specific, Evans 2003) — rendered from bounded-context relationships in the graph and command/event flows in the IR; shows upstream/downstream relationships, anti-corruption layers, shared kernels, integration patterns.
- **Data flow diagrams** — rendered from side-effect surfaces in the graph and command flows in the IR.
- **Ubiquitous language glossaries** — rendered from the domain ontology (Pattern 5) and IR vocabulary; one glossary per bounded context.
- **Architecture Decision Records (ADRs)** — linked from spec deltas (Pattern 26); when a spec delta articulates an architectural change, an ADR is produced as part of the change.

The documentation lives in markdown and standard diagram formats (PlantUML, Mermaid, SVG) so it integrates with the engineering workflow — versioned in Git alongside the code, viewable in pull requests, included in pipelines. Each view is a file generated by its emitter; regenerating is idempotent and fast.

When humans need to *change* the architecture, they don't edit the documentation directly. They edit the substrates — refining the ontology, updating the IR, adjusting the graph annotations — and the documentation re-renders to reflect the change. This is what Cyrille Martraire (*Living Documentation*, 2019) called the discipline of making documentation a byproduct of doing the work. The catalog operationalises that discipline through the same compiler architecture that produces

the code.

---

**[FIGURE] 15.1 — *Documentation as rendered view: many constituencies, one source of truth***

*Diagram showing the substrates at the center feeding multiple documentation emitters in parallel. Center: three substrates — graph (Pattern 3), IR (Pattern 9), ontology (Pattern 5) — depicted as a unified canonical core. Radiating outward: documentation emitter boxes, each producing its own view: "C4 model" (architects), "ER diagrams" (data engineers), "Aggregate map" (DDD practitioners), "Context map" (DDD practitioners), "Data flow" (systems analysts), "Ubiquitous language glossary" (everyone), "ADRs" (governance). Each emitter shows a small thumbnail of its output style. Annotations: "documentation never authored — always rendered" near the center, and small annotations near each constituency reading "wants this view." Style: hub-and-spoke matching FIGURE 11.1 to make the parallel between code emitters and documentation emitters visible. Goal: reader sees that the same compiler discipline applies to both.*

---

## Consequences

Documentation never goes stale relative to the system it describes. Reviewers validate decisions at the abstraction level appropriate to them: architects check the C4 model, data engineers check the ER diagrams, DDD practitioners check aggregate and context maps. Each constituency engages with documentation that is canonical for them, generated from the same source of truth that produced the code.

The substrates themselves become the canonical "documentation" — diagrams and markdown documents are convenience views rendered for human consumption. This inverts the traditional relationship: substrates are not artifacts derived from documentation; documentation is artifacts derived from substrates.

Documentation produced for one engagement becomes reusable across engagements. The catalog of emitters compounds across customers and bounded contexts.

Reviewers in the cockpit (Pattern 25) can request a specific view to validate a decision. Spec deltas (Pattern 26) include the documentation re-renders as part of the change package; reviewers see what the architecture looked like before, what it looks like after, and which substrate changes produced the difference.

The cost is the emitter library. Each view type requires its own emitter; emitters need maintenance as substrate schemas evolve. Some emitters are straightforward (ubiquitous language glossary from ontology); others are subtle (C4 component-level views require heuristics about what counts as a component versus a container). The engineering effort amortises across engagements in a way that hand-authored documentation does not.

This pattern is *next* in Rosetta. The substrates exist; what's missing is the documentation emitter library. The principle follows directly from Pattern 11 extended from code targets to documentation targets — the architecture supports it, and the gap is in the catalog of emitters, not in the underlying approach.

## Related patterns

Pattern 3 (*The Graph as Projection*), Pattern 5 (*Domain Ontology as Independent Substrate*), and Pattern 9 (*The Intermediate Representation*) are the substrates this pattern's emitters consume. Pattern 11 (*Pluggable Emitters*) is the architectural principle this pattern extends from code to documentation — same compiler discipline, different output. Pattern 25 (*The Cockpit*) is where rendered documentation surfaces for human consumption. Pattern 26 (*Spec Deltas as the Unit of Review*) integrates documentation re-renders into change packages.

---

# Part III: Verification

---

**[ILLUSTRATION] P3 — *Verification: the twin in the lab***

*Style: editorial illustration matching the cover's aesthetic. Subject: a watchmaker's workbench with two identical mechanical timepieces side by side, one labelled with subtle "legacy" markings (older brass, more wear), the other "modernized" (cleaner finish but identical movement). Both being checked against a single chronograph. The chronograph's needle is steady — they match. Visual metaphor: two systems, identical behaviour, one source of truth. A magnifying glass reveals that the underlying mechanisms are different but the output is the same. Format: full-page or two-thirds page opener for Part III.*

---

The patterns in this group address how the modernization knows the generated code is right. They presuppose Pattern 2 (*The Legacy as Oracle*) — without a live ground truth to compare against, verification reduces to test suites that the agents wrote.

---

# Pattern 16: Twin Verification

*Status: working.*

## Context

An agentic workflow generating C# code from COBOL, paragraph by paragraph, slice by slice. The agents need to know whether each candidate they produce is correct — behaviourally equivalent to the original legacy paragraph it replaces. Verification has to be fast enough to be part of the agent's inner loop (the iteration cycle in which agents produce, evaluate, refine, repeat), not a downstream activity that happens after the agent has committed to a candidate. The legacy is available as oracle (Pattern 2).

The inner loop is where most of the work happens. An agent typically produces multiple candidates per paragraph — initial translation, refinement after first verification, response to divergence diagnostics, alternative approaches if initial ones fail. Each candidate needs verification. If verification is slow, the agent's exploration is constrained by latency; if verification is fast, the agent can explore aggressively.

## Problem

When verification is slow (running against a remote mainframe, running through a CI pipeline, running against a shared test environment with queueing), the agents can't iterate effectively. They produce a candidate, wait minutes for verification, get a verdict, try again. Most of the wall-clock time is spent waiting. The quality of the iteration suffers because each step is expensive — the agents conserve attempts, miss exploratory opportunities, settle for early candidates that pass rather than refined candidates that pass cleanly.

There is also a correctness problem. When verification is slow, teams cut corners: they verify against test suites the agents themselves wrote (creating the homework-grading problem Pattern 2 addresses), or they verify against sampled inputs rather than representative inputs, or they skip verification at intermediate stages and verify only at the end. Each shortcut reduces confidence in the final result.

## Forces

Real verification against the actual legacy mainframe environment is authoritative but slow and expensive — network latency to the mainframe, cost of mainframe cycles, queueing for shared

resources, security and access restrictions. Synthetic verification (test suites, mocks, fixtures) is fast but disconnected from ground truth — it tests what the team thought to test, not what the legacy actually does.

Local approximations of the legacy can be both fast and authoritative if the approximation is faithful enough. The faithfulness requirement is strict: the local oracle must produce outputs indistinguishable from the legacy's for the inputs being exercised, or the verification verdicts are unreliable. A local approximation that's 99% faithful is worse than no local approximation, because the agents trust it.

## Pattern

Compile the legacy to a portable runtime form and package it as a local container — the Legacy Twin. Run the candidate C# and the Legacy Twin against the same inputs. Compare outputs in-process. Treat any divergence as failure until proven otherwise.

Make the loop fast — milliseconds, not minutes — by keeping everything local: the Twin runs in a Docker container on the same machine as the agent; inputs come from local fixtures; comparison happens in-memory; verdicts return synchronously. The agent generates a candidate, the candidate runs against the Twin, the verdict is immediate, the agent has direct evidence of what's right and what's wrong.

The Twin's faithfulness is non-negotiable. The compilation from legacy to portable form must preserve the legacy's exact semantics — every edge case, every error path, every transactional guarantee, every type coercion. The Twin is not an emulation; it is the legacy code running in a different runtime environment. Its outputs are the legacy's outputs.

Divergence diagnostics are first-class. When the candidate's output differs from the Twin's, the system doesn't just report "FAIL" — it reports a structured diff: which field diverged, what the candidate produced, what the Twin produced, what input caused the divergence, what provenance trail (Pattern 4) leads back to the legacy code that defines the expected behaviour. The agent uses the diagnostic to refine the candidate; the diagnostic becomes the next input to the inner loop.

In Rosetta, the Twin is Raincode-compiled COBOL packaged as a Docker container. Raincode is one of the few tools that compiles COBOL to .NET IL with sufficient fidelity to make this approach practical. The implementation is specific to CICS COBOL; the principle (local containerised oracle in the inner loop) is more general — any legacy stack with a faithful-enough portable compilation target can apply it.

---

**[FIGURE] 16.1 — *Twin Verification: the inner-loop comparison cycle***

*Diagram showing the Twin Verification cycle as a fast, tight loop. Components: (1) candidate C# code emitted by agent; (2) Legacy Twin (Raincode-compiled COBOL container); (3) shared input fixture feeding both; (4) parallel execution boxes; (5) outputs converging into a comparison node; (6) verdict (PASS/FAIL with diff details) returning to the agent. Time annotation: "milliseconds" near the verdict path. Inset showing what divergence detection produces — a structured diff showing field-by-field mismatch with source provenance. Style: tight cyclical flow diagram, accent color for the verdict. Goal: reader sees the inner-loop economics — fast verification changes how aggressively the agent can explore.*

---

## Consequences

The agents iterate against evidence in real time. The cost of being wrong drops, which lets the agents explore more aggressively — try multiple translations, evaluate alternatives, refine until the result is good rather than settling for the first thing that passes. The cost of being right stays the same, which keeps verification honest. The agents converge faster and more honestly because they're matching behaviour the legacy actually exhibits, not behaviour someone wrote down.

Behavioural equivalence becomes the verification standard. Tests check what we thought to test; behavioural equivalence checks the actual output for whatever inputs we exercise. When you have the legacy as oracle, you don't need to imagine in advance what the edge cases are — the Twin exhibits them when exercised.

The cost is the Twin itself. Raincode is one of the few tools that can compile COBOL to .NET IL well enough to make this practical. For other legacy stacks, equivalent tooling may or may not exist; without faithful compilation, the pattern doesn't apply. The Twin must be kept in sync with the legacy if the legacy evolves during the modernization. The container must be packaged, distributed, and maintained — operational concerns that don't exist if verification runs only against the original mainframe.

There is also a coverage caveat. Twin Verification confirms behavioural equivalence for inputs the agents exercise; it does not exhaustively prove equivalence for all possible inputs. Production traffic includes inputs no dev-time corpus anticipated. This is why Pattern 17 (*Hypothesis-Driven Verification*) extends Twin Verification from dev mode into production — production becomes the corpus that catches what dev-time exercise missed.

## Related patterns

Pattern 2 (*The Legacy as Oracle*) is the foundational principle this pattern operationalises — the legacy is the source of truth, and the Twin is how the agents access that truth in the inner loop. Pattern 17 (*Hypothesis-Driven Verification*) extends Twin Verification from dev mode to production mode using Witness telemetry. Pattern 23 (*The Harness as Self-Observing State Machine*) treats Twin Verification as a deterministic gate the agents must pass before promotion. Pattern 4 (*Source Provenance Discipline*) is what makes divergence diagnostics traceable back to specific legacy code.

---

# Pattern 17: Hypothesis-Driven Verification

*Status: in progress.*

## Context

A modernization that has succeeded against dev-time verification (Pattern 16) and is moving to production. In dev mode, the candidate C# matches the Legacy Twin's behaviour over the test corpus. The question is whether it will match in production, where the corpus expands to whatever traffic the system actually receives.

## Problem

Dev-mode verification covers what was exercised. Production reveals what the dev-time corpus didn't anticipate: temporal coupling, end-of-month patterns, edge cases triggered by specific data shapes, environmental dependencies. The behaviour is real, the legacy handles it correctly, and the dev-mode verification has no way to know about it.

Charity Majors has argued for years that you can't stage your way to production knowledge. Distributed systems are hostile to mirroring. What production reveals isn't in the tests or the spec — it's in the running system itself.

## Forces

Production verification needs the same tight feedback as dev-time verification but operates on a different scale and at different stakes. The dual-run period — during which legacy and modernized C# both process real traffic — is finite; eventually one is decommissioned. Whatever the modernization learns about production must be captured before that decommissioning, or it's lost.

The traditional approach (write more tests after production reveals issues) is reactive. By the time tests are written, the divergence has already happened in production.

## Pattern

Deploy a verification framework alongside the modernized C# in production. Capture legacy behaviour on real traffic. Compare it to the modernized C#'s behaviour on the same inputs. Record

divergences as evidence. Replay captured scenarios in dev mode to refine the modernized code.

Distinguish three kinds of drift: *semantic drift* (the modernized C# produces a genuinely different result), *architectural artifact* (the new architecture handles something differently than the legacy did, but the result is equivalent in business terms), and *temporal artifact* (the test fixture in dev mode didn't capture a time-dependent input). Each kind requires different action.

In Rosetta, this framework is called Witness. Witness is the production-mode counterpart to Twin Verification. Specialist observer agents deploy alongside the modernized C# as part of Witness, watching, surfacing anomalies, capturing scenarios. The capture-replay lineage extends from a project I built in 2016 (pmilet/playback, an open-source HTTP capture-replay middleware), now reframed for the agentic era.

---

**[FIGURE] 17.1 — *Witness: production-mode verification across the dual-run period***

*Diagram showing the dual-run architecture during cutover. Top: production traffic flowing in. Middle, side-by-side: the legacy system (still authoritative) and the modernized C# (running alongside). Both process the same traffic. Outputs flow to a comparison layer (Witness). Witness surfaces three kinds of divergence to a feedback panel: "semantic drift" (red), "architectural artifact" (yellow), "temporal artifact" (blue). Captured scenarios feed a fixture library that flows back to dev-mode (Twin Verification, Pattern 16) for replay. Annotation: "production teaches what dev-time corpus couldn't anticipate." Style: production-deployment diagram with the comparison layer visually elevated. Goal: reader sees the Witness/Twin pairing and how production knowledge re-enters the dev loop.*

---

## Consequences

Production becomes a source of evidence rather than a source of incidents. The modernization continues learning past the cutover. SME knowledge that was tacit becomes documented as a byproduct of verification.

The cost is operational complexity. Witness must be deployed in production, which requires the platform engineering to support it (described in Patterns 14 and 16). The drift typology must be operationalised — distinguishing semantic from architectural from temporal drift requires discipline that the verification framework can't fully automate. Some divergences will always require human judgement.

Whether Witness works at scale is the prototype's most uncertain bet. The principle (production reveals what dev mode hides, and that revelation should feed back into the modernization) is sound; whether the operational machinery to capture it efficiently can be built is what's being tested.

## Related patterns

Pattern 2 (*The Legacy as Oracle*) is the foundational principle, now extended into production. Pattern 16 (*Twin Verification*) is the dev-mode counterpart. Pattern 18 (*Behavioural Specifications Grown from Production*) is the next step that builds on the captures Witness produces. Pattern 25 (*The Cockpit*) is where humans review captured divergences and approve action.

# Pattern 18: Behavioural Specifications Grown from Production

*Status: next.*

## Context

A modernization where production-mode verification (Pattern 17) is capturing real legacy behaviour as evidence. The captures accumulate over time, building a corpus of how the system actually behaves under real workloads. The modernized system inherits this corpus; the legacy will eventually be decommissioned. The question becomes what to do with the captured evidence as a long-term artifact.

## Problem

The traditional approach to specifications is to write them before development: someone (an analyst, a domain expert, a product manager) articulates what the system should do, and developers implement it. In legacy modernization this is doubly broken. The original specification was never written; whatever specs exist are reconstructions. And the modernized system is replacing something that already runs, which means the running system itself is more authoritative than any reconstruction.

The captured production behaviour is rich evidence, but it's not yet a specification. It's traffic patterns, request-response pairs, scenarios. To become a specification it needs to be articulated in a form that can drive ongoing engineering — readable by humans, executable as tests, traceable to the requirements they encode.

## Forces

Manually translating production captures into specifications is expensive and arbitrary — different humans would produce different specs from the same captures. Skipping the translation altogether leaves the captures as evidence but not as contracts; they don't survive as a specification once the legacy is gone. Asking an LLM to generate specifications from raw captures produces inconsistent results because the captures themselves don't carry intent.

The captured behaviour does carry intent implicitly — recurring traffic shapes, repeated transformations, consistent pre-conditions. The intent is recoverable if the captures are processed with

discipline.

## Pattern

Process the captured behaviour to produce executable behavioural specifications. Pattern-match recurring traffic shapes across captures. Extract the preconditions that lead to each shape. Extract the outcomes that follow. Generate Given-When-Then specifications grounded in what the system actually did, not in what someone wrote in a planning meeting.

The specifications are executable: they become a regression suite that the modernized system runs against. They are traceable: each specification points back to the captures that grounded it. They are reviewable: humans can read them, validate them against domain understanding, refine them where the captures were ambiguous.

The regression suite grows from what the system actually does. Tacit SME knowledge — the kind that lives in domain experts' heads and is never formally written down — becomes externalised as a byproduct of verification.

## Consequences

The modernization produces a durable artifact that survives the legacy decommissioning: a behavioural specification grounded in production reality. Future development against the modernized system has executable contracts to validate against. Future modernizations of adjacent systems have a precedent to follow.

The cost is the disciplined processing of captures. Pattern-matching at scale requires technique — naive grouping produces too many specifications or too few, and the threshold isn't obvious. Generated specifications need human review because the captures may include noise, errors, or behaviour that shouldn't survive into the modernized system. The quality of the resulting specification depends on the quality of the capture corpus, which depends on Witness running long enough across enough traffic shapes.

This is a direction designed but not yet built. The principle (specifications can grow from production rather than precede it) is consistent with the prototype's broader posture (the legacy is the spec). What remains to be tested is whether the operational machinery to make this efficient can be built, and whether the resulting specifications are useful enough to justify the engineering cost.

## Related patterns

Pattern 17 (*Hypothesis-Driven Verification*) is what produces the captures this pattern processes. Pattern 26 (*Spec Deltas as the Unit of Review*) is what consumes the resulting specifications when they change. Pattern 2 (*The Legacy as Oracle*) is the foundational principle this pattern extends — the spec doesn't precede the system; it emerges from it.

---

# Part IV: Governance

---

**[ILLUSTRATION] P4 — *Governance: the harness, not the leash***

*Style: editorial illustration matching the cover's aesthetic. Subject: a horse-drawn carriage where the horse (powerful, fast, capable) is depicted with elaborate harness — guides on the bridle, traces connecting to the carriage, blinkers framing the field of view. The driver's hands hold the reins lightly. The harness shapes the horse's path without restricting its capacity for work. Visual metaphor: agentic systems thrive within structured constraints, not loose framing or restrictive control. The harness is the architecture that makes intelligent power directable. Format: full-page or two-thirds page opener for Part IV.*

---

The patterns in this group address how the modernization stays coherent across the agentic system. They presuppose verification (Part III) and generation (Part II) — without a working pipeline, governance has nothing to govern.

---

# Pattern 19: Bounded MCP Servers

*Status: working.*

## Context

An agentic system for software engineering, composed of multiple capabilities that the agents need to invoke: graph queries, legacy oracle invocation, scaffold generation, verification execution, telemetry recording, hypothesis testing. The capabilities have distinct concerns and could be implemented independently, but the agents need coordinated access to all of them.

The Model Context Protocol (MCP) — introduced by Anthropic in 2024 — provides a standardised way for AI agents to discover and invoke capabilities through declared tool interfaces. An MCP server exposes a set of tools with typed inputs and outputs; agents discover those tools, decide which to call, and receive results back through the protocol. MCP is the substrate this pattern builds on; what's articulated here is how to decompose capabilities across MCP servers, not the MCP protocol itself.

## Problem

The default approach is to give a single agent broad tool access — direct file system access, direct database access, direct shell commands. This works for small systems and fails at scale.

Cross-cutting access produces unauditable behaviour: when something goes wrong, there's no way to localise the failure because the agent could have touched anything. Implementations become coupled because any agent can reach into any subsystem; replacing one capability requires understanding how every agent interacts with it. Security boundaries collapse: granting an agent access to "the system" means granting it access to everything. Cost accounting becomes impossible: telemetry can't attribute work to specific capabilities because the work is scattered.

The deeper issue is that the agentic system itself is a domain that hasn't been modelled. Without bounded contexts inside the agentic platform, the platform is a monolith with broad surface area — the same architectural smell DDD has been correcting in business systems for two decades.

## Forces

The agents need access to multiple capabilities. The capabilities need to evolve independently — a new graph analysis pass should not require updating every agent that uses graph data. The

system needs to be debuggable and auditable. Security and access control need to be enforceable per capability. Cost and performance need to be attributable.

These pressures pull in different directions. Tight coupling between agents and capabilities makes the agents simpler but the system fragile; loose coupling makes the system robust but requires explicit contracts everywhere. The contracts themselves have to be expressive enough to support agent needs without exposing internals — a hard balance.

## Pattern

Apply bounded contexts to the agentic system itself. Expose each capability through a specialised MCP server with a single, coherent responsibility. The agents access capabilities through server interfaces; they don't reach into implementations directly.

Each MCP server has its own ubiquitous language, its own data model, its own consistency boundary. The Discovery server speaks the language of graphs, ontologies, slices, communities. The Twin server speaks the language of scaffolds, paragraphs, candidates, verification fixtures. The vocabularies don't bleed across boundaries — when the Twin server needs graph information, it queries Discovery's tools and receives data in Discovery's terms, which the Twin server then translates into its own vocabulary internally.

In Rosetta, four MCP servers structure the agentic platform: a Discovery server owns the graph layer (graph queries, ontology lookups, slice discovery); a Legacy server owns the Legacy Twin (oracle invocation, behavioural verification, divergence diagnostics); a Twin server owns the C# generation pipeline (scaffold rendering, paragraph translation, IR construction); a Witness server owns the verification lifecycle (hypothesis generation, production-mode verification, certification). Each server's implementation is private; agents see only the tools the server exposes.

Cross-server interaction follows the same disciplines DDD applies to bounded contexts. When one capability needs another's data, it goes through the public interface, not through shared databases or back-channel access. The integration patterns are explicit: published events (when one server's work emits state changes others consume), conformist consumers (when one server simply accepts another's output as canonical), anti-corruption layers (when one server needs to translate another's vocabulary into its own internal model). The Legacy server is itself an instance of what Nick Tune has called *Expose Legacy Asset* (see glossary): the legacy is wrapped in an explicit interface that modernized subsystems consume, rather than being accessed directly through database connections or in-process coupling.

---

**[FIGURE] 19.1 — *Bounded MCP servers: bounded contexts for the agentic system***

*Diagram showing the four MCP servers as bounded contexts, each with its own boundary. Each server depicted as a box containing: server name, owned capability, owned data, exposed tools. Specifically: Discovery server (owns graph + ontology, exposes graph queries and slice discovery); Legacy server (owns Legacy Twin container, exposes oracle invocation); Twin server (owns scaffold + agent translation pipeline, exposes scaffold generation and code production); Witness server (owns verification lifecycle, exposes hypothesis generation and certification). Above the servers: the orchestrating Cortex agent (Pattern 20) reaches across servers via their MCP interfaces. Below the servers: implementations stay private. Annotations: "interface = contract" at each server boundary. Style: bounded-context diagram with strong visual boundaries between*

*servers. Goal: reader sees the agentic system as a domain composed of bounded contexts, not a monolith with broad agent access.*

---

## Consequences

Implementations evolve independently — a new graph analysis pass changes the Discovery server without disturbing Twin or Witness. Audit trails are clean because every agent action goes through a server boundary and is recorded with structured input/output. Failures localise: when something goes wrong, the audit shows which server saw which inputs and produced which outputs, and the investigation starts at the relevant boundary instead of tracing through unbounded agent activity.

Security becomes per-capability: an agent that needs graph queries gets credentials to the Discovery server only, not blanket access. Cost attribution becomes possible: each server reports its own resource usage, and the cost of a modernization can be decomposed by capability. Performance debugging becomes tractable: latency profiles are localised to specific servers, and bottlenecks are identifiable.

The cost is contract design. Each server's interface must be expressive enough to support the agents' needs without exposing internals. Cross-cutting concerns — orchestration, policy enforcement, end-to-end telemetry — live above the servers (Pattern 20), not within them. The servers themselves can't bypass each other's contracts, even when bypassing would be convenient.

The agentic system, structurally, is a modular monolith of bounded servers — same DDD discipline applied to the agentic platform itself that this catalog applies to the modernized business system. The recursion is not coincidental: bounded contexts are how complex systems remain comprehensible, whether the system is a bank's core ledger or an AI platform for legacy modernization.

My guess is that this pattern will become more common as the field matures. The current default of giving a single agent broad tool access has clear limits at scale, and the discipline of bounded MCP servers is the natural correction.

## Related patterns

Pattern 20 (*The Orchestration Layer Above Bounded Capabilities*) is what coordinates across the bounded servers. Pattern 23 (*The Harness as Self-Observing State Machine*) governs *what* agents do; this pattern governs *how* they do it — which capabilities they can reach and through which interfaces. Pattern 25 (*The Cockpit*) is the human-experience surface above the MCP layer, surfacing what each server has done and what's pending. Pattern 24 (*Reasoning Telemetry as First-Class Output*) is what each server emits about its own decisions, which makes the audit trail end-to-end.

---

# Pattern 20: The Orchestration Layer Above Bounded Capabilities

*Status: working.*

## Context

An agentic system structured around bounded capabilities (Pattern 19), where each capability has a coherent responsibility and a defined interface. The agents need to accomplish goals that span capabilities — discovering bounded contexts in the graph, then using those contexts to scaffold C#, then verifying the scaffolds against the Legacy Twin, then refining based on the verdict. No single capability owns this end-to-end work.

The end-to-end work is itself a domain — the workflow of modernization. The workflow has its own ubiquitous language (slices proposed, scaffolds rendered, candidates verified, gates passed, decisions escalated), its own state, its own evolution over time. It is not part of any single capability's responsibility, but it touches all of them.

## Problem

Without an explicit orchestration layer, the agents themselves end up coordinating across capabilities. This produces brittle behaviour: every agent has to know which capabilities exist, which order to invoke them in, what to do when one capability's output needs to be reshaped before another can consume it. Cross-capability coordination becomes scattered through the agent population, which makes the overall flow opaque and hard to govern.

The instinct to fold orchestration into one of the bounded capabilities — making the Discovery server, say, responsible for invoking Twin and Witness — violates the bounded-context principle. The capability that orchestrates is no longer a bounded context; it's a god object dressed up as a server. Its vocabulary expands to include other capabilities' vocabularies; its state expands to track other capabilities' states; its responsibility expands to include other capabilities' decisions. The bounded discipline that Pattern 19 establishes collapses if any single server absorbs orchestration.

There is also a choice between orchestration and choreography that the catalog needs to make. Choreography (capabilities reacting to events from each other without a central coordinator) is appealing because it preserves bounded discipline, but it makes the overall flow harder to reason about and harder to gate with human approval. Orchestration (a coordinating agent that drives

the workflow) is more legible and more governable, at the cost of introducing a coordinator that must be carefully constrained.

## Forces

Coordination is inherently cross-cutting. It can't be eliminated. But where it lives matters: scattered across agents, it's chaotic; folded into a bounded server, it corrupts that server's coherence. There needs to be a place for cross-cutting work that doesn't compromise either the agents or the bounded servers.

The orchestrator must coordinate without bypassing. If it accesses capabilities directly — through database connections, through internal APIs — the bounded discipline of Pattern 19 collapses. The orchestrator's power must come from broader scope, not from privileged access. It sees the whole workflow but it interacts with capabilities through the same MCP interfaces other agents use.

The orchestrator must also accommodate human-in-the-loop gates. Not every cross-cutting decision is automatable — some require human judgment (validating a hypothesis, approving a scaffold, accepting a divergence as intentional). The orchestrator routes work to humans through the cockpit (Pattern 25) and waits for the response, the same way it routes work to capabilities and waits for their response.

## Pattern

Place an orchestrating agent above the bounded capabilities, with explicit responsibility for cross-cutting decisions. The orchestrator coordinates without bypassing — it accesses bounded capabilities only through the same interfaces other agents use, but it has the broader view that lets it make decisions about sequencing, escalation, and revision.

In Rosetta, this orchestrator is called Cortex. Cortex makes decisions like: when to escalate to a human, when to switch focus from one bounded context to another, when to revisit prior decisions in light of new evidence, when one capability's output needs reshaping before another can consume it. Cortex runs a retrospective sub-agent that learns across modernization sessions, accumulating patterns about which sequences work well and which often produce escalation, feeding those patterns back into future workflow decisions.

Cortex maintains the workflow state — what slice is currently being worked on, what stage it's at, what gates remain, what humans are waiting for. The state lives in durable workflow infrastructure (Pattern 21), surviving process restarts and human absences. Other agents query Cortex about workflow state; Cortex queries them about capability state.

The orchestrator is itself an agent, not a service. Its behaviour is bounded by the harness (Pattern 23), observed through the cockpit (Pattern 25), and emits reasoning telemetry like any other agent (Pattern 24). It coordinates the others, but it doesn't escape the governance that applies to the whole system. The orchestrator's authority comes from its scope, not from privileged status.

## Consequences

Cross-cutting decisions have a place. Agents working in bounded capabilities focus on their own work; the orchestrator handles the coordination. The system as a whole becomes legible — there's

a specific place to look for "why did the work move from this bounded context to that one," and the answer is in the orchestrator's decision log.

Workflow coherence becomes a queryable property. At any moment, the orchestrator can answer: which slices are in flight, which have stalled, which are blocked on humans, which are blocked on capabilities. This visibility makes the cockpit's job tractable — the cockpit doesn't have to reconstruct the workflow from scattered evidence; it queries the orchestrator.

The cost is the orchestrator's own complexity. It has to know enough about every bounded capability to coordinate them, but it must not duplicate their internals. This is a delicate balance — orchestrators that grow too smart start to absorb logic that belongs in the capabilities; orchestrators that stay too simple end up unable to coordinate effectively. Maintaining the right level of abstraction in the orchestrator is ongoing work, often informed by what the harness's self-observation (Pattern 23) surfaces about which decisions the orchestrator gets right and which it doesn't.

Whether to also use a hosted orchestration framework — to handle long-running coordination, persistence across restarts, replay semantics for the workflow — is a separate question that depends on engagement scale. Pattern 21 (*Durable Agentic Workflows*) addresses this question explicitly; for now the principle stands regardless of how durability is implemented: orchestrate above, don't fold into.

There is a choreography alternative that this pattern explicitly rejects for modernization contexts. Choreography — bounded capabilities reacting to each other's events without a coordinator — works well when the workflow shape is stable and the participants are autonomous. Modernization workflows are exploratory, human-gated, and reshape as engagements teach what works. Orchestration's central coordination is the right tradeoff for that context.

## Related patterns

Pattern 19 (*Bounded MCP Servers*) is what the orchestrator coordinates across. Pattern 21 (*Durable Agentic Workflows*) is what keeps the orchestrator's state across the engagement's duration. Pattern 23 (*The Harness as Self-Observing State Machine*) governs the orchestrator's behaviour the same way it governs other agents — and watches its decisions for refinement opportunities. Pattern 24 (*Reasoning Telemetry as First-Class Output*) is what the orchestrator emits about its coordination decisions. Pattern 25 (*The Cockpit*) is where humans observe and intervene in the orchestrator's decisions.

---

# Pattern 21: Durable Agentic Workflows

*Status: in progress.*

## Context

A modernization that runs across weeks or months. Agents work asynchronously, humans intervene at gates, decisions get revisited as evidence accumulates. The platform must keep state across this entire span — across infrastructure restarts, across human absences, across deployment changes, across model upgrades.

## Problem

Agentic workflows that exist only in process memory don't survive the realities of long-running work. A platform restart loses the workflow state. A model upgrade mid-flight loses context. A human reviewer who steps away for a week returns to find the workflow expired. The traditional approach — checkpoint everything to a database periodically and reconstruct on resume — is fragile because the workflow has rich state (open agent conversations, in-flight tool calls, accumulated reasoning) that doesn't reduce cleanly to database rows.

The problem compounds with replay. When an agent's tool call fails partway through, the workflow needs to recover without redoing the work that already succeeded. Without replay-aware infrastructure, partial failures cascade — every tool call that succeeded before the failure has to be redone, every decision that was reached has to be re-derived.

## Forces

The workflow's logical model wants to be simple: states, transitions, gates, with agents operating inside steps. The workflow's physical reality is complex: long durations, partial failures, infrastructure restarts, version skew between models and tools, human intervention timing that the system can't predict. The simple logical model needs to survive the complex physical reality, which requires infrastructure designed for the complexity, not retrofitted to it.

## Pattern

Run the workflow on infrastructure designed for durability: durable workflow infrastructure that persists state automatically, supports replay semantics for failed steps, recovers cleanly from

restarts, and tolerates the time scales the work actually requires. The infrastructure handles state persistence, replay, and recovery; the workflow code expresses the logical model and stays close to the harness definition.

The harness (Pattern 23) is the workflow's *definition* — what states exist, what transitions are valid, what gates apply. The durable workflow infrastructure is the *execution substrate* that runs the harness across time. The two are separable: the same harness can run on different durable infrastructures depending on the engagement scale and operational constraints.

GitHub-native primitives (branch protection, required status checks, GitHub Actions, issue templates) materialize a subset of gates as enforceable mechanisms in the development workflow. These are useful where they apply — code review gates, deployment gates, scaffold validation gates. They're not the durable execution substrate; they're a complementary layer for gates that happen to align with version-control workflow.

For longer-running coordination — the actual modernization workflow that spans weeks — the platform needs explicit durable workflow infrastructure (in Rosetta, this is Azure Durable Functions for the coordination layer). The workflow code is written as if it were synchronous and short-lived; the infrastructure handles persistence and replay underneath.

## Consequences

Workflows survive infrastructure realities. Restarts don't lose state. Long human absences don't expire workflows. Tool-call failures don't cascade. Model upgrades during a workflow don't corrupt prior decisions. The platform becomes operationally credible at the time scales real engagements require.

The audit trail becomes durable too. Every state transition is recorded by the infrastructure, available for retrospective analysis. Compliance reviewers in regulated industries have explicit records of the workflow's progression — not just the final state, but every intermediate decision and the conditions under which it was reached.

The cost is the infrastructure dependency itself. Running on durable workflow infrastructure adds operational complexity: it has to be hosted, maintained, monitored, scaled. The workflow code has constraints — certain operations don't replay cleanly, certain patterns don't survive serialization — that pure in-memory workflows don't have. Developers must learn the infrastructure's semantics and design within them.

Whether the durable infrastructure is hosted (a cloud platform's managed service) or self-hosted is a separate operational decision that depends on the engagement's compliance posture, scale, and integration requirements. The pattern is agnostic to that choice; what matters is that the substrate is designed for durability, not retrofitted for it.

## Related patterns

Pattern 23 (*The Harness as State Machine*) is the workflow definition that runs on the durable substrate. Pattern 19 (*Bounded MCP Servers*) and Pattern 20 (*The Orchestration Layer Above Bounded Capabilities*) are coordinated through the durable workflow's execution. Pattern 20 (*Harness Self-Observation and Refinement*) consumes the durable trace as input to its analysis.

# Pattern 22: Heuristics as Explicit Artifacts

*Status: in progress.*

## Context

An agentic system where specialised agents make decisions across many contexts: slice-discovery agents propose where slice boundaries belong; tier-classification agents decide which scaffold shape to apply to which bounded context; ontology-recovery agents score candidate domain terms; anti-corruption layer detectors identify integration points needing translation; similarity agents weigh whether two paragraphs implement the same intent. Each of these decisions involves applying heuristics — rules about what counts as evidence for what.

The heuristics are not optional. Slice boundaries cannot be inferred without rules about what makes a structural cluster cohesive enough to be a slice. Tier classification cannot proceed without thresholds on coupling, change frequency, and strategic value. Ontology recovery cannot weight candidate terms without scoring rules. The question is not whether the heuristics exist — they do, in every agentic system that makes such decisions — but where they live and how they evolve.

## Problem

The dominant practice in agentic systems is to bake heuristics into agent prompts. The slice-discovery agent's prompt explains what to look for: "paragraphs that share data and predicates are likely to belong to the same slice; XCTL between paragraphs typically marks a bounded-context transition; ..." The tier-classification agent's prompt encodes the thresholds: "if cyclomatic complexity is over X and coupling is over Y, classify as tier 3..."

This works briefly and fails as the system scales. The failures are characteristic:

- **Implicit knowledge becomes invisible.** Heuristics buried in prompts are not findable. When the team needs to know "what rule did we apply for slice boundary detection?" they must read prompts — sometimes scattered across dozens of agent definitions — and reconstruct the rule from natural language. The reconstruction is unreliable.
- **Refinement drifts through prompt-engineering.** When a heuristic needs adjustment, the change is made by editing a prompt. The change has no version history that distinguishes it from other prompt edits. It cannot be A/B tested. Its effect is observable only through the downstream behaviour of the agent.
- **Audit trail breaks.** When an agent makes a decision and a reviewer asks "why?", the agent cites its reasoning in natural language, but the heuristic it applied is not a named artifact.

The reasoning is post-hoc rationalisation of prompt-embedded knowledge, not citation of a queryable rule.
- **Composition fails.** When two agents must apply the same heuristic — slice-discovery and slice-validation, for instance — the heuristic must be duplicated across prompts. Drift between the copies is inevitable.

The deeper failure is treating heuristics as implicit context rather than as first-class artifacts. The compiler principle (Pattern 8) articulates that deterministic decisions belong in deterministic infrastructure; heuristics are deterministic decisions, but they have been hiding in probabilistic prompt content.

## Forces

Heuristics need to be observable. When an agent applies a heuristic, the audit trail must record which heuristic was applied and what evidence supported it. Heuristics need to be versionable. When a heuristic is refined, the refinement must be reviewable, reversible, and traceable to its rationale. Heuristics need to be composable. Multiple agents may apply the same heuristic; refining once must update everywhere.

But heuristics also need to be contextually applied. A heuristic that holds for tier-3 bounded contexts may not apply in tier-0. A heuristic for slice discovery in CICS COBOL may not apply for batch JCL. The catalog of heuristics must support specialisation by context.

## Pattern

Treat heuristics as first-class queryable artifacts, not as implicit knowledge in agent prompts. Build a heuristic catalog where each heuristic is a named, structured entry:

- **Heuristic identifier** — a stable name (`xctl-bounded-context-transition`, `tier-3-coupling-threshold`, `ontology-candidate-vocabulary-overlap`).
- **Conditions of application** — what context the heuristic applies to (tier, language, bounded context, slice characteristic).
- **Evidence weights** — for each piece of evidence the heuristic considers, the contribution to the decision (XCTL between paragraphs in different bounded contexts → 0.8 confidence of slice transition).
- **Version and validation status** — when the heuristic was introduced, when it was last refined, what evidence validated it.
- **Observability hooks** — what telemetry the heuristic emits when applied.

Specialised agents access the catalog through queries: a slice-discovery agent encountering an XCTL queries `heuristics:by-cue("XCTL", context:current_tier)` and receives the relevant heuristics with their weights. The agent applies the heuristics, emits a reasoning record (Pattern 25) citing which heuristics were applied with which evidence, and the audit trail is intact.

The catalog itself is built with the same compiler discipline as the rest of the modernization pipeline. Heuristic definitions live in structured files (YAML, JSON, or domain-specific schema); validators enforce required fields; versioning is git-native; refinements go through review like any other architectural change. The catalog is queryable through an MCP server (Pattern 19): agents discover heuristics through tool calls, not through prompt content.

Refinement happens through structured evolution. When the harness's self-observation (Pattern 23) detects that a heuristic is producing high error rates in a specific context, it surfaces the heuristic for review. An architect (or a retrospective agent operating under human authorisation) refines the heuristic — adjusts weights, narrows conditions of application, adds new evidence types — and the refinement is recorded. The next time the heuristic is applied, agents see the refined version.

The heuristic catalog is itself a substrate that the compiler architecture treats as first-class, alongside the graph (Pattern 3), the IR (Pattern 9), and the domain ontology (Pattern 5). Documentation emitters (Pattern 15) can render the catalog as documentation: which heuristics exist, what they decide, what their evidence weights are, what their refinement history is. The catalog becomes legible to humans without requiring them to read agent prompts.

Catalog entries themselves can be partially derived from operational history. Uberto Barbini has reported experimenting with generating rule files for AI assistants from git history and pull-request comments — the team's own corrective behaviour over time becomes input to the rules the next agent applies. The same idea operates at platform scale here: when reviewers reject scaffolds or flag candidates with consistent rationale, that rationale is signal about which heuristics need refinement or which new heuristics need authoring. The retrospective agent (Pattern 23) can consume PR conversation telemetry as one of its evidence streams, surfacing candidate catalog updates that an architect then validates. The lesson does not stay in the head of the reviewer who wrote the comment; it propagates into the catalog.

## Consequences

Agent reasoning becomes auditable at the rule level. When a slice is proposed, the trace cites the specific heuristics that drove the proposal; when a reviewer challenges the proposal, the conversation can address whether the heuristic is correct, not whether the agent "got it right." The unit of disagreement is the rule, not the outcome.

Refinement compounds across the engagement. Lessons learned from one bounded context — that this specific cue is more informative than the catalog initially weighted — become catalog updates that benefit every subsequent bounded context. Without the catalog, lessons stay in individual prompts and don't propagate.

Composition works. Multiple agents apply the same heuristics by querying the same catalog; refinement updates all consumers simultaneously. There is no prompt-copy drift because there are no prompt copies.

The cost is the discipline of structured heuristic authoring. Heuristics that would have been one sentence in a prompt now require schema-conformant catalog entries with named fields, validated conditions, and observability hooks. The authoring cost is higher per heuristic; the system-wide cost — across many agents and many engagements — is lower.

There is also a discovery cost. When a new kind of decision arises (a new agentic capability needs to make a new kind of judgment), the team must decide what heuristics apply and add them to the catalog. This work is more visible and slower than adding sentences to prompts. The visibility is the point.

The principle is genuinely first-mover. The compiler principle (Pattern 8) articulates the broader division between deterministic and probabilistic work; this pattern operationalises the deterministic side specifically for the case of decision heuristics. No catalog the field has produced articulates

"heuristics as queryable, versionable artifacts" as an explicit pattern. The closest precedents — rule engines in business systems, ML feature stores, OPA policy as code — solve adjacent problems in adjacent domains. Applying the discipline to agentic decision-making is the contribution.

## Related patterns

Pattern 8 (*The Compiler Principle*) is the broader principle this pattern operationalises — deterministic decisions live in deterministic infrastructure. Pattern 7 (*Vertical Slice Discovery*) is one of the principal consumers — slice-boundary heuristics live in the catalog. Pattern 10 (*Tier-Aware Scaffolding*) consumes tier-classification heuristics. Pattern 17 (*Hypothesis-Driven Verification*) consumes divergence-categorisation heuristics. Pattern 19 (*Bounded MCP Servers*) hosts the heuristic catalog as a queryable capability. Pattern 23 (*The Harness as Self-Observing State Machine*) detects when heuristics need refinement based on operational evidence. Pattern 25 (*Reasoning Telemetry as First-Class Output*) cites which heuristics each agent applied.

---

# Pattern 23: The Harness as Self-Observing State Machine

*Status: working.*

## Context

An agentic system producing C# code from COBOL inside a complex multi-stage process: ingestion, analysis, scaffold rendering, paragraph translation, verification, gate transitions, eventual cutover. Each stage has prerequisites and postconditions. The agents must operate within these constraints across the entire workflow. Over time, the harness accumulates operational history: which gates fired, which transitions were taken, which paths were never explored, which states agents got stuck in.

## Problem

The dominant instinct in agentic systems is to multiply agents — more specialised agents, more coordination layers, hierarchies and swarms. The framing is: *if one agent is good, many agents are better.* In my experience this is the wrong direction. What produces tractable modernization is not more agents but **better harness**: more deterministic constraints, more verifiable invariants, more explicit gates, less informal coordination between agents. Harness engineering is the discipline most underdeveloped in current agentic systems for software work.

The concrete failure mode: early versions of agentic systems use prompt engineering to tell the agents what to do at each stage. This fails as work scales. Agents drift, edge cases multiply, prompts grow into unmaintainable artifacts. Telling agents what not to do in natural-language instructions is advisory; the agents treat the instructions as suggestions. The result: candidates that match dev-time verification but are architectural disasters — code that works but is structured terribly, handlers that do the right thing through paths no human would maintain. Behavioural equivalence is necessary but not sufficient.

A second problem emerges once the harness exists. Without explicit observation of its own operation, the harness degrades silently. Gates calibrate wrong — never failing (useless) or always failing (blocking legitimate work). Transitions become over-engineered for cases that never arise. Cycles develop because gate criteria don't give agents enough signal to decide. Escalation rates climb because gates that should be automatable are not. The traditional response — periodic manual review — is slow, subjective, and becomes infeasible as the harness grows.

## Forces

The agents need substantial freedom inside each stage to translate paragraphs, refine candidates, iterate against verification. They need strict constraints between stages to prevent architectural drift, premature promotion, bypass of human review. Free-form prose can't enforce strict constraints reliably. Strict constraints implemented as rigid procedure prevent useful agentic exploration.

A second tension applies to the harness itself: it is operational infrastructure that should be stable (constant changes produce churn that propagates to agents, humans, and tooling) and living infrastructure that should evolve as the engagement teaches what works (refusing to refine it produces a calcified workflow that no longer fits the work). Stability and responsiveness are both real concerns.

## Pattern

Implement the modernization workflow as a typed state machine in code. Each step has a defined entry condition, a defined exit condition, and a set of agents authorised to operate inside that step. Transitions happen through gates — pre-tool-use hooks check prerequisites, post-tool-use hooks check invariants. Inside a step, agents have substantial freedom; between steps, the state machine is determinate.

Enforce the state machine through GitHub-native infrastructure: branch protection rules, required status checks, issue templates, GitHub Actions. Each project has a `scaffold-meta.json` file that defines the workflow constitution: which steps exist, what gates apply, who has authority for which transitions.

The harness also observes itself. Collect structured telemetry on every gate evaluation, every transition, every escalation, every cycle through a state. Make the telemetry queryable and analyzable. Analyze continuously to detect inefficiencies: *gate hit rate* (gates that never fail are candidates for removal; gates that always fail are candidates for recalibration), *path frequency* (transitions that are never taken are candidates for removal; transitions that dominate suggest the harness should be simplified around them), *cycle detection* (cycles longer than threshold suggest gates don't provide enough signal for agents to decide), *time-in-state distribution* (states with disproportionate time spent suggest poorly-defined work or unclear exit criteria), *escalation rate* (rising escalation rates at a particular gate suggest the gate isn't decidable automatically and should be redesigned).

A retrospective agent processes the telemetry continuously and proposes refinements: gates to remove, gates to recalibrate, transitions to add or remove, states to consolidate, escalation criteria to revise. Each proposal carries its evidence — observations grounding it, change proposed, predicted impact. Humans validate refinements through the cockpit (Pattern 25). Approved changes update the harness definition; the approval itself is recorded with evidence and predicted outcome. If the predicted outcome doesn't materialize, the retrospective agent surfaces the discrepancy.

In Rosetta, this is called the harness. The harness is what governs the agents — not prompts, not goodwill, not human supervision at every step. Birgitta Böckeler's framing of guides (artifacts that steer before action) and sensors (checks that observe after action) describes the harness's structure. Nick Tune's framing of workflow as code, with transitions as events, describes its implementation.

The harness also enforces a discipline on the unit of agent-produced change. Uberto Barbini's *one prompt, one commit* principle — the agent never commits its own work; commits are atomic units that

the developer (or, at platform scale, a gated reviewer or automated invariant check) authorises after inspection — applies directly here. At individual-developer scale, the commit is a unit of human control over agent output. At platform scale, the same principle holds with a different surface: each agent-produced change is a candidate that passes through gates (Pattern 23), invariant checks (Pattern 8), and verification (Patterns 16, 17) before it becomes part of the system. The principle is the same: the agent proposes; the harness (or the human, when the harness defers) disposes. The commit is never a side effect of agent execution.

The harness must also recognise characteristic execution-time failure modes — patterns Barbini has named at the developer scale that recur at platform scale (see *Agent execution failure modes* in the glossary). *Loop of death* — the agent fixes A, breaks B; fixes B, breaks A — surfaces in the harness as cycle detection in the telemetry: when the same paragraph or scaffold enters and exits the same gate repeatedly, the harness escalates rather than letting the agent thrash. *Misunderstanding the requirement* — the agent works on the wrong part of the code — surfaces as scaffold boundary violations: when an agent attempts edits outside its authorised scope, the PreToolUse hook blocks the call. *Desperate changes* — the agent escalates beyond its scope when blocked — surfaces as invariant violations: when an agent attempts to modify the scaffold contract itself or reach into another bounded context, the harness rejects the attempt and records the escalation as evidence for heuristic refinement (Pattern 22). The taxonomy is operationally useful: each failure mode has a specific detection mechanism and a specific corrective response.

---

**[FIGURE] 23.1 — *The harness as self-observing state machine: stages, gates, transitions, and refinement loop***

*Diagram showing the modernization workflow as a typed state machine with a feedback loop. Top half: States (left to right): Discovery → Analysis → Scaffold Rendering → Paragraph Translation → Dev Verification → Production Verification → Cutover. Between states: gate boxes with check icons indicating pre/post hooks. Annotations on transitions: required status checks, human approval gates, automated invariant checks. Sub-region inside each state shows "agents have freedom here" with a small swarm icon. Above the diagram: the harness definition file (`scaffold-meta.json`) shown as the source of truth for the state machine. Below: GitHub-native enforcement (branch protection, required checks, Actions) shown as the runtime layer. Bottom half: telemetry pipeline collecting gate evaluations, transitions, cycles, time-in-state metrics, feeding a retrospective agent that proposes refinements; refinement proposals route to humans via cockpit (Pattern 25) for validation; approved refinements update the harness definition. Style: horizontal flow at top with clear state boundaries and visible gate mechanisms; feedback loop at bottom showing self-observation cycle. Goal: reader sees that the workflow is determinate at the level of stages, agentic within each stage, and continuously refined based on its own operational evidence.*

---

## Consequences

The agents operate inside a cage of deterministic rules. The cage is enabling: inside it, agents have substantial freedom; outside it, they can't make architectural decisions, promote work to production, or bypass human review. Failures are localised — when something goes wrong, the audit shows which gate failed, which transition was attempted, which invariant was violated.

The harness evolves with the work. Inefficiencies surface explicitly with evidence rather than being

absorbed quietly by frustrated developers. The state machine matures as the engagement teaches what works. Operators see the harness as a living artifact, not a static deliverable. The audit trail extends to the harness itself — compliance reviewers in regulated industries can demonstrate not only that the workflow followed the rules, but that the rules themselves are continually validated against the work. The system becomes self-improving without becoming self-modifying — humans remain in the loop for structural changes.

The cost is engineering discipline. The state machine has to actually be implemented in code, with real types and real validation. Hooks have to actually check what they claim to check. The GitHub-native infrastructure has to be configured for each project consistently. Prose-based governance is cheaper to write but doesn't survive contact with real workloads. The telemetry pipeline and analysis layer add their own engineering cost — structured observation requires the harness to be instrumented at design time, not retrofitted. The retrospective agent itself is a non-trivial system — pattern detection over long-running workflows is its own engineering problem. The proposed refinements need to be validated against more than just the metrics they're calibrated against, or the harness optimizes for the wrong things.

Harness engineering remains the discipline most underdeveloped in current agentic systems for software work. The default assumption — that agents can be trusted to make sensible decisions inside whatever loose framing they're given — fails as the work scales. The work the field is doing on agent multiplication is largely the wrong investment; the work the field needs is investment in harnesses. Self-observation is what allows the harness to remain calibrated to reality as the work evolves rather than becoming a calcified workflow that no longer fits.

## Related patterns

Pattern 8 (*The Compiler Principle*) provides the underlying motivation: governance is deterministic infrastructure that constrains probabilistic work. Pattern 19 (*Bounded MCP Servers*) is the architectural decomposition the harness governs over. Pattern 20 (*The Orchestration Layer Above Bounded Capabilities*) operates inside the harness's gates. Pattern 21 (*Durable Agentic Workflows*) produces the durable execution trace from which telemetry is drawn. Pattern 24 (*Reasoning Telemetry as First-Class Output*) provides the agent-side counterpart — telemetry of agent reasoning that complements telemetry of harness operation. Pattern 25 (*The Cockpit*) is where humans interact with the harness's gates and validate proposed refinements. Pattern 26 (*Spec Deltas as the Unit of Review*) is one of the artifacts the harness's gates can require.

# Pattern 24: Reasoning Telemetry as First-Class Output

*Status: in progress.*

## Context

An agentic system where agents make decisions continuously: classifying paragraphs, proposing slices, translating COBOL, validating verification verdicts. The decisions are recorded — the cockpit (Pattern 25) shows what was decided, the audit trail shows when. But knowing *what* an agent decided is only part of the question; sometimes what matters is *how* the agent reached the decision.

## Problem

Decision visibility is not reasoning visibility. Two agents can reach the same decision through very different reasoning processes. One reasons carefully, considers alternatives, weighs evidence; the other pattern-matches superficially and arrives at the answer that happened to fit. Both produce the same observable output, but the quality of the work is different. Without visibility into reasoning, the system can't distinguish — which means it can't audit, can't learn, can't improve.

The problem becomes critical in regulated domains. Compliance reviewers don't just need to verify that a decision was made correctly; they need to verify that the decision-making process was sound. *"The agent decided X"* is insufficient; *"the agent considered alternatives Y and Z, weighed them against criteria A, B, C, and reached X with confidence N"* is what compliance requires.

## Forces

LLM reasoning is naturally opaque — the model produces outputs without exposing intermediate state. Asking the model to explain its reasoning post-hoc produces rationalizations that may or may not reflect what actually happened during inference. Asking the model to expose reasoning during inference (chain-of-thought, scratchpad approaches) produces traces that are useful but expensive in tokens and not always faithful to internal state.

The system needs reasoning visibility that's structured (queryable, analyzable), reliable (faithfully reflecting how decisions were reached), and affordable (not requiring 10x token budgets). Achieving all three is its own design problem.

## Pattern

Treat reasoning telemetry as a first-class output of every agentic decision, not as a debugging afterthought. Each decision carries a structured record: what evidence was considered, what alternatives were evaluated, what heuristics were applied, what confidence was assigned, what tools were consulted in the process. The record is structured (typed schema, queryable), not free-form prose.

Generate the reasoning record explicitly. When an agent classifies a paragraph as tier 2, the record captures: which structural signals contributed (cyclomatic complexity, coupling, change frequency), what tier candidates were considered (the agent didn't just pick tier 2; it evaluated tier 1, 2, and 3 and picked tier 2 over the alternatives), with what relative confidence, and citing what evidence. The record exists alongside the decision, not as a separate artifact.

Make reasoning queryable across decisions. *"Show me all decisions where the agent considered tier 1 but chose tier 2 with low confidence"* should be answerable. *"Show me decisions where the agent's confidence was below threshold"* should be queryable. *"Show me decisions where the reasoning trace cited a particular heuristic"* should produce a list.

Use the reasoning telemetry for retrospective learning. A retrospective agent analyzes patterns in the reasoning across many decisions: which heuristics correlate with later corrections, which evidence types are most predictive of correct decisions, which kinds of reasoning lead agents into cycles. The patterns inform refinements to agent prompting, tooling, or scaffolding.

## Consequences

The system becomes auditable at the level of reasoning, not just at the level of outputs. Compliance reviewers in regulated domains can verify that decision processes are sound, not just that decisions are correct. Retrospective analysis becomes possible because reasoning is captured in structured form, not lost between sessions.

The agents themselves benefit from explicit reasoning. The act of generating the structured trace forces the agent to articulate its reasoning, which tends to improve decision quality (a phenomenon documented in chain-of-thought prompting). Decisions made with explicit reasoning records are, on average, more careful than decisions made without them.

The cost is the token and engineering overhead. Reasoning telemetry adds tokens to every decision; the structured schema must be designed and maintained; the queryable storage must be operationalized. Some decisions don't benefit from extensive reasoning capture (trivial classification with high confidence); the system must distinguish so it doesn't pay the cost everywhere.

The faithfulness question — whether the reasoning trace reflects the agent's actual decision process or is post-hoc rationalization — remains open. Different prompting strategies produce different fidelity profiles. The pattern is in active development; how to operationalize the trace generation reliably is part of what's being tested.

## Related patterns

Pattern 23 (*The Harness as State Machine*) governs when reasoning capture is required. Pattern 20 (*Harness Self-Observation and Refinement*) consumes reasoning telemetry as input to its analysis of

agent behavior across the workflow. Pattern 25 (*The Cockpit*) is where humans review reasoning traces alongside decisions when intervention is needed.

---

# Pattern 25: The Cockpit

*Status: in progress.*

## Context

An agentic modernization platform with a typed harness, bounded MCP servers, an orchestration layer, and verification machinery. The agents produce work continuously; the platform records every decision. Humans must review certain decisions, approve certain transitions, intervene when something goes wrong. The amount of activity is too large for ad-hoc oversight.

## Problem

Without a visible operational surface where humans can see what the agents are doing, intervention happens only when something has already gone wrong. Routine decisions that should have human authorisation pass through the harness without review because the human didn't know they were happening. Audit trails exist but are post-hoc; they document what happened, not what was about to happen.

Agentic systems without a visible cockpit are systems that will eventually produce harm. Not because the agents are malicious, but because invisible work scales badly. When humans can't see what's happening, they can't intervene. When they can't intervene, the agents end up making decisions humans should have made.

## Forces

Routine decisions should not require human attention; if they did, the system would be no better than a manual process. Critical decisions must require human attention; if they didn't, the system would lack accountability. The boundary between routine and critical depends on context: a transition that is routine in one phase becomes critical in another. The cockpit must accommodate this without forcing a fixed taxonomy.

## Pattern

Build a dedicated operational surface — a cockpit — where humans see what the agents are doing, validate decisions the graph has surfaced, approve transitions between harness states, review diffs

between generated C# and original COBOL, intervene when verification finds something the agents can't resolve.

The cockpit surfaces, at minimum:

- **Graph decisions** — when the analysis surfaces a bounded context, the architect sees the proposed boundary with the source evidence that supports it.
- **Translation candidates** — when agents produce candidate C#, a reviewer sees the COBOL on one side, the candidate on the other, the verification verdict, the test cases exercised.
- **Gate transitions** — when the harness reaches a gate requiring human authority, the relevant human sees the request with the evidence package the harness has assembled.
- **Spec deltas** — when a change to the system has been articulated as a structured delta (Pattern 26), the reviewer reads the delta to understand intent, with code-level details available but not the unit of review.
- **Audit trail** — every decision by every agent and every human, traceable later for compliance, debugging, or learning.
- **Diff views** — side-by-side comparison between original and generated code, with source coordinates linking the two.

In Rosetta, this cockpit is called Studio. It sits above the harness (which handles governance at the deterministic level) and provides governance at the human-experience level.

---

**[FIGURE] 25.1 — *The cockpit: human authorisation surface for agentic work***

*Mockup-style diagram showing a cockpit UI sketch (not pixel-perfect, but suggestive of the layout). Top bar: harness state indicator showing current stage with progress through the workflow. Left panel: queue of pending decisions awaiting human attention — each item shows the decision type, the agent that surfaced it, the urgency. Center panel: detail view of a selected decision — for example, a translation candidate with COBOL on the left, candidate C# on the right, verification verdict at the bottom, source provenance links inline. Right panel: evidence package — what the harness assembled to support this decision (graph nodes touched, IR fragments produced, related decisions). Bottom bar: action buttons — Approve, Reject, Request More Info, Escalate. Annotations: "humans authorise; agents execute" in a corner. Style: clean operational dashboard sketch, suggestive of real UI without claiming to be the final design. Goal: reader understands what the cockpit looks like and what unit of human action it supports.*

---

## Consequences

Humans operate the agentic system with authority and visibility. Routine work flows; critical decisions surface. The audit trail is operational, not just retrospective — humans can follow the system's reasoning forward, not just backward. Modernizations in regulated industries become operable because compliance reviewers can verify any decision the system made.

The cost is the cockpit itself. Building a UX appropriate for agentic systems is its own discipline — different from traditional system UIs in grammar, not just in surface. Traditional UIs surface state and let users command; agentic UIs surface decisions and let users authorise. The unit of human action changes. Designing for it is ongoing work.

Studio in Rosetta today is in an early form. The schema for what it must surface is clear; the UX is still evolving. I'm building it as the platform matures, learning what reviewers actually need by watching how they use early versions.

## Related patterns

Pattern 23 (*The Harness as State Machine*) is what the cockpit operates above. Pattern 19 (*Bounded MCP Servers*) is what feeds the cockpit's data. Pattern 4 (*Source Provenance Discipline*) is what makes diff views possible. Pattern 26 (*Spec Deltas as the Unit of Review*) is consumed in the cockpit's review interface.

---

# Pattern 26: Spec Deltas as the Unit of Review

*Status: next.*

## Context

A system being modernized or evolving post-modernization, where changes are continuously produced — by agents, by humans, or in collaboration. Each change has implementation artifacts (modified code, new tests, updated configurations). Reviewers must understand each change well enough to approve its promotion. The volume of changes is too high for full code-level review of every one, but consequential enough that approval can't be skipped.

## Problem

AI-generated output has a volume problem. Agents tend to produce more — more documentation, more alternatives, more justifications, more context — than reviewers can process. The economics is asymmetric: generation scales with compute, review scales with human attention. A modernization that generates 50 slice analyses per day saturates reviewers if each is 5 pages; the same modernization generating 50 single-page deltas remains tractable. *More is not better. Less is better.*

Without an artifact that articulates the change in terms of the system's requirements, reviewers reconstruct intent from code. This forces them to operate at the wrong level: they're reading diffs of implementation when what they need to know is *what requirements changed and why*. The review becomes detailed in the wrong sense — granular about implementation, vague about intent. Combined with AI's volume tendency, this produces a workflow where reviewers face thousands of lines of code changes per change set, with intent buried somewhere in the diff.

The instinct to capture intent through commit messages or PR descriptions is fragile. Those are textual, inconsistent, optional, and not validatable against the code. A commit message can claim the change does one thing while the code does another, and nothing in the workflow catches the discrepancy.

## Forces

Implementation is the operational truth — the code that will actually run. But reviewers are not always implementers; they need to understand the *what* and the *why* of a change without recon-

structing the *how*. Capturing intent separately from code duplicates work if done manually. Skipping the duplication leaves reviewers to derive intent themselves, which is slow, error-prone, and inconsistent.

The captured intent must be structured enough to be validated against the code, accessible enough that humans read it quickly, and durable enough that the history of intents tells the system's evolution.

## Pattern

For each change to the system, produce two synchronised artifacts: the implementation change (code) and a *spec delta* — a structured document that articulates which requirements changed, what is added, what is removed, what is modified. The spec delta is the primary unit of review. Reviewers understand the change by reading the delta; implementation details are available if needed, but they are not the unit of review.

The delta is structured, not narrative. It uses a defined format — sections for added requirements, removed requirements, modified requirements, with each item linked to the parts of the implementation that realise it. The structure allows two things narrative descriptions can't: validation that the delta and the code reflect the same change, and aggregation of deltas across time to produce a history of how the system evolved at the requirement level.

Generation of the delta is part of the change workflow, not an afterthought. When an agent produces a change, the agent also produces the delta. When a human produces a change, the workflow requires the delta before merge. The harness (Pattern 23) can enforce this: a gate that requires the spec delta to exist and to be validated against the implementation before promotion.

The discipline generalises beyond review: AI output must be sized for human consumption rates throughout the modernization. A pattern catalog that generates 5-page analyses for every slice fails not because the analyses are wrong, but because no architect can read them. A modernization that produces 100 reasoning records per day fails by saturation, not by error. Spec deltas are the operational realisation of this discipline at the review surface; the principle — *output volume is itself an architectural concern* — applies everywhere AI generates content for human consumption.

## Consequences

Reviewers operate at the right level. Review becomes faster because reviewers don't reconstruct intent; the delta provides it directly. The system's history becomes legible — a new contributor can read the sequence of deltas to understand how the requirements evolved, rather than archaeologising commits to infer the same. Compliance reviewers in regulated environments have explicit records of what each change was meant to accomplish.

The volume discipline pays back across the whole modernization workflow. Fewer artifacts produced, denser information per artifact, structured for human scanning rates — these are properties that compound. A modernization that respects human consumption rates remains tractable as it scales; one that doesn't saturates its reviewers and stalls at the bottleneck of attention.

The traceability deepens. Each delta links to its implementation, which through source provenance (Pattern 4) links back to the legacy or the design that motivated it. The whole chain — from orig-

inal requirement, through implementation, through delta, through any subsequent revision — is queryable.

The cost is the discipline of keeping deltas synchronised with code. This isn't free. Generating useful deltas requires that someone or something understands the change well enough to articulate its intent in structured form. For agent-produced changes, this means the agent's workflow includes delta generation, and the generation has to be reliable. For human-produced changes, this means contributors learn to write deltas, and the workflow requires them. Either way, the synchronisation cost is real and ongoing.

The pay-off justifies the cost when the system is consequential enough that review matters and complex enough that intent isn't obvious from code. For research prototypes that pivot frequently, deltas may be overhead. For production systems in regulated industries that must demonstrate change control, deltas may be the most important artifact in the workflow.

This pattern is designed but not yet built in Rosetta. The principle (review at the level of intent, not implementation) is consistent with the prototype's broader posture (governance through structured artifacts, not through prose). What remains to be tested is whether the operational machinery to generate and validate deltas reliably can be built, and whether reviewers find them useful enough to justify the engineering cost.

## Related patterns

Pattern 23 (*The Harness as State Machine*) is what enforces the requirement that changes produce deltas. Pattern 25 (*The Cockpit*) is where reviewers consume deltas. Pattern 18 (*Behavioural Specifications Grown from Production*) produces a different kind of specification artifact — those grown from production behaviour, where deltas articulate intent of changes. The two specification patterns are complementary: deltas describe intended change, behavioural specifications describe observed system behaviour.

---

# Pattern 27: Rollout and Cutover at Bounded Context Granularity

*Status: working.*

## Context

A modernization that has produced one or more bounded contexts of modernized code, verified against the Legacy Twin (Pattern 16), validated for behavioural equivalence (Pattern 17), and gated through the harness (Pattern 23). The team must now deploy the modernized contexts into production, route traffic to them, and eventually decommission the legacy contexts they replace. The transition is incremental — running both systems in parallel while traffic shifts — and culminates in cutover, when the modernized system becomes the authoritative system of record.

The transition is the moment of truth for the modernization. Up to this point, the work was internal: discovery, generation, verification. From this point on, every decision affects production. Errors are observable. Customers feel the consequences. Regulators take notice.

## Problem

Two failure modes dominate this stage. The first is big-bang cutover: shutting down the legacy and bringing up the modernized system in a single deployment window. The team prepares for months, runs migrations overnight, and discovers in the morning that something is wrong. Rollback is difficult because the legacy may have been decommissioned; forward fixes are difficult because production traffic is now exposing edge cases the team hadn't anticipated. The system either limps along while the team patches in production or has to be reverted at high cost.

The second failure mode is naive strangler fig: extracting endpoints one at a time, with each endpoint routed independently. The endpoint granularity is too fine. Endpoints that share business logic, data, or transactional boundaries end up split across modernized and legacy systems in ways that violate consistency. The team discovers mid-rollout that "modernize one endpoint at a time" doesn't compose because the endpoints aren't actually independent units.

The deeper failure is choosing the wrong unit of rollout. Programs are too internal — users don't experience programs. Endpoints are too narrow — they don't carry the consistency boundaries the business depends on. Bounded contexts are the right unit: they are operationally meaningful (one capability the business recognises), consistent (the aggregates inside them maintain invariants together), and decomposable (their command/event surface is the public contract).

## Forces

The transition cannot be instantaneous; it spans weeks to months during which legacy and modernized must coexist. But the transition cannot be arbitrary either — it must preserve consistency, support rollback, expose evidence about whether the modernized system is performing correctly. The mechanism must accommodate progress (traffic shifting from legacy to modernized) and reversibility (traffic shifting back if problems emerge).

Customers experience capabilities, not bounded contexts. A capability — process insurance claim, settle trade, generate report — typically spans one or more bounded contexts. Rollout granularity must align with capabilities so that customers see coherent behaviour: a capability is either served by the legacy or by the modernized system, not split mid-flow.

Regulated industries impose additional constraints. Cutover events must be auditable: which context was switched, when, by whom, with what evidence supporting the decision. Rollback procedures must be tested before they are needed. The system must remain in a known-good state at every moment of the transition.

## Pattern

Roll out at bounded context granularity. Each bounded context goes through its own rollout phase, culminating in its own cutover. The order of bounded contexts is determined by the capability map (Pattern 1) — strategic-core capabilities receive the most preparation and deliver the most business value early; commodity capabilities follow with lighter ceremony.

For each bounded context, the rollout proceeds in phases:

- **Parallel run** — the modernized context runs alongside the legacy context. Production traffic is processed by both. Witness (Pattern 17) compares outputs in real time, surfacing divergences for review. No customer impact yet — the modernized context's outputs are observed but not used.
- **Shadow validation** — over a period of days to weeks (proportional to capability importance), the parallel run accumulates evidence. Divergences are categorised and addressed. The team builds confidence that the modernized context produces correct outputs across the full traffic shape, not just the dev-time corpus.
- **Incremental routing** — a routing layer (typically API gateway, transaction router, or message bus topology) begins directing a fraction of traffic to the modernized context as authoritative. The percentage starts small (1%, 5%, 10%) and grows as evidence accumulates. Customers on the modernized path receive its outputs; the legacy continues to process the rest.
- **Canary monitoring** — during incremental routing, Witness watches for divergences between the modernized context's behaviour and historical baselines. Anomalies trigger automated reduction of the routed percentage or full rollback to legacy.
- **Cutover** — when evidence supports full traffic on the modernized context (typically: parallel run with zero unexplained divergences for N days, canary percentage at 100% for M days, business sign-off), the bounded context cuts over. The legacy context stops receiving new traffic.
- **Legacy retention period** — the legacy context remains available for some period after cutover (weeks to months) as comparison reference. Post-cutover divergence detection runs against it; rollback to legacy remains possible if a previously-undetected issue emerges.

- **Decommission** — when the retention period passes without incident and confidence is high, the legacy context is decommissioned. Its resources are reclaimed; its data is archived per compliance requirements.

The routing layer is itself a first-class artifact. It implements anti-corruption (Evans, 2003) between modernized and legacy contexts during the transition — translating vocabularies, mediating protocols, isolating each side from the other's accidents. When all contexts have cut over, the routing layer's translation work is complete and it can be simplified or retired.

Cutover readiness is gate-driven, not calendar-driven. The harness (Pattern 23) holds the cutover transition as a state with explicit prerequisites: Twin Verification (Pattern 16) stable for the required duration, Witness reports clean, business approval recorded, rollback procedure tested. Cutover happens when prerequisites are met; cutover does not happen when they are not, regardless of project pressure.

Rollback is rehearsed before it is needed. The team practices rolling back from cutover to legacy in non-production environments, documents the procedure, and validates that data written to the modernized system during the rollout period can be reconciled with legacy state. A rollback that has not been tested is not a rollback; it is a hope.

Within a bounded context's rollout, the team must also decide the *order* in which capabilities migrate. Nick Tune has named two opposing strategies: *Migrate Reads First*, which unblocks downstream consumers quickly by routing query paths to the modernized system while writes continue against the legacy; and *Migrate Writes First*, which unblocks new use cases that depend on the modernized write model. The choice is strategic — driven by which downstream pressure dominates — not technical. Definitions and pointers are in the glossary.

## Consequences

Risk distributes across the transition. Each bounded context is a small bet rather than the entire modernization. Problems surface in one context at a time and are addressed without affecting the rest of the rollout. The aggregate risk across the modernization is the sum of per-context risks, which is much smaller than the risk of a single big-bang cutover.

Business value delivers incrementally. The first cut-over context delivers value as soon as its rollout completes — typically months before the full modernization. Subsequent contexts add value progressively. The customer does not wait until everything is migrated to see returns.

Operational evidence accumulates. By the time the last context cuts over, the team has rehearsed cutover multiple times, validated their playbook against multiple capabilities, refined Witness and the routing layer through actual operational experience. The final cutover is informed by the prior ones, not improvised.

The cost is the routing layer and the discipline of phased rollout. The routing infrastructure must be designed, built, and operated; the phased process must be followed; the rollback procedures must be tested. None of this is free. Compared to a single big-bang attempt, however, the phased approach is dramatically lower risk and dramatically more predictable.

Martin Fowler's *StranglerFigApplication* essay (Fowler, 2004) articulates the canonical incremental replacement pattern this catalog applies. Sam Newman's elaboration in *Monolith to Microservices* (Newman, 2019) extends the pattern with operational detail. What this catalog contributes is the

granularity choice — bounded context as unit of rollout — which aligns Fowler's strangler fig with DDD's strategic design. The granularity is where the technique becomes specifically applicable to mainframe modernization, where endpoint-level granularity is too fine for the consistency boundaries that matter and program-level granularity is too coarse for incremental delivery.

## Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines the order in which bounded contexts roll out. Pattern 16 (*Twin Verification*) validates each context before its parallel run begins. Pattern 17 (*Hypothesis-Driven Verification*) operates Witness during the rollout, categorising divergences and surfacing them for review. Pattern 23 (*The Harness as Self-Observing State Machine*) holds rollout and cutover as harness states with explicit gates. Pattern 25 (*The Cockpit*) is where humans observe rollout progress and authorise cutover transitions. Pattern N2 (*Dual-Run Coexistence*) provides the data-layer infrastructure that sustains parallel run across legacy and modernized contexts. Pattern 12 (*Commands and Events as Logical Boundary*) ensures that bounded contexts on opposite sides of the rollout boundary continue to communicate through their established contracts.

---

# Pattern 28: Dual-Run Coexistence: CDC, Reconciliation, and the Bridge Period

*Status: in progress.*

## Context

A mainframe modernization in its rollout phase (Pattern 27): bounded contexts are migrating one at a time, with parallel run periods where modernized and legacy systems coexist. The legacy continues to be authoritative for contexts not yet cut over; the modernized system is authoritative for contexts that have cut over; some contexts are in shadow validation or incremental routing, neither fully legacy nor fully modernized. The coexistence is not transient — it spans weeks to months, sometimes longer.

This coexistence period is operationally distinct from both the pre-modernization state (legacy alone) and the post-modernization state (modernized alone). It has its own architectural requirements that neither end-state addresses, and modernization teams that fail to design for it find themselves improvising during the most fragile phase of the migration.

## Problem

Modernization teams design the *target state* carefully but the *transition state* poorly. The architecture diagrams show the modernized system; the cutover plan covers the day of the switch; the legacy decommissioning is scheduled. The coexistence period — the weeks or months between first context cutover and last context cutover — is treated as a transitional inconvenience rather than as an architectural concern in its own right.

The consequences are operational. Data written to the legacy must reach the modernized system in near-real-time, or modernized contexts read stale state and produce wrong answers. Data written to the modernized system must propagate back to the legacy where the legacy still serves dependent contexts, or those contexts diverge. State changes must be observable across both systems for audit and debugging. Failures in either system must be detected and reconciled before they accumulate into unrecoverable divergence.

Without explicit dual-run architecture, teams improvise. They build ad-hoc data pipelines, write reconciliation scripts when production exposes drift, patch failed propagations manually. Each improvisation works in the immediate case and fails in the next case the team didn't anticipate.

The cost accumulates; trust in the modernization erodes; rollback becomes increasingly attractive because the team has lost confidence that the dual-run is correct.

The deeper failure is treating data layer migration as an afterthought. The modernization patterns above (Parts I-III) address how to migrate code; data layer migration has been the least articulated territory in modernization practice. Bringing data forward — keeping it consistent during dual-run, reconciling drift, deciding which side is authoritative for which data at which stage — deserves its own discipline.

## Forces

Both systems must remain operationally consistent during dual-run. Customers using the modernized system must see the same business state customers using the legacy system see; if Alice's account balance reads as $1,000 in the legacy and $950 in the modernized system, the modernization has lost integrity, regardless of how correctly each system processed its individual transactions.

The data layer is heterogeneous. The legacy mainframe runs DB2, VSAM files, IMS hierarchies, generation data groups, sequential files on tape. The modernized system runs PostgreSQL, SQL Server, event streams, document stores. Replication must bridge the heterogeneity without losing the semantics of either side.

The direction of data flow is not constant. During the early rollout phase, most data flows legacy → modernized (legacy is authoritative, modernized is observing). During incremental routing, both directions are active (each side is authoritative for the contexts it owns). Near cutover completion, most data flows modernized → legacy (modernized is authoritative, legacy is reference). The infrastructure must support all three regimes and transitions between them.

## Pattern

Treat the dual-run period as a first-class architectural phase with its own infrastructure. Three categories of mechanism, applied as needed per bounded context:

**Change Data Capture** (**CDC**) captures changes in the legacy data stores and replicates them into the modernized system. The mechanism varies by store: DB2 publishes change logs that can be consumed by IBM InfoSphere CDC, Oracle GoldenGate, or Debezium-style readers; VSAM changes are captured through journaling; IMS changes through DPROP or equivalent. The captured stream is transformed and applied to the modernized side's storage — sometimes 1:1, often with reshape (the legacy's denormalised tables become the modern's normalised tables, or vice versa). Latency targets depend on consistency requirements: sub-second for tightly coupled contexts; minutes for reference data; nightly batch for analytical capabilities.

CDC is one of two integration shapes available during dual-run. The alternative, which Nick Tune has called *Application-level Events*, captures changes inside the legacy application code and emits them as semantically rich domain events rather than as storage-layer change records. The trade-off is real: CDC is easier to instrument on legacy and captures every modification, but leaks the legacy's storage model into downstream consumers; application-level events carry richer context and avoid that coupling, but require intervention in legacy code. The choice is per data domain. The glossary entry *CDC vs Application-level Events* gives definitions and pointers.

**Dual-write strategies** apply when modernized writes must reflect into the legacy because the legacy still serves dependent contexts. Three patterns dominate: - *Synchronous dual-write* — the modernized code writes to both stores in the same transaction. Strong consistency, but fragile: any failure in the legacy fails the modernized transaction. - *Asynchronous dual-write through queue* — the modernized code writes locally and enqueues a propagation message; a consumer applies the change to the legacy. Eventually consistent, more resilient, but introduces propagation lag the consuming contexts must tolerate. - *Outbox-pattern dual-write* — the modernized code writes locally to its store and to an outbox table in the same transaction; a separate process drains the outbox into a queue or directly into the legacy. Reliable, transactional, well-understood; this is the recommended default.

**Reconciliation procedures** detect and remediate drift between the two sides. Reconciliation runs continuously (sampling reads on both sides and comparing) or on schedule (comparing complete tables nightly), depending on capability criticality. When drift is detected, the response is determined by ownership: if the legacy still owns the data, modernized state is corrected from legacy; if modernized owns it, legacy is corrected from modernized. Reconciliation is not just diff detection — it includes remediation playbooks for the discovered drift, and metrics that track drift rates over time.

**Bridge APIs** translate between legacy and modernized vocabularies when contexts on each side must communicate during the transition. The bridge implements anti-corruption (Evans, 2003) — it speaks the legacy's protocol on one side and the modernized's protocol on the other, preserving each context's vocabulary. As contexts cut over, the bridge's translation surface shrinks; eventually it can be retired.

The state of authority is explicit per data domain. The dual-run architecture documents, for each data domain (customers, accounts, transactions, products, ...), which side is currently authoritative and which side is replicating. The authority transitions are scheduled: customers become modernized-authoritative on date X; accounts on date Y; transactions on date Z. The schedule aligns with bounded-context cutover (Pattern 27).

Witness (Pattern 17) monitors the dual-run continuously. Output divergences are categorised: legitimate state drift (one side correctly received an update the other side missed), expected behavioural difference (the modernized system deliberately produces different output than the legacy in a specific case), or unexpected divergence (something is wrong and requires investigation). The categorisation surfaces problems early enough to remediate before cutover.

## Consequences

The dual-run period becomes manageable rather than chaotic. Data flows are designed, not improvised. Drift is detected automatically, not discovered through customer complaints. The team has confidence that the modernized system's state remains consistent with the legacy's throughout the migration.

Cutover becomes operationally safer. By the time a bounded context cuts over, its data has been flowing through CDC for weeks; reconciliation has verified consistency continuously. Cutover is the formal recognition of a state that has already been operating, not a leap of faith.

Rollback remains feasible throughout dual-run. Because data flows both ways during the transition, rolling back a context to legacy is mechanically possible — the legacy has been receiving updates

that occurred during the modernized period.

The cost is the dual-run infrastructure itself: CDC pipelines per data domain, outbox-pattern implementations in modernized code, reconciliation jobs with their own schedules and alerting, bridge APIs with their own operational concerns. For systems with dozens of data domains, the infrastructure is substantial. It amortises across the dual-run period; it is wasted if the modernization tries to skip it and improvise.

There is also a cost in latency tolerance. Dual-run introduces propagation lag — sub-second to nightly — that the modernized code must tolerate. Contexts requiring strictly consistent reads across data domains may need to wait until both sides cut over before they can rely on the modernized system fully.

Martin Kleppmann's *Designing Data-Intensive Applications* (Kleppmann, 2017) provides the canonical treatment of replication, consistency, and dual-write trade-offs. Pat Helland's work on eventual consistency (Helland, 2007) informs the consistency-model choices. CDC patterns have been in industry practice for decades. What this catalog contributes is the framing of the dual-run period as a first-class architectural phase in mainframe modernization specifically, with explicit data-authority schedules, reconciliation discipline, and bridge APIs as anti-corruption infrastructure — not as ad-hoc improvisation.

## Related patterns

Pattern 27 (*Rollout and Cutover at Bounded Context Granularity*) is the operational frame this pattern serves — dual-run is what sustains the parallel run periods that rollout requires. Pattern 17 (*Hypothesis-Driven Verification*) monitors the dual-run, detecting drift and categorising divergences. Pattern 4 (*Source Provenance Discipline*) extends through CDC — every change in the modernized system traces back to the legacy source that originated it. Pattern A (*Transactional Boundaries as First-Class Migration Concern*) determines the consistency model for dual-write per context. Pattern 12 (*Commands and Events as Logical Boundary*) provides the logical contracts that the bridge APIs translate.

---

# Antipatterns

---

**[INFOGRAPHIC] AP — *The antipatterns and their correctives***

*Style: editorial infographic, single-page format. The antipatterns arranged as panels around a central visual. Each panel: antipattern name in bold, one-sentence summary, and an arrow pointing to the corrective pattern(s) numbered. Center: a small visual representing "what the field has been doing" vs "what the patterns correct" — perhaps two side-by-side schematics, one labelled "drift," one labelled "discipline." Earth-tone palette matching the cover. Goal: reader gets a one-page reference that maps every antipattern to its corrective patterns. Useful for review at end of catalog.*

---

A pattern catalog is most useful when it names what it's built against. The antipatterns below are failure modes the field has encountered, articulated briefly. Naming them helps clarify what the patterns above are correcting.

**Jobol.** Generated C# that has been mechanically translated from COBOL but retains COBOL's structure, idioms, and shape. The result reads like COBOL written in C# syntax: deeply nested control flow, primitive obsession, monolithic procedural style. The compiler principle (Pattern 8) and tier-aware scaffolding (Pattern 10) are correctives.

**Silent semantics loss.** A modernization that produces syntactically reasonable C# but loses behavioural detail in the translation. Edge cases, error paths, transactional guarantees disappear quietly. The user doesn't know until production reveals the loss. The legacy as oracle (Pattern 2) and Twin Verification (Pattern 16) are correctives.

**False clean code.** Generated C# that follows modern idioms (dependency injection, clean architecture, async/await) without preserving the legacy's actual behaviour. The code looks like what a senior developer would write today; it just doesn't do what the legacy does. Hypothesis-driven verification (Pattern 17) is the corrective: behavioural equivalence is the standard, not aesthetic alignment with current best practice.

**Frozen architecture.** A modernization that lifts the legacy's architectural decisions into the modernized system, treating them as preserved value rather than as historical artifacts. The result is modern code with thirty-year-old architectural commitments embedded in it. In DDD terms, this is failing to perform strategic design — accepting whatever bounded contexts the legacy implicitly created, whatever subdomain types the legacy implicitly chose, whatever was once core that has since become commodity. Pluggable emitters (Pattern 11) and tier-aware scaffolding (Pattern 10, grounded in Evans' core/supporting/generic typology) make architectural decisions explicit and replaceable.

**Vendor oracle.** Trusting a single vendor's tooling as the authoritative ground truth for the modernization. When the vendor's tool produces output, the modernization treats that output as correct. The vendor becomes the oracle, displacing the legacy itself. The legacy as oracle (Pattern 2) inverts this: the legacy is the truth source, and any tool — vendor or otherwise — is verified against it.

**Behavioural equivalence without ontology.** A modernization that achieves perfect behavioural fidelity to the legacy — Twin Verification passes, Hypothesis-Driven Verification confirms, every gate in the harness is green — and yet produces a modernized system that inherits the legacy's ontological confusion. Three definitions of "active customer" survive the cut. Two interpretations of "balance" remain incompatible. Vocabulary that meant one thing in 1992 still means another in

2008, now expressed in clean modern code that makes the confusion harder to detect. The modernization is technically successful and fundamentally broken. In DDD terms, the modernization preserved tactical implementations but never established a ubiquitous language; the result is a modernized polyglot, fluent in three dialects of the same domain, mastering none. Anthony Alcaraz's argument applies with force: the orchestration is fine, the grounding is missing, the output is confidently wrong. The legacy as oracle (Pattern 2) is necessary but not sufficient; *Domain Ontology as Independent Substrate* (Pattern 5) is the corrective. Behavioural equivalence preserves what the legacy does. Ontology decides what the legacy *should have been about* — and the modernization team has to answer that question independently.

**Synchronisation antipatterns during incremental migration.** Three failure modes recur when incremental migration is overlaid on a real legacy landscape, all documented by Nick Tune. *Bi-directional Model Sync* keeps a legacy entity and a modernized entity in continuous round-trip synchronisation, with each side's invariants having to survive translation through the other's model; the arrangement is fragile and resists retirement. *Asymmetrical Validation* compounds the problem: the modernized side enforces stricter rules than the legacy, while the legacy holds years of data that does not satisfy those rules, producing chronic drift the team cannot resolve without amnesty or backfill. *Tri-directional Sync* is the combinatorial form: when the migration is overlaid on a landscape that already contains multiple legacy systems with overlapping data, the team must build and operate not two synchronisation flows but six, twelve, or more. The corrective is structural: define data authority per domain on an explicit schedule (Pattern 28), use anti-corruption layers (Pattern 12) to translate vocabularies rather than synchronise models, and resist the instinct to keep all sides equally authoritative throughout the transition. Definitions and pointers are in the glossary.

**Agent army.** An agentic system that scales by multiplying agents — more specialised agents, more coordination layers, hierarchies, swarms, agentic mesh. The instinct is *if one agent is good, many agents are better*. The operational failure is twofold. First, coordination overhead grows faster than capability — the system spends more compute on agents talking to each other than on doing the work. Second, the army generates vast volumes of output: documentation, code, alternatives, justifications, reasoning records. The volume saturates the reviewers downstream; humans cannot consume at the rate the agents produce. The modernization stalls at the bottleneck of cognitive load, not at any technical limit. The corrective is structural: build a harness (Pattern 23), not an army. The discipline of deterministic constraints, verifiable invariants, and explicit gates produces tractable modernization; more agents do not. Spec deltas (Pattern 26) operationalise the volume discipline at the review surface — fewer artifacts, denser articulation. *More is not better. Less is better.*

---

**[TABLE] AP.1 — *Antipatterns and their corrective patterns***

*Two-column reference table. Left column: antipattern name and brief summary (one phrase). Right column: corrective pattern numbers and names with brief explanation of how each pattern protects against the antipattern. Rows: Jobol → Pattern 8, 10; Silent semantics loss → Pattern 2, 16; False clean code → Pattern 17; Frozen architecture → Pattern 10, 11; Vendor oracle → Pattern 2; Behavioural equivalence without ontology → Pattern 2, 5; Synchronisation antipatterns during incremental migration → Pattern 12, 28; Agent army → Pattern 23, 26. Style: clean reference table with row banding. Goal: when reviewer encounters an antipattern in practice, they can find the patterns that correct it without re-reading the catalog.*

---

---

# Glossary

Definitions of the DDD and modernization terms used throughout the catalog. Where a concept has been more thoroughly developed in canonical sources, the definition includes a pointer for further reading.

**Agent execution failure modes.** Three characteristic patterns of agent misbehaviour during execution, named by Uberto Barbini at the individual-developer scale and recurring at platform scale. *Loop of death*: the agent fixes one issue and breaks another, oscillating indefinitely. *Misunderstanding the requirement*: the agent works on the wrong part of the code, often because of poor naming or duplicated logic — a problem that gets worse, not better, with AI, which makes code quality and DDD discipline more important rather than less. *Desperate changes*: when the agent cannot find a solution within the current constraints, it escalates — rewriting large parts of the system, ignoring design rules, even modifying external libraries. The corrective in all three cases is structural: detect the failure mode (cycle detection, scaffold-boundary violation, invariant violation) and intervene, rather than letting the agent continue. *See Uberto Barbini,* Process Over Magic: Beyond Vibe Coding* (2026); related to Pattern 23 (The Harness as Self-Observing State Machine) and Pattern 22 (Heuristics as Explicit Artifacts).*

**Aggregate.** In DDD tactical design, a cluster of domain objects (entities and value objects) treated as a single unit for the purposes of data changes. The aggregate has a root that mediates external access; invariants on the aggregate hold across the cluster. *See Evans 2003, Chapter 6; Vernon 2013, Chapter 10.*

**Anti-corruption layer.** A translation layer between two bounded contexts (or between a modernized system and a legacy system) that prevents concepts from one polluting the model of the other. The anti-corruption layer is itself a bounded context, with the responsibility of speaking both vocabularies and translating between them. *See Evans 2003, Chapter 14.*

**Asymmetrical Validation.** A migration antipattern in which the modernized system enforces stricter validation than the legacy, while the legacy holds years of data that does not satisfy those rules. Synchronisation between the two sides repeatedly fails on data the legacy considers valid and the modernized side rejects, producing chronic drift and operational burden. *See Nick Tune, legacy-modernization.io; related to the* Behavioural Equivalence Without Ontology* antipattern.*

**Autonomous Bubble.** A variant of the Bubble pattern in which the new subsystem holds its own local data store, populated asynchronously from legacy events through an anti-corruption layer. Unlike a pure Bubble, the autonomous bubble can fulfil reads without calling back into the legacy, which decouples its runtime from legacy availability and performance. *See Nick Tune, legacy-modernization.io; related to Pattern 14 (Transitional Architecture) and Pattern 12 (Commands and Events as Logical Boundary).*

**Bi-directional Model Sync.** A migration antipattern in which a legacy entity and a modernized entity are kept in continuous synchronisation across both directions of writes. The arrangement is fragile because each side's invariants must round-trip through the other's model; semantic mismatches surface as recurrent drift, and the synchronisation becomes load-bearing infrastructure that resists retirement. *See Nick Tune, legacy-modernization.io; related to the* Behavioural Equivalence Without Ontology* antipattern.*

**Bounded context.** An explicit boundary within which a particular domain model applies. Inside the boundary, terms have a single meaning; across boundaries, the same term may mean different things. The bounded context is the central unit of strategic design in DDD. *See Evans 2003, Chapter 14; Martin Fowler's BoundedContext entry.*

**Bubble.** A migration pattern in which a new subsystem is built with a clean target domain model and accesses legacy data exclusively through an anti-corruption layer, without local persistence and without synchronisation. The bubble eventually "pops" when the legacy data store is retired. Useful when the target model can be designed independently but the underlying data must remain in the legacy during the transition. *See Nick Tune, legacy-modernization.io; related to Pattern 14* (*Transitional Architecture*).

**CDC vs Application-level Events.** A migration design decision between capturing data changes at the storage layer (Change Data Capture) versus emitting domain events from within the legacy application code. CDC is easier to instrument on legacy systems and captures every modification, but it leaks the legacy's storage model into downstream consumers. Application-level events carry richer semantic context and avoid that coupling, but require intervention in legacy code. *See Nick Tune, legacy-modernization.io; related to Pattern 28* (*Dual-Run Coexistence*) *and Pattern 12* (*Commands and Events as Logical Boundary*).

**Context map.** A representation of the relationships between bounded contexts in a system — which contexts share concepts, which translate between each other, which are upstream or downstream of others. *See Evans 2003, Chapter 14; Vernon 2013, Chapter 3.*

**Core domain.** The subdomain that differentiates the business — the part of the system that gives the organisation its competitive advantage. Strategic design focuses architectural investment on the core domain. *See Evans 2003, Chapter 15.*

**Distillation.** The strategic design activity of separating the core domain from supporting and generic subdomains, articulating the canonical concepts and language of the core, and protecting it from being absorbed into commodity concerns. *See Evans 2003, Chapter 15.*

**Domain event.** A statement about something significant that happened in the domain, modelled as a first-class object. Domain events are immutable, named in past tense, and form the basis of event-driven architectures within DDD. *See Vernon 2013, Chapter 8.*

**Drifting Domain Model.** The phenomenon in which the target domain model evolves *during* the migration itself, as the team's understanding sharpens through contact with the legacy and with domain experts. Concepts that started as one-to-one renames (Employee → Collaborator) end up restructured (Collaborator with one Profile and many Working Agreements, of which Contract is one subtype). The migration is not a translation; it is a re-articulation. *See Nick Tune, legacy-modernization.io; related to Pattern 5* (*Domain Ontology as Independent Substrate*).

**Event Storming.** A workshop technique developed by Alberto Brandolini for collaboratively recovering domain understanding through orange sticky notes representing domain events arranged on a timeline. Particularly useful for legacy archaeology and for establishing ubiquitous language with domain experts. *See Brandolini,* Introducing EventStorming.

**Expose Legacy Asset.** A migration pattern in which functionality or data inside the legacy is made available to modernized subsystems through an explicit interface — typically a synchronous API or a stream of domain events — rather than through direct database access or in-process coupling. The interface serves as a stable contract while the legacy itself remains in place. *See Nick Tune, legacy-modernization.io; related to Pattern 19* (*Bounded MCP Servers*) *and Pattern 12* (*Commands and Events as Logical Boundary*).

**Generic subdomain.** A subdomain that the business needs but does not differentiate on — typically capabilities that can be bought or built off-the-shelf (logging, authentication, reference data).

Strategic design favours minimal investment in generic subdomains. *See Evans 2003, Chapter 15.*

**Hexagonal architecture.** A pattern (also called ports and adapters) in which the domain logic sits at the centre, surrounded by adapter layers that translate between the domain and external concerns (databases, APIs, UIs). Used in this catalog for tier-3 strategic core scaffolds. *See Alistair Cockburn's hexagonal architecture writings.*

**Migrate Reads First / Migrate Writes First.** Two opposite ordering strategies within an incremental migration of a single capability. *Reads first* migrates query paths to the modernized system while writes continue against the legacy, typically using events or CDC to keep the modernized read model fresh; it unblocks downstream consumers quickly. *Writes first* migrates the write path to the modernized system while reads continue against the legacy until the read model is ready; it unblocks new use cases that depend on the new write model. The choice is strategic, not technical: it depends on which downstream pressure dominates. *See Nick Tune, legacy-modernization.io; related to Pattern 27 (Rollout and Cutover at Bounded Context Granularity).*

**Republishing Legacy Events.** A migration technique in which an autonomous bubble (or equivalent transitional subsystem) consumes events from the legacy and re-emits them in the target domain vocabulary, allowing downstream consumers to decouple from the legacy model *before* the migration is complete. Consumers integrate against the canonical events from the start; the bubble bridges the vocabulary gap during transition. *See Nick Tune, legacy-modernization.io; related to Pattern 12 (Commands and Events as Logical Boundary).*

**Strategic design.** The level of DDD that addresses bounded contexts, subdomain types, context maps, and the distribution of architectural attention across the system. Strategic design happens at the system level. *See Evans 2003, Part IV.*

**Subdomain types (core / supporting / generic).** Evans' typology for classifying parts of the domain by their relationship to business differentiation. Core subdomains differentiate; supporting subdomains support the core; generic subdomains are needed but undifferentiated. The typology drives architectural investment decisions. *See Evans 2003, Chapter 15.*

**Supporting subdomain.** A subdomain that supports the core but does not itself differentiate the business. Strategic design treats supporting subdomains as candidates for moderate investment — enough structure to evolve, not enough to over-engineer. *See Evans 2003, Chapter 15.*

**Tactical design.** The level of DDD that addresses aggregates, entities, value objects, domain events, repositories, services, and how each bounded context's domain model is structured internally. Tactical design happens at the bounded context level. *See Evans 2003, Parts I–III; Vernon 2013.*

**Tri-directional Sync.** A migration antipattern that emerges when an incremental migration is overlaid on a landscape that already contains multiple legacy systems holding overlapping data. The "two-way sync" framing collapses: with three systems the team must build and operate six synchronisation flows; with four, twelve. The combinatorial cost is not a design failure but a mathematical consequence of incremental migration with pre-existing legacy plurality. *See Nick Tune, legacy-modernization.io; related to the* Behavioural Equivalence Without Ontology* antipattern.*

**Ubiquitous language.** The shared vocabulary of a domain, used consistently by domain experts, developers, and the system itself (in code, in conversations, in documentation). The ubiquitous language is established within a bounded context — the same term may have different ubiquitous languages in different contexts. *See Evans 2003, Chapter 2.*

**Vertical slice.** A unit of work that crosses multiple architectural layers to deliver a single user-visible capability. In modernization, the vertical slice typically corresponds to a use case within a bounded context, framed by the aggregates it touches. The slice is the unit of agentic translation in this catalog.

**For broader DDD reference**, accessible entry points include Vladik Khononov's *Learning Domain-Driven Design* (2021), the DDD Crew GitHub repository, and the Bounded Context Canvas by Nick Tune and Krisztina Hirth. For deeper engagement, Eric Evans' original *Domain-Driven Design* (2003) and Vaughn Vernon's *Implementing Domain-Driven Design* (2013) remain the canonical sources.

# Reference implementations in Rosetta

The patterns above are written abstractly because principles outlive implementations. This section names the concrete technologies that realise each pattern in Rosetta today. It's a snapshot — as of early 2026 — and will date faster than the patterns themselves. When a technology changes, this section updates; the pattern bodies stay stable.

Only patterns with concrete implementations today appear here. Patterns marked *next* are designed but not yet built, so they have no reference implementation to record.

**Pattern 2 — The Legacy as Oracle** - Legacy compiler: Raincode (compiles COBOL to .NET IL) - Container runtime: Docker - Local execution: developer workstation, in-process invocation

**Pattern 3 — The Graph as Projection** - Graph database: Neo4j - Schema: COBOL/CICS-specific property graph (programs, paragraphs, data structures, control flow, side effects, predicates, entry points) - Query language: Cypher - Ingestion: custom COBOL/CICS parser feeding the graph schema

**Pattern 4 — Source Provenance Discipline** - Provenance fields: `source_file`, `start_line`, `end_line` on every node - Carried through: graph nodes, IR elements, generated C# (as comments and metadata)

**Pattern 6 — The Graph and the Index as Complementary Substrates** - Graph: Neo4j (Pattern 3) - Semantic index: Azure AI Search - Embedding granularity: paragraph-level and program-level - Vocabulary inference from comments, display literals, naming conventions, IR - Synchronization: shared ingestion pipeline updates both substrates from the same source events

*Pattern 5 (Domain Ontology as Independent Substrate) is status next; vocabulary inference and similarity clustering from Pattern 6 are starting points but the ontology substrate itself is not yet built.*

**Pattern 7 — Vertical Slice Discovery from Structural and Behavioural Signals** - Structural analysis: Cypher queries over the Neo4j graph - Behavioural signal (when available): observation telemetry from the Witness production layer - CICS-specific anchors: `RETURN TRANSID/COMMAREA` cycles as primary slice boundary signal

**Pattern 8 — The Compiler Principle** - Deterministic emitter: Roslyn SyntaxFactory - Validation layer: architect review through Rosetta Studio (Pattern 24) - Probabilistic layer: GitHub Copilot SDK-driven agents inside the rendered scaffold

**Pattern 9 — The Intermediate Representation** - IR name: WolverineIntentModel - Schema: typed C# classes representing commands, events, handlers, sagas, aggregates, side-effect surfaces - Grounding: each IR element references the graph nodes that derived it

**Pattern 10 — Tier-Aware Scaffolding** - Tier annotation: stored on bounded context nodes in the graph - Scaffold variants: VSA emitter (tier 0–1), hybrid emitter (tier 2), hexagonal emitter (tier 3) - Heuristic derivation: cyclomatic complexity, coupling, change frequency

**Pattern 11 — Pluggable Emitters** - Current emitters: vertical slice (Wolverine), hexagonal (Wolverine with ports/adapters), hybrid (Wolverine with lightweight domain models) - Future targets under consideration: Jade (event-sourced workloads), Java (Spring Boot or Quarkus emitter)

**Pattern 12 — Commands and Events as Logical Boundary** - Framework: Wolverine for current implementations - Transactional guarantees: Wolverine transactional outbox preserves CICS-equivalent consistency semantics - In-process default: Wolverine handles dispatching synchronously when contexts share a process - Distributed extension: same command/event surface flows over messaging when contexts are extracted to services

**Pattern 16 — Twin Verification** - Local oracle: Raincode-compiled COBOL packaged as Docker container (Legacy Twin) - Comparison: in-process semantic comparison of outputs against the Legacy Twin - Test framework: xUnit - Inner loop: candidate generation → dual execution → semantic comparison → verdict in milliseconds

**Pattern 17 — Hypothesis-Driven Verification** - Framework: Witness (production-mode counterpart to Twin Verification) - Capture-replay lineage: pmilet/playback (open-source HTTP capture-replay middleware, 2016) - MCP integration: Witness MCP Server owns the evidence lifecycle

**Pattern 18 — Behavioural Specifications Grown from Production** - Status: designed, not yet built; planned as extension of Witness pattern-matching layer

**Pattern 19 — Bounded MCP Servers** - Protocol: Model Context Protocol (MCP) - SDK: MCP SDK for .NET - Server count in current architecture: four (Discovery, Legacy, Twin, Witness)

**Pattern 20 — The Orchestration Layer Above Bounded Capabilities** - Orchestrating agent: Cortex (Rosetta-internal name) - Framework: Microsoft Agent Framework (MAF) - Retrospective layer: dedicated retrospective agent that learns across sessions

**Pattern 21 — Durable Agentic Workflows** - Durable workflow infrastructure: Azure Durable Functions - Hosted agentic platform: Azure AI Foundry - GitHub-native gates: branch protection, required status checks, GitHub Actions, issue templates - Persistence layer for state, replay semantics for failed steps

**Pattern 23 — The Harness as State Machine** - State machine implementation: typed C# (Microsoft Agent Framework) - Hooks: PreToolUse and PostToolUse hooks defined in code - Per-project contract: scaffold-meta.json - GitHub-native enforcement: branch protection rules, required status checks, GitHub Actions

**Pattern 20 — Harness Self-Observation and Refinement** - Status: designed, not yet built

**Pattern 24 — Reasoning Telemetry as First-Class Output** - Telemetry schema: structured records alongside each agentic decision - Storage: queryable through standard observability infrastructure - Status: in active development as part of the broader observability work

**Pattern 25 — The Cockpit** - Implementation: Rosetta Studio - Surfaces: graph decisions, translation candidates, gate transitions, spec deltas, audit trail, diff views

**Pattern 26 — Spec Deltas as the Unit of Review** - Specification format: OpenSpec (or equivalent structured format) - Status: designed, not yet built; planned to integrate with the harness gate model

---

# Closing

Twenty-eight patterns and seven antipatterns. The catalog isn't comprehensive — it's calibrated to what Project Rosetta has surfaced over a year of building. Other patterns exist in the field; some I haven't encountered yet, some I've encountered but not yet articulated, some are being articulated by others doing parallel work.

The patterns above are reports from inside an experiment. Patterns marked *working* have been validated inside the Rosetta prototype. None has yet been applied to a real customer engagement — that's the next phase. Patterns marked *in progress* are being built. Patterns marked *next* are designed but not yet built — proposals based on validated principles, not commitments to construction.

I expect the catalog to evolve. Some patterns will sharpen with use. Some will turn out to be specific to CICS COBOL and not generalise. Some will be replaced by patterns I haven't yet found. The catalog as it stands is a snapshot of what the experiment has taught me at this moment.

Lineage. The Gang of Four's *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994) established the form this catalog follows. Eric Evans' *Domain-Driven Design* (2003) is the discipline this catalog applies to a territory where DDD is rarely attempted; Vaughn Vernon's *Implementing Domain-Driven Design* (2013) is the tactical reference that informs how the IR encodes aggregates and events. Michael Feathers' *Working Effectively with Legacy Code* is the foundation Pattern 2 stands on. Kent Beck's Exploristan framing is the methodological frame for the whole prototype. Alberto Brandolini's Event Storming is the workshop technique that complements Pattern 5's ontology recovery. Specific patterns owe debts to specific people — Charity Majors, Nick Tune, Jeremy Miller, Birgitta Böckeler, Anthony Alcaraz, Uberto Barbini — named in the body of the patterns where their contributions actually shaped the work. The catalog accumulates from work done in the open by many practitioners across the DDD and modernization communities; my contribution is organising what has helped me into a vocabulary that may help others bridge the two.

If you're a DDD practitioner curious about how the discipline scales when AI assistance meets blackfield mainframe modernization, I'd be interested to hear which patterns resonate, which feel forced, what's missing. If you're a mainframe modernization practitioner less familiar with DDD, I'd be interested to hear whether the framing helps clarify what the work actually is, or whether it adds vocabulary without adding clarity. The catalog improves through use.

— Pierre